HỌC VIỆN CÔNG NGHỆ BƯU CHÍNH VIỄN THÔNG

KHOA CÔNG NGHỆ THÔNG TIN 1

BÁO CÁO CUỐI KHÓA
HỆ THỐNG THÔNG MINH

TỪ HỌC MÁY ĐẾN HỆ ĐA TÁC TỬ:
XÂY DỰNG HỆ THỐNG THÔNG MINH
HỖ TRỢ SÀNG LỌC ĐÁI THÁO ĐƯỜNG

(Mở đầu – Chương 7)

Giảng viên hướng dẫn:	PGS. TS. Trần Đình Quế
Sinh viên thực hiện:	Nguyễn Tự Kiên
Mã sinh viên:	B23DCCN465
Nhóm lớp:	CT01
Nhóm tiểu luận:	07

Hà Nội, tháng 11 năm 2026

Mục lục

DANH MỤC TỪ VIẾT TẮT . . . . . **vi**

MỞ ĐẦU **1**

CHƯƠNG 1. Lịch sử phát triển của trí tuệ nhân tạo **5**
1.1. Khái niệm trí tuệ nhân tạo 5
1.2. Các giai đoạn phát triển 5
1.2.1. Nền tảng tiền đề (trước 1956) 5
1.2.2. Khai sinh và thời kỳ lạc quan (1956 – đầu thập niên 1970) . . . 6
1.2.3. Mùa đông AI thứ nhất (khoảng 1974 – 1980) 6
1.2.4. Hệ chuyên gia và mùa đông AI thứ hai (1980 – đầu thập niên 1990) 6
1.2.5. Học máy thống kê (thập niên 1990 – 2000) 7
1.2.6. Kỷ nguyên học sâu (2006 – 2019) 7
1.2.7. Mô hình nền tảng, AI tạo sinh và tác tử (2020 – nay) 7
1.3. Thực nghiệm: bốn thế hệ mô hình trên bộ Pima 8
1.3.1. Dữ liệu 8
1.3.2. Mô hình 10
1.3.3. Kết quả và nhận xét 11
1.4. Triển khai 13
1.5. Tiểu kết chương 1 13

CHƯƠNG 2. Các kỹ thuật học máy cơ bản **14**
2.1. Bài toán học và quy trình xây dựng mô hình 14
2.2. Tối ưu hóa bằng hạ gradient 14
2.3. Các thuật toán 15
2.3.1. Hồi quy tuyến tính 15
2.3.2. Hồi quy logistic 15
2.3.3. K láng giềng gần nhất 15
2.3.4. Naive Bayes 16
2.3.5. Cây quyết định và các phương pháp tổ hợp 16
2.3.6. Máy vector hỗ trợ 16
2.3.7. Gom cụm K-means và giảm chiều PCA 17
2.3.8. Quá khớp, thiếu khớp và điều chuẩn 17
2.3.9. Thước đo đánh giá 17

2.4. Dữ liệu NHANES 2011–2018 18
2.5. Cài đặt 20
2.6. Kết quả và nhận xét 21
2.6.1. Phân loại 21
2.6.2. Hồi quy HbA1c 24
2.6.3. Gom cụm không giám sát 25
2.6.4. So sánh tổng hợp 26
2.7. Triển khai 26
2.8. Tiểu kết chương 2 27

CHƯƠNG 3. Mạng nơ-ron tích chập (CNN) 28
3.1. Động lực của CNN 28
3.2. Các lớp của CNN 28
3.2.1. Lớp tích chập 28
3.2.2. Hàm kích hoạt 29
3.2.3. Lớp gộp 29
3.2.4. Chuẩn hóa theo lô 30
3.2.5. Dropout, lớp kết nối đầy đủ và softmax 30
3.2.6. Hàm mất mát và huấn luyện 30
3.2.7. Tăng cường dữ liệu và học chuyển giao 30
3.2.8. Các kiến trúc tiêu biểu 31
3.2.9. Giải thích bằng Grad-CAM 31
3.3. Dữ liệu RetinaMNIST 32
3.4. Cài đặt 33
3.5. Kết quả và nhận xét 35
3.6. Triển khai 38
3.7. Tiểu kết chương 3 39

CHƯƠNG 4. Mạng nơ-ron hồi quy (RNN) 40
4.1. Dữ liệu tuần tự và kiến trúc RNN 40
4.2. Lan truyền ngược theo thời gian và vấn đề biến mất gradient 40
4.3. LSTM và GRU 41
4.3.1. Bộ nhớ dài – ngắn hạn (LSTM) 41
4.3.2. Đơn vị hồi quy có cổng (GRU) 42
4.3.3. Các mở rộng 42
4.3.4. Đánh giá mô hình tuần tự và các bẫy thường gặp 42
4.3.5. RNN và Transformer cho chuỗi thời gian 43
4.4. Dữ liệu ShanghaiT1DM / ShanghaiT2DM 43
4.5. Cài đặt 44
4.6. Kết quả và nhận xét 46
4.6.1. Phân tích sai số theo dải đường huyết 48
4.7. Triển khai 49

4.8. Tiểu kết chương 4 50

CHƯƠNG 5. Đồ thị tri thức và sinh văn bản tăng cường truy xuất 51
5.1. Đồ thị tri thức 51
5.1.1. Định nghĩa và mô hình dữ liệu 51
5.1.2. Xây dựng đồ thị tri thức 52
5.1.3. Suy luận và nhúng đồ thị tri thức 52
5.2. Sinh văn bản tăng cường truy xuất 53
5.2.1. Nguyên lý 53
5.2.2. Quy trình 53
5.2.3. GraphRAG 54
5.2.4. So sánh RAG và tinh chỉnh 55
5.2.5. Thách thức vận hành 55
5.2.6. Đánh giá hệ thống RAG 56
5.3. Dữ liệu 56
5.3.1. Knowledge Base tiếng Việt và đồ thị tri thức 56
5.3.2. MedQuAD — phần NIDDK 57
5.4. Thực nghiệm 58
5.4.1. Thiết lập 58
5.4.2. Kết quả truy xuất trên KB tiếng Việt 60
5.4.3. Kết quả truy xuất trên MedQuAD 62
5.4.4. Kết quả sinh câu trả lời 63
5.4.5. Ví dụ: một câu hỏi đa bước qua ba chế độ 64
5.5. Triển khai 65
5.6. Tiểu kết chương 5 65

CHƯƠNG 6. Học sâu trên đồ thị 67
6.1. Dữ liệu đồ thị và các bài toán 67
6.2. Mạng nơ-ron đồ thị 67
6.2.1. Khung truyền thông điệp 67
6.2.2. Nền tảng phổ: từ biến đổi Fourier trên đồ thị đến GCN 68
6.2.3. Mạng tích chập đồ thị (GCN) 68
6.2.4. GraphSAGE 70
6.2.5. Mạng chú ý trên đồ thị (GAT) 70
6.2.6. Làm trơn quá mức 70
6.2.7. Năng lực biểu diễn và phép thử Weisfeiler–Lehman 71
6.2.8. Các bài toán khác và ứng dụng y sinh 71
6.3. Dữ liệu Pubmed-Diabetes 72
6.4. Cài đặt 73
6.5. Kết quả và nhận xét 75
6.5.1. Ablation các lựa chọn thiết kế của GCN 78
6.5.2. Học chuyển dẫn và học quy nạp 78

6.5.3. Chi phí tính toán . . . 79
6.5.4. Phân tích lỗi và trọng số chú ý của mô hình triển khai . . . 80
6.5.5. Hạn chế . . . 81
6.6. Triển khai . . . 81
6.7. Tiểu kết chương 6 . . . 81

CHƯƠNG 7. Hệ đa tác tử . . . 83
7.1. Tác tử và hệ đa tác tử . . . 83
7.1.1. Khái niệm tác tử . . . 83
7.1.2. Hệ đa tác tử . . . 83
7.1.3. Tác tử dựa trên mô hình ngôn ngữ lớn . . . 84
7.2. Kiến trúc hệ thống . . . 85
7.2.1. Tám tác tử . . . 85
7.2.2. Giao tiếp qua MessageBus . . . 85
7.2.3. Luồng xử lý một lượt chat . . . 87
7.2.4. Giao thức tranh biện . . . 87
7.2.5. Tác tử gợi ý can thiệp bằng Q-learning . . . 88
7.2.6. Bộ nhớ, nguồn gốc dữ liệu và chịu lỗi . . . 89
7.2.7. So sánh với các framework đa tác tử . . . 90
7.3. Đánh giá hệ đa tác tử . . . 91
7.3.1. Dữ liệu và kịch bản . . . 91
7.3.2. K1 — tính đúng đắn và vận hành . . . 92
7.3.3. K2 — giá trị của việc phối hợp nhiều tác tử . . . 94
7.3.4. K3 — từ tin nhắn tự do . . . 95
7.3.5. Ví dụ một lượt hội chẩn có tranh biện hai vòng . . . 95
7.4. Triển khai . . . 97
7.5. Tiểu kết chương 7 . . . 99

KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN . . . 100

TÀI LIỆU THAM KHẢO . . . 103

DANH MỤC TỪ VIẾT TẮT

Viết tắt	Tiếng Anh	Tiếng Việt
ADA	American Diabetes Association	Hiệp hội Đái tháo đường Hoa Kỳ
AGE	A Graph Extension (Apache AGE)	Phần mở rộng đồ thị cho PostgreSQL
AI	Artificial Intelligence	Trí tuệ nhân tạo
API	Application Programming Interface	Giao diện lập trình ứng dụng
AUC	Area Under the (ROC) Curve	Diện tích dưới đường ROC
BM25	Best Matching 25	Hàm xếp hạng BM25
BMI	Body Mass Index	Chỉ số khối cơ thể
BN	Batch Normalization	Chuẩn hóa theo lô
BPTT	Backpropagation Through Time	Lan truyền ngược theo thời gian
CF	Certainty Factor	Hệ số chắc chắn
CGM	Continuous Glucose Monitoring	Theo dõi glucose liên tục
CNN	Convolutional Neural Network	Mạng nơ-ron tích chập
CV	Coefficient of Variation	Hệ số biến thiên
DL	Deep Learning	Học sâu
DR	Diabetic Retinopathy	Bệnh võng mạc đái tháo đường
ĐTĐ	–	Đái tháo đường
FIPA	Foundation for Intelligent Physical Agents	Tổ chức chuẩn hóa tác tử FIPA
FPG	Fasting Plasma Glucose	Glucose huyết tương lúc đói
GAT	Graph Attention Network	Mạng chú ý trên đồ thị
GBDT	Gradient Boosted Decision Trees	Cây quyết định tăng cường gradient
GCN	Graph Convolutional Network	Mạng tích chập đồ thị
GNN	Graph Neural Network	Mạng nơ-ron đồ thị
GRU	Gated Recurrent Unit	Đơn vị hồi quy có cổng
HbA1c	Glycated Hemoglobin	Huyết sắc tố gắn glucose
HNSW	Hierarchical Navigable Small World	Đồ thị thế giới nhỏ phân cấp
ICDR	International Clinical Diabetic Retinopathy scale	Thang phân độ lâm sàng quốc tế bệnh võng mạc ĐTĐ
KB	Knowledge Base	Cơ sở tri thức
KG	Knowledge Graph	Đồ thị tri thức
KNN	K-Nearest Neighbors	K láng giềng gần nhất
LLM	Large Language Model	Mô hình ngôn ngữ lớn
LSTM	Long Short-Term Memory	Bộ nhớ dài – ngắn hạn

Viết tắt	Tiếng Anh	Tiếng Việt
MAD	Mean Average Distance	Khoảng cách trung bình giữa các biểu diễn
MAE	Mean Absolute Error	Sai số tuyệt đối trung bình
MARD	Mean Absolute Relative Difference	Sai lệch tương đối tuyệt đối trung bình
MAS	Multi-Agent System	Hệ đa tác tử
ML	Machine Learning	Học máy
MLP	Multilayer Perceptron	Perceptron nhiều lớp
MRR	Mean Reciprocal Rank	Trung bình nghịch đảo thứ hạng
NHANES	National Health and Nutrition Examination Survey	Khảo sát Sức khỏe và Dinh dưỡng Quốc gia Hoa Kỳ
nDCG	Normalized Discounted Cumulative Gain	Lợi ích tích lũy chiết khấu chuẩn hóa
OGTT	Oral Glucose Tolerance Test	Nghiệm pháp dung nạp glucose đường uống
PCA	Principal Component Analysis	Phân tích thành phần chính
PLA	Perceptron Learning Algorithm	Thuật toán học perceptron
PR-AUC	Area Under the Precision–Recall Curve	Diện tích dưới đường Precision–Recall
QWK	Quadratic Weighted Kappa	Hệ số Kappa có trọng số bậc hai
RAG	Retrieval-Augmented Generation	Sinh văn bản tăng cường truy xuất
ReLU	Rectified Linear Unit	Hàm kích hoạt tuyến tính chỉnh lưu
RF	Random Forest	Rừng ngẫu nhiên
RL	Reinforcement Learning	Học tăng cường
RMSE	Root Mean Squared Error	Căn bậc hai sai số bình phương trung bình
RNN	Recurrent Neural Network	Mạng nơ-ron hồi quy
ROC	Receiver Operating Characteristic	Đường cong đặc trưng hoạt động
RRF	Reciprocal Rank Fusion	Hợp nhất thứ hạng nghịch đảo
SGD	Stochastic Gradient Descent	Hạ gradient ngẫu nhiên
SVM	Support Vector Machine	Máy vector hỗ trợ
SVR	Support Vector Regression	Hồi quy vector hỗ trợ
TBR / TIR / TAR	Time Below / In / Above Range	Thời gian dưới / trong / trên khoảng mục tiêu
TF-IDF	Term Frequency – Inverse Document Frequency	Tần suất từ – nghịch đảo tần suất văn bản

Danh sách hình vẽ

Hình MĐ.1. Vị trí các chương trong kiến trúc hệ thống chatbot đa tác tử . . . 3

Hình 1.1. Phân bố tám đặc trưng của bộ Pima theo nhãn (mật độ chuẩn hóa) 9
Hình 1.2. So sánh sáu mô hình đại diện các giai đoạn lịch sử trên Pima . . 12

Hình 2.1. Tỉ lệ ĐTĐ theo tuổi, phân bố BMI theo nhãn và tỉ lệ thiếu dữ liệu 19
Hình 2.2. Ma trận tương quan Pearson giữa các đặc trưng và nhãn 19
Hình 2.3. Đường ROC và Precision–Recall của tám mô hình trên tập kiểm tra 22
Hình 2.4. Đánh đổi độ chệch – phương sai: AUC train và validation theo k của KNN và độ sâu cây 23
Hình 2.5. Ba tầng đầu của cây quyết định: câu hỏi đầu tiên là tuổi, sau đó là tuổi (nhánh trẻ) và tiền sử gia đình (nhánh lớn tuổi) 23
Hình 2.6. Độ quan trọng hoán vị của các đặc trưng (Gradient Boosting) và ma trận nhầm lẫn 23
Hình 2.7. HbA1c dự đoán so với thực tế: mô hình co về trung bình 24
Hình 2.8. Chọn k bằng inertia và silhouette; bốn cụm chiếu trên hai thành phần chính 25

Hình 3.1. Ba ảnh mẫu của mỗi mức DR trong RetinaMNIST 32
Hình 3.2. Phân bố nhãn theo tập: lớp 0 chiếm 45%, lớp 4 chỉ 5% 33
Hình 3.3. Loss huấn luyện, accuracy và QWK trên validation theo epoch . 36
Hình 3.4. Ma trận nhầm lẫn của MobileNetV3-L trên tập kiểm tra (số ảnh và tỉ lệ theo hàng) 37
Hình 3.5. Bản đồ đặc trưng sau khối 1 (cạnh, độ sáng) và khối 3 (vùng trừu tượng) của CNN tự thiết kế 37
Hình 3.6. Grad-CAM trên tầng `layer4` của ResNet-18 với một ảnh của mỗi mức DR 38

Hình 4.1. Phân bố glucose theo loại ĐTĐ, trung vị theo giờ trong ngày và một bản ghi típ 1 trong 4 ngày 44
Hình 4.2. RMSE theo tầm dự báo và lưới sai số Clarke của LSTM ở tầm 30 phút 46
Hình 4.3. Dự báo trước 30 phút trên 24 giờ liên tục của một bệnh nhân thuộc tập kiểm tra 47
Hình 4.4. Độ lớn gradient của dự báo theo từng điểm đầu vào trên chuỗi dài 48 bước (chuẩn hóa theo bước cuối) 48

Hình 5.1. Hai pha của RAG: nạp tri thức (trên) và trả lời câu hỏi (dưới) . . 53

Hình 5.2. Đồ thị tri thức ĐTĐ: 33 nút thuộc 6 loại, 40 cạnh thuộc 7 loại quan hệ (kích thước nút tỉ lệ với bậc) 57
Hình 5.3. Độ dài đoạn của KB tiếng Việt, độ dài câu trả lời và phân bố loại câu hỏi của MedQuAD-NIDDK 58
Hình 5.4. So sánh các bộ truy xuất trên KB tiếng Việt (trái) và MedQuAD-NIDDK (phải) 61

Hình 6.1. Phân bố nhãn, phân bố bậc (thang log–log) và số từ khác 0 của mỗi bài báo 73
Hình 6.2. Accuracy theo số nhãn huấn luyện mỗi lớp: đồ thị có lợi nhất khi nhãn khan hiếm 76
Hình 6.3. t-SNE của 3.000 bài báo: đặc trưng TF-IDF gốc (trái) và biểu diễn ẩn của GCN (phải) 76
Hình 6.4. Làm trơn quá mức: accuracy và MAD của GCN theo số tầng . . 77
Hình 6.5. Loss huấn luyện và accuracy validation theo epoch (20 nhãn/lớp, một phép chia) 77

Hình 7.1. Luồng xử lý một lượt chat: Orchestrator rẽ nhánh theo ý định và mức đủ dữ liệu 87
Hình 7.2. Thời gian xử lý của từng tác tử và tần suất các loại chất vấn trong K1 93
Hình 7.3. K2: độ nhạy, độ đặc hiệu, F1 của các cách ra quyết định, và tỉ lệ ĐTĐ thật theo mức nguy cơ của hội đồng 94
Hình 7.4. Kiến trúc triển khai qua Cloudflare Tunnel 98

Danh sách bảng

Bảng MĐ.1. Bảy bộ dữ liệu dùng trong báo cáo 2

Bảng 1.1. Tóm tắt các giai đoạn phát triển chính của trí tuệ nhân tạo 8
Bảng 1.2. Năm mẫu đầu tiên của bộ Pima (số 0 ở cột sinh lý là giá trị thiếu) 9
Bảng 1.3. Cơ sở luật của hệ chuyên gia 10
Bảng 1.4. Kết quả kiểm định chéo phân tầng 10 phần trên Pima (trung bình ± độ lệch chuẩn) 11

Bảng 2.1. Năm mẫu đầu của bộ NHANES đã xử lý (một số cột) 18
Bảng 2.2. Thống kê mô tả các biến liên tục của NHANES 19
Bảng 2.3. Dò siêu tham số SVM (RBF) trên tập validation 21
Bảng 2.4. Kết quả phân loại ĐTĐ trên tập kiểm tra NHANES (4.130 người, ngưỡng Youden) 21
Bảng 2.5. Dự đoán HbA1c (%) từ 13 yếu tố nguy cơ (20.413 người có HbA1c, chia 80/20) 24
Bảng 2.6. Hồ sơ trung bình của bốn cụm K-means và tỉ lệ ĐTĐ (nhãn không dùng khi gom cụm) 25
Bảng 2.7. So sánh định tính các thuật toán trong bối cảnh hệ thống 26

Bảng 3.1. Các kiến trúc CNN tiêu biểu 31
Bảng 3.2. Số ảnh theo mức DR và trọng số lớp dùng trong hàm mất mát . . 33
Bảng 3.3. Kích thước tensor và số tham số qua từng lớp của CNN tự thiết kế 35
Bảng 3.4. Kết quả phân độ DR trên tập kiểm tra RetinaMNIST (400 ảnh) . 35

Bảng 4.1. Ba dòng đầu của một bản ghi CGM (bệnh nhân 1001, ĐTĐ típ 1) 43
Bảng 4.2. Chỉ số kiểm soát đường huyết của hai nhóm theo đồng thuận quốc tế về Time in Range [63] 44
Bảng 4.3. Kích thước tensor và số tham số của các mô hình hồi quy ($d = 3$, $m = 64$) 45
Bảng 4.4. Dự báo glucose trên 17 bệnh nhân của tập kiểm tra (18.012 cửa sổ) 46
Bảng 4.5. Kết quả tầm 30 phút theo loại ĐTĐ trên tập kiểm tra 47
Bảng 4.6. Sai số dự báo 30 phút theo dải glucose tham chiếu (tập kiểm tra, LSTM 2 tầng) 49

Bảng 5.1. So sánh RAG và tinh chỉnh (fine-tuning) trong bối cảnh chatbot y tế 55
Bảng 5.2. Ví dụ đoạn văn trong KB và bộ ba trong đồ thị tri thức 56
Bảng 5.3. Truy xuất trên KB tiếng Việt (60 câu hỏi, 36 đoạn) 60
Bảng 5.4. Hit@3 theo loại câu hỏi trên KB tiếng Việt 60

Bảng 5.5. Truy xuất câu trả lời trên MedQuAD-NIDDK (1.192 câu hỏi, 3.381 đoạn 800 ký tự) 62
Bảng 5.6. Ảnh hưởng của kích thước đoạn (bộ lai BM25 + MiniLM trên MedQuAD) 62
Bảng 5.7. Truy xuất chéo ngôn ngữ: 166 câu hỏi ĐTĐ dịch sang tiếng Việt, tìm trên kho tiếng Anh 63
Bảng 5.8. Chất lượng câu trả lời của Gemini theo ba chế độ (60 câu hỏi tiếng Việt) 63
Bảng 5.9. Câu trả lời thật của ba chế độ cho một câu hỏi đa bước 64

Bảng 6.1. Năm nút đầu tiên: mã PubMed, nhãn, bậc và các từ có TF-IDF cao nhất 72
Bảng 6.2. Mười từ có TF-IDF trung bình cao nhất của mỗi lớp (dạng từ gốc sau stemming) 72
Bảng 6.3. Cấu hình và số tham số của các mô hình (đặc trưng vào 500 chiều, 3 lớp) 74
Bảng 6.4. Accuracy phân loại nút trên Pubmed-Diabetes (trung bình ± độ lệch chuẩn) 75
Bảng 6.5. Ablation GCN trên giao thức bán giám sát (20 nhãn/lớp, 10 phép chia) 78
Bảng 6.6. Học chuyển dẫn và học quy nạp trên Pubmed-Diabetes (60/20/20, 3 lần chạy) 79
Bảng 6.7. Số tham số và thời gian huấn luyện trung bình một lần chạy (GPU RTX 3050) 79
Bảng 6.8. Ma trận nhầm lẫn của GAT triển khai trên 3.944 bài báo giữ riêng (20%) 80

Bảng 7.1. Các tác tử, thông điệp chúng xử lý và kỹ thuật bên trong 85
Bảng 7.2. Giao thức tranh biện của hội đồng 88
Bảng 7.3. So sánh cách tiếp cận của hệ thống với các framework đa tác tử dựa trên LLM 90
Bảng 7.4. Ví dụ tin nhắn sinh cho kịch bản K3 và giá trị đúng tương ứng . 91
Bảng 7.5. K1: tính đúng đắn và các chỉ số vận hành của hội đồng (399 ca) . 93
Bảng 7.6. K2 (chỉ có FPG): phát hiện ĐTĐ so với nhãn NHANES (399 ca, 91 người ĐTĐ) 94
Bảng 7.7. K3: trích xuất từ tin nhắn tiếng Việt tự do 95
Bảng 7.8. Luồng 16 thông điệp của một lượt hội chẩn thật (thời gian xử lý ghi ở thông điệp trả lời) 96
Bảng 7.9. Các điểm truy cập của hệ thống đã triển khai 98
Bảng 7.10. Tổng hợp kết quả thực nghiệm của bảy chương 100

MỞ ĐẦU

1. Lý do chọn đề tài

Trí tuệ nhân tạo (AI) đã đi một chặng đường dài từ những chương trình chứng minh định lý của thập niên 1950 tới các hệ thống tác tử dựa trên mô hình ngôn ngữ lớn hiện nay. Mỗi giai đoạn để lại một lớp kỹ thuật vẫn đang được dùng: luật chuyên gia, học máy thống kê, mạng nơ-ron tích chập và hồi quy, đồ thị tri thức, học sâu trên đồ thị và hệ đa tác tử. Một hệ thống thông minh thực tế hiếm khi chỉ dùng một kỹ thuật duy nhất. Nó thường là sự phối hợp của nhiều mô hình chuyên biệt, mỗi mô hình mạnh ở một loại dữ liệu, được điều phối để cùng đưa ra một quyết định có căn cứ.

Đái tháo đường (ĐTĐ) là một bài toán điển hình cho cách tiếp cận đó. Theo IDF Diabetes Atlas [1], năm 2021 có khoảng 537 triệu người trưởng thành mắc ĐTĐ, và gần một nửa chưa được chẩn đoán. Dữ liệu để sàng lọc lại rất đa dạng: chỉ số tự khai (tuổi, BMI, tiền sử gia đình) phù hợp với học máy trên bảng; ảnh đáy mắt dùng để phát hiện biến chứng võng mạc là đất của CNN; chuỗi đo đường huyết liên tục cần mô hình tuần tự như RNN; hướng dẫn lâm sàng (ADA 2024 [2], Quyết định 5481/QĐ-BYT [3]) là tri thức văn bản cần RAG và đồ thị tri thức; y văn có cấu trúc trích dẫn là dữ liệu đồ thị. Không mô hình đơn lẻ nào xử lý được tất cả. Cần một hệ đa tác tử phân công, đối chất và tổng hợp các kết luận.

Báo cáo này trình bày tiến trình phát triển của AI qua bảy chủ đề của môn học. Mỗi chủ đề được kiểm chứng bằng một thành phần chạy thật của cùng một hệ thống: **chatbot đa tác tử hỗ trợ sàng lọc nguy cơ đái tháo đường**, triển khai công khai tại `agent.nits.io.vn`. Cách làm này giúp lý thuyết của từng chương có một chỗ đứng cụ thể trong sản phẩm, và các chương nối với nhau như các bộ phận của một hệ thống.

2. Mục tiêu

1. Hệ thống hóa lịch sử AI và các kỹ thuật nền tảng: học máy cơ bản, CNN, RNN, đồ thị tri thức và RAG, mạng nơ-ron đồ thị, hệ đa tác tử. Với mỗi mô hình, giải thích chi tiết từng lớp (layer) và hàm hợp tạo nên nó.
2. Với mỗi chương, chọn một bộ dữ liệu công khai liên quan tới ĐTĐ. Mô tả đường dẫn, kích thước, mẫu dữ liệu và phân bố, kèm nhận xét về chất lượng dữ liệu.
3. Cài đặt và so sánh các mô hình trên cùng một giao thức đánh giá, kèm mô hình

đối chứng. Báo cáo bằng thước đo phù hợp với từng bài toán (ROC-AUC, QWK, RMSE và lưới Clarke, Hit@k và MRR, độ chính xác phân loại nút).

4. Đóng gói mô hình tốt nhất của mỗi chương thành dịch vụ web và triển khai qua Cloudflare Tunnel, để có thể kiểm tra trực tiếp từ Internet.

3. Đối tượng, phạm vi và dữ liệu

Đối tượng nghiên cứu là các mô hình và thuật toán của môn Hệ thống thông minh, áp dụng vào bài toán sàng lọc ĐTĐ. Bảng MĐ.1 liệt kê bảy bộ dữ liệu được dùng. Tất cả đều công khai và tải được bằng mã nguồn kèm theo. Thực nghiệm được thiết kế để chạy trên một máy tính xách tay (CPU Intel i5-11400H, 16 GB RAM, GPU RTX 3050 4 GB). Mục tiêu là kiểm chứng lý thuyết, không phải đạt kết quả tốt nhất hiện có. Hệ thống chỉ phục vụ học tập và nghiên cứu, *không thay thế chẩn đoán của bác sĩ.*

Bảng MĐ.1. Bảy bộ dữ liệu dùng trong báo cáo

Chương	Bộ dữ liệu	Quy mô	Bài toán
1	Pima Indians Diabetes [4]	768 người, 8 đặc trưng	Phân loại ĐTĐ: luật chuyên gia → perceptron → MLP
2	NHANES 2011–2018 [5]	20.646 người, 13 đặc trưng	Phân loại, hồi quy HbA1c, gom cụm
3	RetinaMNIST [6]	1.600 ảnh đáy mắt	Phân độ bệnh võng mạc ĐTĐ (5 mức)
4	ShanghaiT1DM/T2DM [7]	112 bệnh nhân, 125 bản ghi CGM	Dự báo glucose 15–60 phút
5	KB ADA 2024 + MedQuAD [8]	36 đoạn + 1.192 hỏi–đáp	Truy xuất và sinh câu trả lời có trích dẫn
6	Pubmed-Diabetes [9]	19.717 bài báo, 44.324 cạnh	Phân loại nút trên đồ thị trích dẫn
7	NHANES (tập kiểm tra)	400 ca, 400 tin nhắn	Đánh giá đầu–cuối hệ đa tác tử

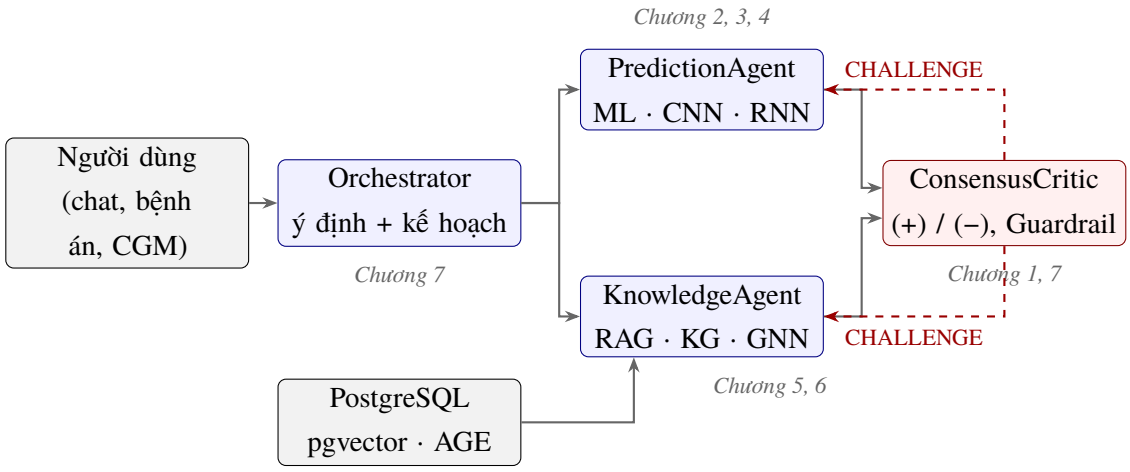
4. Phương pháp nghiên cứu

Báo cáo kết hợp bốn phương pháp. *Tổng hợp tài liệu*: hệ thống hóa kiến thức từ giáo trình và các bài báo gốc (Perceptron, LSTM, ResNet, GCN, GAT, RAG…). *Lịch sử – logic*: trình bày mỗi kỹ thuật như lời giải cho hạn chế của kỹ thuật đi trước. *Thực nghiệm*

có đối chứng: mỗi chương có ít nhất một mô hình nền đơn giản (persistence, MLP không có tích chập, MLP không dùng đồ thị, chỉ BM25, một tác tử đơn lẻ…). Dữ liệu được chia theo đơn vị độc lập (theo người, theo bệnh nhân) để tránh rò rỉ, siêu tham số chọn trên tập validation, còn tập kiểm tra chỉ dùng một lần. *Kỹ nghệ hệ thống*: đóng gói mô hình thành API, kiểm thử tự động, và triển khai bằng Docker cùng Cloudflare Tunnel.

5. Kiến trúc tổng thể của hệ thống

Hình MĐ.1 cho thấy vị trí của từng chương trong hệ thống. Người dùng trò chuyện với chatbot bằng văn bản tự do hoặc tải lên bệnh án. *Orchestrator* (Chương 7) phân loại ý định rồi giao việc cho các tác tử chuyên biệt qua một *MessageBus*. *PredictionAgent* dùng mô hình học máy (Chương 2), CNN (Chương 3) và RNN (Chương 4). *KnowledgeAgent* dùng RAG và đồ thị tri thức (Chương 5), cùng GNN để gán nhãn y văn (Chương 6). *ConsensusCriticAgent* tổ chức tranh biện, liệt kê bằng chứng đồng thuận (+) và mâu thuẫn (−) trước khi đưa ra kết luận. Hệ chuyên gia luật ở Chương 1 tồn tại dưới dạng các ngưỡng ADA và lớp *Safety Guardrail*. Đó là phần tri thức tường minh mà mọi mô hình học được đều phải tuân theo.



Hình MĐ.1. Vị trí các chương trong kiến trúc hệ thống chatbot đa tác tử

Toàn bộ hệ thống được đóng gói bằng Docker Compose với bốn dịch vụ: PostgreSQL 17 (có pgvector và Apache AGE), SeaweedFS (lưu file bệnh án), backend FastAPI (phục vụ cả API và giao diện) và `cloudflared`. `cloudflared` mở một kết nối đi ra (outbound) tới mạng Cloudflare, nhờ đó tên miền `agent.nits.io.vn` trỏ được về máy chủ mà không cần mở cổng hay có IP công khai [10]. Mỗi chương có một endpoint riêng dưới `/api/lab/chX/...` và một thẻ trên trang thử nghiệm `/lab`. Chatbot đầy đủ nằm ở trang gốc `/`.

6. Bố cục của báo cáo

Ngoài Mở đầu, Kết luận và Tài liệu tham khảo, báo cáo gồm bảy chương. **Chương 1** điểm lại lịch sử AI và tái hiện bốn thế hệ mô hình trên bộ Pima. **Chương 2** trình bày các thuật toán học máy cơ bản và so sánh tám bộ phân loại trên NHANES. **Chương 3** phân tích từng lớp của CNN và áp dụng vào phân độ bệnh võng mạc. **Chương 4** trình bày RNN, LSTM, GRU và bài toán dự báo đường huyết. **Chương 5** xây dựng đồ thị tri thức và đánh giá các chiến lược RAG. **Chương 6** trình bày GCN, GraphSAGE, GAT trên đồ thị trích dẫn y văn. **Chương 7** mô tả kiến trúc hệ đa tác tử, giao thức tranh biện và đánh giá đầu–cuối. Mỗi chương đều có bốn phần: cơ sở lý thuyết, dữ liệu, cài đặt và đánh giá, triển khai.

CHƯƠNG 1. LỊCH SỬ PHÁT TRIỂN CỦA TRÍ TUỆ NHÂN TẠO

Chương này tóm lược gần tám thập kỷ phát triển của AI. Mục tiêu là chỉ ra rằng mỗi cách tiếp cận ra đời để khắc phục giới hạn của cách tiếp cận trước. Phần thực nghiệm cuối chương tái hiện dòng lịch sử đó trên một bộ dữ liệu y khoa kinh điển: cùng một bài toán dự đoán đái tháo đường được giải lần lượt bằng hệ chuyên gia luật (thập niên 1970–80), perceptron (1958), ADALINE (1960), mạng nhiều lớp với lan truyền ngược (1986) và các phương pháp học máy thống kê hiện đại.

1.1. Khái niệm trí tuệ nhân tạo

Trí tuệ nhân tạo là ngành khoa học máy tính nghiên cứu cách xây dựng hệ thống thực hiện được những nhiệm vụ vốn đòi hỏi trí thông minh của con người: nhận thức, suy luận, học, lập kế hoạch, giao tiếp và ra quyết định. Russell và Norvig [11] phân loại các định nghĩa AI theo hai trục: *suy nghĩ* hay *hành động*, *giống người* (ví dụ phép thử Turing [12]) hay *hợp lý*. Cách tiếp cận *tác tử hợp lý* được dùng phổ biến nhất: tác tử nhận tri giác từ môi trường và chọn hành động tối đa hóa một thước đo hiệu năng, nên đo đạc và tối ưu được bằng toán học. Đây cũng là khái niệm trung tâm của Chương 7.

AI, học máy (ML) và học sâu (DL) là ba khái niệm lồng nhau. Học máy là nhánh của AI tập trung vào việc học quy luật từ dữ liệu thay vì lập trình tường minh. Học sâu là nhánh của học máy, dùng mạng nơ-ron nhiều lớp để tự học biểu diễn phân cấp. Phần lớn hệ thống hiện nay vẫn là *AI hẹp*, giỏi một nhiệm vụ cụ thể. *AI tổng quát* (AGI) có năng lực ngang con người trên phạm vi rộng vẫn là mục tiêu nghiên cứu.

1.2. Các giai đoạn phát triển

1.2.1. Nền tảng tiền đề (trước 1956)

Ba dòng chảy khoa học hội tụ để tạo nên AI. Thứ nhất là *logic và lý thuyết tính toán*: máy Turing (1936) hình thức hóa khái niệm thuật toán và gợi ra ý tưởng rằng tư duy, nếu là xử lý ký hiệu, có thể cơ giới hóa. Thứ hai là *mô hình nơ-ron*: McCulloch và Pitts (1943) [13] mô tả nơ-ron nhân tạo bằng tổng có trọng số qua một ngưỡng, còn Hebb (1949) đề xuất quy tắc "cùng kích hoạt thì liên kết mạnh lên". Thứ ba là *điều khiển học* và máy tính điện tử, cho phép kiểm chứng ý tưởng trên phần cứng thật. Năm 1950, Turing [12] đặt

câu hỏi "Máy có thể suy nghĩ không?" và đề xuất "trò chơi bắt chước", một tiêu chí hành vi thay cho tranh luận trừu tượng về bản chất của tư duy.

1.2.2. Khai sinh và thời kỳ lạc quan (1956 – đầu thập niên 1970)

Hội thảo Dartmouth mùa hè năm 1956 do McCarthy, Minsky, Rochester và Shannon tổ chức [14] đã khai sinh thuật ngữ *Artificial Intelligence*. Hai thập niên sau đó có nhiều chương trình gây ấn tượng: Logic Theorist (1956) chứng minh định lý toán học, LISP (1958) trở thành ngôn ngữ chủ đạo, ELIZA (1966) mô phỏng nhà trị liệu chỉ bằng luật khớp mẫu. Cũng trong giai đoạn này, Rosenblatt (1958) [15] công bố *perceptron*, mạng nơ-ron một lớp học được trọng số từ dữ liệu, còn Widrow và Hoff (1960) [16] đề xuất ADALINE với quy tắc học delta, tiền thân trực tiếp của hạ gradient ngày nay. Hướng tiếp cận chủ đạo khi đó là *AI biểu tượng*: tri thức được biểu diễn bằng ký hiệu và luật, còn trí thông minh được xem là quá trình tìm kiếm và suy diễn trên các biểu diễn đó.

1.2.3. Mùa đông AI thứ nhất (khoảng 1974 – 1980)

Các chương trình chạy tốt trên "thế giới đồ chơi" nhưng thất bại với bài toán thực vì ba lý do: *bùng nổ tổ hợp* của không gian tìm kiếm, *thiếu tri thức thường thức*, và *giới hạn của mạng một lớp*. Năm 1969, Minsky và Papert [17] chứng minh perceptron một lớp không biểu diễn được hàm XOR. Khi đó chưa có cách huấn luyện mạng nhiều lớp, nên nghiên cứu mạng nơ-ron gần như đình trệ. Báo cáo Lighthill (1973) [18] kết luận AI không đạt các mục tiêu đã hứa, và tài trợ ở Anh và Mỹ bị cắt giảm mạnh.

1.2.4. Hệ chuyên gia và mùa đông AI thứ hai (1980 – đầu thập niên 1990)

AI hồi sinh nhờ *hệ chuyên gia*: tri thức của chuyên gia trong một lĩnh vực hẹp được mã hóa thành luật NẾU–THÌ, kết hợp với một bộ máy suy diễn tiến hoặc lùi. MYCIN (Stanford) chẩn đoán nhiễm trùng máu với khoảng 600 luật. Để xử lý sự không chắc chắn, MYCIN gắn cho mỗi luật một *hệ số chắc chắn* (Certainty Factor – CF) và gộp chúng theo công thức của Shortliffe và Buchanan [19]:

$$\mathrm{CF}(H \mid E_1 \wedge E_2) = \mathrm{CF}_1 + \mathrm{CF}_2\,(1 - \mathrm{CF}_1), \qquad \mathrm{CF}_1, \mathrm{CF}_2 \geq 0. \tag{1.1}$$

Công thức (1.1) có tính giao hoán và kết hợp, luôn cho kết quả trong $[0, 1]$: mỗi bằng chứng mới chỉ "lấp" thêm một phần của khoảng còn thiếu. XCON (1980) là thành công thương mại lớn đầu tiên. Cùng thời gian, Rumelhart, Hinton và Williams (1986) [20] phổ biến thuật toán *lan truyền ngược*, cho phép huấn luyện mạng nhiều lớp và giải được bài

toán XOR. Tuy vậy hệ chuyên gia sớm bộc lộ giới hạn: thu nhận tri thức tốn kém, cơ sở luật khó bảo trì, hệ thống "giòn" trước tình huống ngoài luật. Thị trường máy LISP sụp đổ năm 1987 và mùa đông AI thứ hai bắt đầu.

1.2.5. Học máy thống kê (thập niên 1990 – 2000)

Bài học của hai mùa đông là tri thức thế giới thực quá lớn và mơ hồ để mã hóa thủ công. Trọng tâm vì thế chuyển sang các phương pháp *học từ dữ liệu* có nền tảng xác suất: mạng Bayes (Pearl, 1988), máy vector hỗ trợ (Cortes và Vapnik, 1995) [21], rừng ngẫu nhiên (Breiman, 2001) [22], boosting [23], LSTM (1997) [24] và LeNet-5 (1998) [25]. Đặc điểm chung của giai đoạn này là hiệu năng phụ thuộc nặng vào *kỹ nghệ đặc trưng* thủ công (SIFT, HOG, TF-IDF).

1.2.6. Kỷ nguyên học sâu (2006 – 2019)

Học sâu dùng mạng nhiều lớp để tự học biểu diễn phân cấp từ dữ liệu thô [26]. Hinton và cộng sự (2006) [27] khơi lại sự quan tâm bằng tiền huấn luyện từng lớp. ImageNet (2009) [28] cung cấp hàng triệu ảnh gán nhãn. Năm 2012, AlexNet [29] vô địch ILSVRC với lỗi top-5 khoảng 15,3

1.2.7. Mô hình nền tảng, AI tạo sinh và tác tử (2020 – nay)

GPT-3 (2020) [30] với 175 tỉ tham số cho thấy mô hình đủ lớn có thể học từ vài ví dụ trong câu lệnh. Tinh chỉnh theo chỉ dẫn và học tăng cường từ phản hồi người dùng [31] dẫn tới ChatGPT (2022). Năm 2024, giải Nobel Vật lý và Hóa học lần lượt vinh danh nền tảng mạng nơ-ron (Hopfield, Hinton) và AlphaFold [32]. Giai đoạn 2025–2026 nổi bật với ba xu hướng. *Mô hình đa phương thức bản địa* xử lý văn bản, ảnh, âm thanh trong cùng một mạng. *Mô hình suy luận* tăng lượng tính toán lúc suy luận để "nghĩ" nhiều bước. *AI tác tử* có khả năng lập kế hoạch, dùng công cụ, ghi nhớ và tự sửa lỗi. Khi nhiều tác tử phối hợp, ta có hệ đa tác tử, chủ đề của Chương 7. Đồng thời, *ảo giác* của LLM khiến tri thức tường minh trở lại dưới dạng đồ thị tri thức và RAG (Chương 5).

Bảng 1.1. Tóm tắt các giai đoạn phát triển chính của trí tuệ nhân tạo

Giai đoạn	Hướng chủ đạo	Thành tựu tiêu biểu	Hạn chế dẫn tới giai đoạn sau
1943–1955	Nền tảng lý thuyết	Nơ-ron McCulloch–Pitts, phép thử Turing	Chưa có phần cứng, chương trình
1956–1974	AI biểu tượng, tìm kiếm	Dartmouth, Logic Theorist, Perceptron, ADALINE	Bùng nổ tổ hợp; XOR → mùa đông 1
1980–1987	Hệ chuyên gia	MYCIN, XCON; lan truyền ngược	Khó thu nhận, bảo trì tri thức → mùa đông 2
1990–2011	Học máy thống kê	SVM, rừng ngẫu nhiên, LSTM, LeNet-5	Phụ thuộc đặc trưng thủ công
2012–2019	Học sâu	AlexNet, ResNet, AlphaGo, Transformer	Cần nhiều dữ liệu, khó giải thích
2020–2024	Mô hình nền tảng	GPT-3, ChatGPT, AlphaFold, RAG	Ảo giác, chi phí
2025–nay	Đa phương thức, tác tử	Mô hình suy luận, hệ đa tác tử, GraphRAG	An toàn, quản trị tác tử tự chủ

Những thách thức lớn vẫn còn: thiên lệch dữ liệu, khả năng giải thích, độ tin cậy, quyền riêng tư và khung pháp lý (Đạo luật AI của EU năm 2024; Chiến lược AI quốc gia của Việt Nam, Quyết định 127/QĐ-TTg [33]). Trong y tế, các rủi ro này càng nghiêm trọng, nên hệ thống của báo cáo luôn kèm lớp luật an toàn và trích dẫn nguồn.

1.3. Thực nghiệm: bốn thế hệ mô hình trên bộ Pima

1.3.1. Dữ liệu

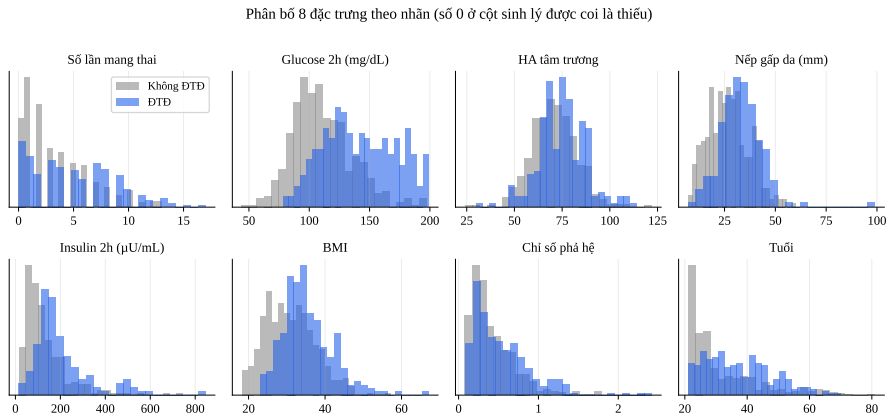
Bộ **Pima Indians Diabetes** do Viện Quốc gia về Đái tháo đường, Tiêu hóa và Thận Hoa Kỳ (NIDDK) thu thập. Smith và cộng sự công bố bộ này năm 1988 khi dùng thuật toán ADAP, một mạng dạng perceptron, để dự đoán khởi phát ĐTĐ [4]. Nhờ vậy, bộ dữ liệu này có ý nghĩa lịch sử riêng với chương này.

- **Nguồn**: UCI Machine Learning Repository, bản sao tại `https://www.kaggle.com/datasets/uciml/pima-indians-diabetes-database`; mã nguồn tải từ `https://raw.githubusercontent.com/jbrownlee/Datasets/master/pima-indians-diabetes.data.csv`.

- **Kích thước**: 768 phụ nữ gốc Pima ≥ 21 tuổi, 8 đặc trưng số, 1 nhãn; tệp CSV 23 KB.
- **Nhãn**: 268 người (34,9%) mắc ĐTĐ trong vòng 5 năm, 500 người không mắc.

Bảng 1.2. Năm mẫu đầu tiên của bộ Pima (số 0 ở cột sinh lý là giá trị thiếu)

Mang thai	Glucose	HA	Da	Insulin	BMI	Phả hệ	Tuổi	Nhãn
6	148	72	35	0	33,6	0,627	50	1
1	85	66	29	0	26,6	0,351	31	0
8	183	64	0	0	23,3	0,672	32	1
1	89	66	23	94	28,1	0,167	21	0
0	137	40	35	168	43,1	2,288	33	1



Hình 1.1. Phân bố tám đặc trưng của bộ Pima theo nhãn (mật độ chuẩn hóa)

Nhận xét. (i) Glucose 2 giờ sau nghiệm pháp là đặc trưng phân tách mạnh nhất: hai phân bố gần như dịch hẳn sang hai phía quanh 140 mg/dL, đúng ngưỡng rối loạn dung nạp glucose của ADA. (ii) Bộ dữ liệu có *giá trị thiếu ẩn*: số 0 không có nghĩa sinh lý nhưng xuất hiện ở insulin (374 dòng, 48,7%), độ dày nếp da (227 dòng), huyết áp (35), BMI (11) và glucose (5). Nếu không xử lý, mô hình sẽ học sai rằng "insulin = 0" là một mức đo. Thực nghiệm thay số 0 bằng trung vị của *tập huấn luyện trong từng lượt kiểm định* để tránh rò rỉ. (iii) Lớp dương chiếm 34,9%, lệch vừa phải, nên ngoài accuracy cần báo cáo ROC-AUC, F1, độ nhạy và độ đặc hiệu. (iv) Với 768 mẫu, kết quả một lần chia train/test dao động mạnh, nên dùng kiểm định chéo phân tầng 10 phần.

1.3.2. Mô hình

Mỗi mô hình đại diện cho một giai đoạn ở Bảng 1.1. Các mô hình 1 đến 4 được tự cài đặt bằng NumPy để thấy rõ cơ chế.

(1) Hệ chuyên gia. Sáu luật viết tay theo ngưỡng ADA/WHO (Bảng 1.3). Mỗi luật thỏa mãn đóng góp một CF, gộp lại theo (1.1). Đầu ra là độ tin "mắc ĐTĐ", kết luận dương khi CF ≥ 0,5. Hệ không học gì từ dữ liệu, toàn bộ "tri thức" nằm trong bảng luật.

Bảng 1.3. Cơ sở luật của hệ chuyên gia

#	Luật (NẾU … THÌ mắc ĐTĐ với độ tin CF)	CF
R1	Glucose 2 giờ ≥ 200 mg/dL (ngưỡng chẩn đoán ĐTĐ)	0,95
R2	140 ≤ glucose 2 giờ < 200 (rối loạn dung nạp glucose)	0,55
R3	BMI ≥ 30 (béo phì)	0,25
R4	Tuổi ≥ 45	0,20
R5	Chỉ số phả hệ ≥ 0,5 (tiền sử gia đình mạnh)	0,20
R6	Số lần mang thai ≥ 4	0,10

Ví dụ với mẫu đầu tiên (glucose 148, BMI 33,6, tuổi 50, phả hệ 0,627, mang thai 6 lần): R2, R3, R4, R5, R6 cùng thỏa mãn. Gộp lần lượt theo (1.1): $0{,}55 \to 0{,}55 + 0{,}25 \cdot 0{,}45 = 0{,}6625 \to 0{,}73 \to 0{,}784 \to 0{,}806$. Kết luận là dương tính, khớp với nhãn thật.

(2) Perceptron Rosenblatt. Với nhãn $y \in \{-1, +1\}$, mô hình dự đoán $\hat{y} = \operatorname{sign}(\mathbf{w}^\top \mathbf{x} + b)$. Thuật toán học perceptron (PLA) chỉ cập nhật khi phân loại sai:

$$\text{nếu } y_i(\mathbf{w}^\top \mathbf{x}_i + b) \le 0: \quad \mathbf{w} \leftarrow \mathbf{w} + \eta\, y_i \mathbf{x}_i, \qquad b \leftarrow b + \eta\, y_i. \tag{1.2}$$

PLA chỉ hội tụ khi dữ liệu tách được tuyến tính, điều Pima không thỏa mãn. Vì vậy cài đặt dùng biến thể "pocket": giữ lại bộ trọng số ít lỗi nhất qua 100 epoch.

(3) ADALINE. Widrow–Hoff cực tiểu sai số bình phương của đầu ra *tuyến tính* (trước hàm dấu), $J(\mathbf{w}) = \frac{1}{2}\sum_i (y_i - \mathbf{w}^\top \mathbf{x}_i - b)^2$, bằng hạ gradient:

$$\mathbf{w} \leftarrow \mathbf{w} + \eta \sum_i (y_i - \mathbf{w}^\top \mathbf{x}_i - b)\, \mathbf{x}_i. \tag{1.3}$$

Khác biệt then chốt so với PLA: hàm mất mát liên tục và lồi, nên luôn hội tụ, kể cả khi dữ liệu không tách được.

(4) MLP với lan truyền ngược. Mạng 8–16–1 là một hàm hợp:

$$\hat{p} = \sigma(\mathbf{W}_2^\top \sigma(\mathbf{W}_1^\top \mathbf{x} + \mathbf{b}_1) + b_2), \qquad \sigma(z) = \frac{1}{1+e^{-z}}, \tag{1.4}$$

huấn luyện bằng entropy chéo nhị phân $L = -\frac{1}{n}\sum[y \log \hat{p} + (1-y)\log(1-\hat{p})]$. Quy tắc chuỗi cho gradient ở lớp ra là $\delta_2 = (\hat{\mathbf{p}} - \mathbf{y})/n$. Ở lớp ẩn, gradient được truyền ngược qua $\mathbf{W}_2$ và nhân với đạo hàm sigmoid: $\delta_1 = (\delta_2 \mathbf{W}_2^\top) \odot \mathbf{h} \odot (1 - \mathbf{h})$. Đoạn mã 1.1 cài đặt đúng các phương trình này.

(5) Hồi quy logistic và **(6) Gradient Boosting** (scikit-learn [34]) đại diện cho học máy thống kê hiện đại. Chúng được trình bày chi tiết ở Chương 2.

Đoạn mã 1.1. MLP một lớp ẩn với lan truyền ngược bằng NumPy (`lab/ch1_history.py`)

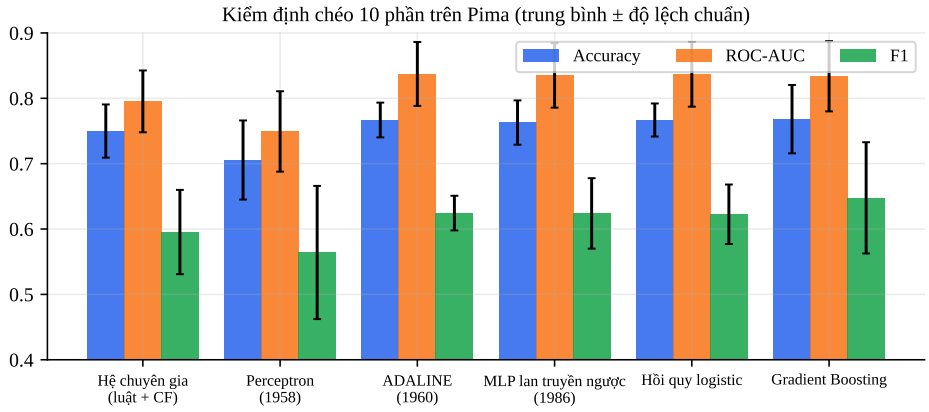
```
def fit(self, X, y):
    y = y.reshape(-1, 1).astype(float)
    for _ in range(self.epochs):
        h = sigmoid(X @ self.W1 + self.b1)                # lan truyền xuôi: lớp ẩn
        p = sigmoid(h @ self.W2 + self.b2)                # lớp ra: xác suất mắc ĐTĐ
        d2 = (p - y) / n                                  # ∂L/∂z2 (sigmoid + entropy chéo)
        d1 = (d2 @ self.W2.T) * h * (1 - h)               # quy tắc chuỗi qua lớp ẩn
        self.W2 -= self.lr * (h.T @ d2 + self.l2 * self.W2)
        self.b2 -= self.lr * d2.sum(0)
        self.W1 -= self.lr * (X.T @ d1 + self.l2 * self.W1)
        self.b1 -= self.lr * d1.sum(0)
```

1.3.3. Kết quả và nhận xét

Bảng 1.4 và Hình 1.2 tổng hợp kết quả kiểm định chéo 10 phần.

Bảng 1.4. Kết quả kiểm định chéo phân tầng 10 phần trên Pima (trung bình ± độ lệch chuẩn)

Mô hình (năm)	Accuracy	ROC-AUC	F1	Độ nhạy	Độ đặc hiệu
Hệ chuyên gia, luật + CF	0,750 ± 0,041	0,795 ± 0,047	0,595	0,530	0,868
Perceptron (1958)	0,706 ± 0,061	0,749 ± 0,062	0,564	0,564	0,782
ADALINE (1960)	0,767 ± 0,027	**0,837** ± 0,049	0,624	0,556	0,880
MLP lan truyền ngược (1986)	0,763 ± 0,034	0,835 ± 0,049	0,624	0,568	0,868
Hồi quy logistic	0,767 ± 0,025	**0,837** ± 0,050	0,623	0,556	0,880
Gradient Boosting	**0,768** ± 0,052	0,834 ± 0,054	**0,648**	**0,616**	0,850



Hình 1.2. So sánh sáu mô hình đại diện các giai đoạn lịch sử trên Pima

Có bốn nhận xét chính.

Thứ nhất, hệ chuyên gia chỉ với sáu luật đạt AUC 0,795, cao hơn perceptron học từ dữ liệu. Tri thức y khoa cô đọng (ngưỡng glucose của ADA) rất mạnh, và đây là lý do các ngưỡng này vẫn còn trong hệ thống hiện đại dưới dạng KnowledgeAgent và Safety Guardrail (Chương 7). Tuy vậy, độ nhạy chỉ đạt 0,53: luật cứng bỏ sót những người có glucose dưới ngưỡng nhưng tổ hợp nhiều yếu tố nguy cơ. Đó chính là tính "giòn" của hệ chuyên gia.

Thứ hai, perceptron là mô hình yếu nhất và có phương sai lớn nhất (độ lệch chuẩn của độ nhạy là 0,16). Số lỗi mỗi epoch của PLA không giảm đơn điệu: thuật toán chỉ phản ứng với mẫu sai nên "nhảy" liên tục khi dữ liệu không tách được tuyến tính, trong khi hàm mất mát của ADALINE và MLP giảm mượt.

Thứ ba, ADALINE với cùng một mô hình tuyến tính nhưng có hàm mất mát liên tục đã tăng AUC từ 0,749 lên 0,837. Đây là bài học quan trọng nhất của chương: *cách tối ưu quan trọng không kém kiến trúc*. ADALINE ngang bằng hồi quy logistic, vì cả hai đều là mô hình tuyến tính được tối ưu bằng gradient.

Thứ tư, MLP và Gradient Boosting không vượt được mô hình tuyến tính trên 768 mẫu. Quan hệ giữa các đặc trưng và nhãn ở Pima gần tuyến tính, còn dữ liệu quá ít để mô hình phi tuyến phát huy. Gradient Boosting cho F1 và độ nhạy cao nhất nhưng phương sai lớn. Kết quả này khớp với các báo cáo thường thấy, khoảng 76–78% accuracy trên Pima, và với con số 76% của ADAP năm 1988 [4].

Bài toán XOR. Trên bốn điểm của XOR, perceptron chỉ đạt 50% (2/4 điểm) dù huấn luyện bao lâu. Đây đúng là phản ví dụ của Minsky và Papert. MLP 2–4–1 huấn luyện bằng lan truyền ngược đạt 100%, với xác suất đầu ra (0,000; 0,999; 0,999; 0,001). Hai lớp nơ-ron tạo ra hai siêu phẳng, và tổ hợp của chúng tách được vùng "chéo" mà một

siêu phẳng không làm nổi. Thí nghiệm nhỏ này tóm tắt bước chuyển từ mùa đông AI thứ nhất sang thời kỳ hồi sinh của mạng nơ-ron năm 1986.

1.4. Triển khai

Endpoint `POST /api/lab/ch1/pima` nhận tám chỉ số của một người. Nó trả về kết luận của hệ chuyên gia kèm danh sách luật đã kích hoạt và CF sau mỗi bước gộp, cùng điểm của perceptron và xác suất của MLP. Các mô hình được huấn luyện trên toàn bộ 768 mẫu và nạp từ tệp `data/models/ch1_pima.joblib`. Trên trang `/lab` (thẻ "Chương 1"), người dùng nhập chỉ số và thấy ngay ba "thế hệ" mô hình cho kết quả khác nhau ra sao trên cùng một ca.

Đoạn mã 1.2. Gọi endpoint Chương 1 và kết quả trả về (rút gọn)

```
curl -s https://agent.nits.io.vn/api/lab/ch1/pima -H 'Content-Type: application/json' \
  -d '{"pregnancies":6,"glucose":148,"blood_pressure":72,"skin_thickness":35,
       "insulin":0,"bmi":33.6,"pedigree":0.627,"age":50}'
# {"expert": {"cf": 0.8056, "positive": true,
#             "fired": [["Glucose 2h 140–199 (rối loạn dung nạp)", 0.55], ["BMI ≥ 30 (béo
 ↪  phì)", 0.25], ...],
#             "cf_trace": [0.55, 0.6625, 0.73, 0.784, 0.8056]},
#  "perceptron": {"score": 0.0274, "positive": true},
#  "mlp": {"probability": 0.7058, "positive": true}, "imputed": ["insulin"]}
```

1.5. Tiểu kết chương 1

Lịch sử AI là chuỗi chu kỳ giữa kỳ vọng và thực tế. AI biểu tượng mạnh về suy luận nhưng bất lực trước sự phức tạp của thế giới thực. Hệ chuyên gia thành công thương mại nhưng vấp nút thắt thu nhận tri thức. Học máy thống kê và học sâu chuyển trọng tâm sang học từ dữ liệu. Thực nghiệm trên Pima tái hiện đúng dòng chảy đó: luật chuyên gia giàu tri thức nhưng cứng nhắc; perceptron học được nhưng bất ổn và không giải được XOR; ADALINE cho thấy sức mạnh của tối ưu gradient; lan truyền ngược mở khóa mô hình phi tuyến. Hệ thống ở các chương sau không loại bỏ giai đoạn nào. Nó kết hợp luật tường minh (ngưỡng ADA, guardrail) với các mô hình học từ dữ liệu, mỗi loại ở đúng chỗ mạnh của mình.

CHƯƠNG 2. CÁC KỸ THUẬT HỌC MÁY CƠ BẢN

Học máy là "lõi dự đoán" của PredictionAgent. Chương này trình bày nền tảng chung (bài toán học, hàm mất mát, tối ưu, đánh giá) và các thuật toán cổ điển: hồi quy tuyến tính, hồi quy logistic, KNN, Naive Bayes, cây quyết định, rừng ngẫu nhiên, gradient boosting, SVM, K-means và PCA. Thực nghiệm dùng dữ liệu thật của khảo sát NHANES với 13 đặc trưng mà người dùng chatbot có thể tự khai. Mục tiêu là so sánh tám bộ phân loại, bốn mô hình hồi quy và một thuật toán gom cụm trên cùng giao thức đánh giá.

2.1. Bài toán học và quy trình xây dựng mô hình

Theo Mitchell, một chương trình *học* từ kinh nghiệm E đối với nhiệm vụ T và thước đo P nếu hiệu năng trên T, đo bằng P, tăng theo E. Trong học có giám sát, ta có tập $\mathcal{D} = \{(\mathbf{x}_i, y_i)\}_{i=1}^{n}$ lấy mẫu độc lập từ một phân phối $p(\mathbf{x}, y)$ chưa biết, và tìm hàm f_θ cực tiểu *rủi ro thực nghiệm* có điều chuẩn:

$$\hat{\theta} = \arg\min_{\theta} \frac{1}{n} \sum_{i=1}^{n} \ell(f_\theta(\mathbf{x}_i), y_i) + \lambda\, \Omega(\theta). \tag{2.1}$$

Hồi quy dùng ℓ là sai số bình phương. Phân loại dùng entropy chéo hoặc hàm hinge. Ω thường là chuẩn L_2 (Ridge) hoặc L_1 (Lasso). Ngoài học có giám sát còn có học không giám sát (gom cụm, giảm chiều), học bán giám sát (Chương 6) và học tăng cường (TreatmentRLAgent ở Chương 7).

Quy trình xây dựng một mô hình gồm sáu bước: (1) xác định bài toán và thước đo; (2) thu thập, làm sạch dữ liệu; (3) chia tập train/validation/test; (4) tiền xử lý (điền khuyết, chuẩn hóa, mã hóa) — *chỉ học tham số tiền xử lý trên tập train*; (5) huấn luyện và chọn siêu tham số trên validation; (6) đánh giá một lần trên test rồi triển khai. Việc dùng tập test để chọn mô hình làm kết quả lạc quan giả tạo, gọi là *rò rỉ dữ liệu*.

2.2. Tối ưu hóa bằng hạ gradient

Khi (2.1) không có nghiệm đóng, ta lặp $\theta_{t+1} = \theta_t - \eta \nabla_\theta J(\theta_t)$. Hạ gradient ngẫu nhiên (SGD) ước lượng gradient trên một lô nhỏ B, nên mỗi bước rẻ hơn và có nhiễu giúp thoát điểm yên ngựa. Adam [35] duy trì trung bình động của gradient $\mathbf{m}_t$ và của

bình phương gradient $\mathbf{v}_t$:

$$\begin{aligned} \mathbf{m}_t &= \beta_1 \mathbf{m}_{t-1} + (1-\beta_1)\mathbf{g}_t, \\ \mathbf{v}_t &= \beta_2 \mathbf{v}_{t-1} + (1-\beta_2)\mathbf{g}_t^2, \\ \theta_{t+1} &= \theta_t - \eta \, \frac{\hat{\mathbf{m}}_t}{\sqrt{\hat{\mathbf{v}}_t} + \epsilon}, \end{aligned} \tag{2.2}$$

với $\hat{\mathbf{m}}_t = \mathbf{m}_t/(1-\beta_1^t)$, $\hat{\mathbf{v}}_t = \mathbf{v}_t/(1-\beta_2^t)$ là hai bản hiệu chỉnh độ chệch. Adam được dùng cho mọi mô hình học sâu ở các chương 3–6.

2.3. Các thuật toán

2.3.1. Hồi quy tuyến tính

Mô hình $\hat{y} = \mathbf{w}^\top \mathbf{x} + b$, cực tiểu $\sum (y_i - \hat{y}_i)^2$, có nghiệm đóng $\hat{\mathbf{w}} = (\mathbf{X}^\top \mathbf{X})^{-1}\mathbf{X}^\top \mathbf{y}$. Thêm điều chuẩn L_2 (Ridge) cho nghiệm $(\mathbf{X}^\top \mathbf{X} + \lambda \mathbf{I})^{-1}\mathbf{X}^\top \mathbf{y}$, ổn định khi các đặc trưng tương quan (ví dụ BMI và vòng eo). Khi đặc trưng đã chuẩn hóa z-score, w_j là mức thay đổi của y khi x_j tăng một độ lệch chuẩn, nên đọc được trực tiếp.

2.3.2. Hồi quy logistic

Cho phân loại nhị phân, mô hình ước lượng xác suất $p(y{=}1 \mid \mathbf{x}) = \sigma(\mathbf{w}^\top \mathbf{x} + b)$ và cực tiểu entropy chéo:

$$\begin{aligned} J(\mathbf{w}) &= -\frac{1}{n}\sum_{i=1}^{n} \left[y_i \log p_i + (1-y_i)\log(1-p_i) \right] + \frac{\lambda}{2}\|\mathbf{w}\|^2, \\ \nabla_{\mathbf{w}} J &= \frac{1}{n}\mathbf{X}^\top(\mathbf{p} - \mathbf{y}) + \lambda \mathbf{w}. \end{aligned} \tag{2.3}$$

J lồi nên có cực tiểu toàn cục duy nhất. Trong y khoa, e^{w_j} là *tỉ số chênh* (odds ratio, OR): odds mắc bệnh nhân lên e^{w_j} lần khi x_j tăng một đơn vị, ở đây là một độ lệch chuẩn. Mở rộng sang K lớp, ta thay sigmoid bằng softmax $p_k = e^{z_k}/\sum_j e^{z_j}$, đây cũng là lớp ra của mọi mạng phân loại ở các chương sau.

2.3.3. K láng giềng gần nhất

KNN [36] không có pha huấn luyện. Với một điểm mới, thuật toán tìm k điểm gần nhất theo khoảng cách Euclid trên dữ liệu đã chuẩn hóa, rồi bỏ phiếu. Phiếu có thể đánh trọng số $1/d$ theo khoảng cách. k nhỏ cho biên quyết định gồ ghề, phương sai lớn và dễ quá khớp. k lớn cho biên trơn hơn nhưng độ chệch tăng. Chi phí dự đoán là $O(nd)$ cho

mỗi điểm, và hiệu quả giảm khi số chiều tăng, do "lời nguyền số chiều".

2.3.4. Naive Bayes

Áp dụng định lý Bayes với giả định các đặc trưng độc lập có điều kiện theo lớp:

$$p(y \mid \mathbf{x}) \propto p(y) \prod_{j=1}^{d} p(x_j \mid y), \qquad p(x_j \mid y = c) = \mathcal{N}(x_j;\ \mu_{jc}, \sigma_{jc}^2) \text{ (bản Gauss).} \quad (2.4)$$

Mô hình học rất nhanh, nhưng giả định độc lập bị vi phạm nặng khi đặc trưng tương quan (BMI với vòng eo, huyết áp tâm thu với tâm trương). Kết quả là xác suất bị đẩy về 0 hoặc 1, hiệu chuẩn kém. Thực nghiệm dưới đây sẽ cho thấy rõ điều này.

2.3.5. Cây quyết định và các phương pháp tổ hợp

Cây quyết định chia không gian đặc trưng thành các hình hộp bằng chuỗi câu hỏi "$x_j \le t$?". Tại mỗi nút, thuật toán CART chọn cặp (j, t) cực đại mức giảm độ hỗn tạp. Độ hỗn tạp đo bằng chỉ số Gini $G = 1 - \sum_k p_k^2$ hoặc entropy $H = -\sum_k p_k \log p_k$, với p_k là tỉ lệ lớp k trong nút. Cây dễ giải thích, xử lý được tương tác phi tuyến và không cần chuẩn hóa. Đổi lại, cây sâu quá khớp mạnh, và một thay đổi nhỏ trong dữ liệu có thể đổi cả cây.

Rừng ngẫu nhiên [22] huấn luyện B cây trên các mẫu bootstrap. Tại mỗi lần chia, cây chỉ xét một tập con ngẫu nhiên $\sqrt{d}$ đặc trưng, rồi lấy trung bình dự đoán của các cây. Nếu các cây có phương sai σ^2 và tương quan ρ, phương sai của trung bình là $\rho\sigma^2 + \frac{1-\rho}{B}\sigma^2$. Việc chọn đặc trưng ngẫu nhiên làm giảm ρ, nhờ đó giảm phương sai mà không tăng độ chệch.

Gradient boosting [23] xây cây *tuần tự*: cây thứ m khớp với gradient âm ("phần dư giả") của hàm mất mát đối với dự đoán hiện tại, $F_m = F_{m-1} + \nu\, h_m$, với ν là tốc độ học. Bản HistGradientBoosting (giống LightGBM [37]) rời rạc hóa đặc trưng thành 255 thùng. Nhờ vậy huấn luyện nhanh, và bản này xử lý giá trị thiếu tự nhiên bằng cách học hướng đi cho NaN tại mỗi nút. Đây là mô hình ML chạy trong chatbot.

2.3.6. Máy vector hỗ trợ

SVM [21] tìm siêu phẳng có *lề* lớn nhất. Với dữ liệu không tách được, ta thêm biến chùng ξ_i:

$$\min_{\mathbf{w}, b, \xi} \ \frac{1}{2}\|\mathbf{w}\|^2 + C \sum_i \xi_i \quad \text{s.t.} \quad y_i(\mathbf{w}^\top \phi(\mathbf{x}_i) + b) \ge 1 - \xi_i,\ \xi_i \ge 0. \quad (2.5)$$

Bài toán đối ngẫu chỉ phụ thuộc vào tích vô hướng, nên thay được bằng hàm nhân $K(\mathbf{x}, \mathbf{x}') = \exp(-\gamma\|\mathbf{x} - \mathbf{x}'\|^2)$ (RBF). Hàm quyết định là $f(\mathbf{x}) = \sum_{i \in SV} \alpha_i y_i K(\mathbf{x}_i, \mathbf{x}) + b$, chỉ gồm các *vector hỗ trợ*. C điều khiển mức phạt vi phạm lề, γ điều khiển độ "cục bộ" của nhân. Tham số `class_weight` nhân C theo nghịch đảo tần suất lớp để bù lệch lớp. SVM không trả về xác suất. Xác suất được hiệu chuẩn thêm bằng phương pháp Platt [38]. Hồi quy vector hỗ trợ (SVR) dùng hàm mất mát ϵ-không nhạy: sai số nhỏ hơn ϵ không bị phạt.

2.3.7. Gom cụm K-means và giảm chiều PCA

K-means [39] cực tiểu tổng bình phương khoảng cách tới tâm cụm, $\sum_k \sum_{\mathbf{x} \in C_k} \|\mathbf{x} - \mu_k\|^2$ (inertia). Thuật toán lặp hai bước tới khi hội tụ: gán mỗi điểm vào tâm gần nhất, rồi cập nhật tâm bằng trung bình các điểm. Chọn k bằng "khuỷu tay" của inertia hoặc hệ số silhouette [40], $s = (b - a)/\max(a, b)$, với a là khoảng cách trung bình tới các điểm cùng cụm và b là khoảng cách trung bình tới cụm gần nhất khác. PCA chiếu dữ liệu lên các vector riêng ứng với trị riêng lớn nhất của ma trận hiệp phương sai, giữ lại phương sai nhiều nhất. Ở đây PCA dùng để trực quan hóa cụm trên mặt phẳng.

2.3.8. Quá khớp, thiếu khớp và điều chuẩn

Sai số kỳ vọng phân rã thành độ chệch^2 + phương sai + nhiễu. Mô hình quá đơn giản thì thiếu khớp: sai số train và test cùng cao. Mô hình quá phức tạp thì quá khớp: train rất tốt, test kém. Các núm điều chỉnh gồm k của KNN, độ sâu cây, C và γ của SVM, λ của Ridge, số cây và tốc độ học của boosting. Mục 2.6 vẽ đường cong train/validation theo k và theo độ sâu để thấy rõ đánh đổi này.

2.3.9. Thước đo đánh giá

Với phân loại nhị phân, từ ma trận nhầm lẫn (TP, FP, TN, FN) ta có độ nhạy (recall) $= \frac{TP}{TP+FN}$, độ đặc hiệu $= \frac{TN}{TN+FP}$, độ chính xác (precision, PPV) $= \frac{TP}{TP+FP}$ và $F_1 = \frac{2PR}{P+R}$. Các thước đo này phụ thuộc vào ngưỡng. Ở đây ngưỡng được chọn trên validation theo chỉ số Youden J = độ nhạy + độ đặc hiệu − 1 [41]. ROC-AUC đo khả năng xếp hạng, không phụ thuộc ngưỡng: đó là xác suất một ca dương được chấm điểm cao hơn một ca âm ngẫu nhiên. PR-AUC (average precision) nhạy hơn khi lớp dương hiếm; mốc ngẫu nhiên của nó bằng tỉ lệ dương. Điểm Brier $\frac{1}{n}\sum(p_i - y_i)^2$ đo độ hiệu chuẩn của xác suất, quan trọng khi chatbot nói "nguy cơ 34%". Với hồi quy, ta dùng RMSE, MAE và $R^2 = 1 - \frac{\sum(y-\hat{y})^2}{\sum(y-\bar{y})^2}$.

2.4. Dữ liệu NHANES 2011–2018

NHANES (National Health and Nutrition Examination Survey) là khảo sát sức khỏe quốc gia của CDC Hoa Kỳ [5]. Mỗi chu kỳ hai năm, khảo sát kết hợp phỏng vấn, khám thực thể và xét nghiệm máu trên mẫu đại diện dân số.

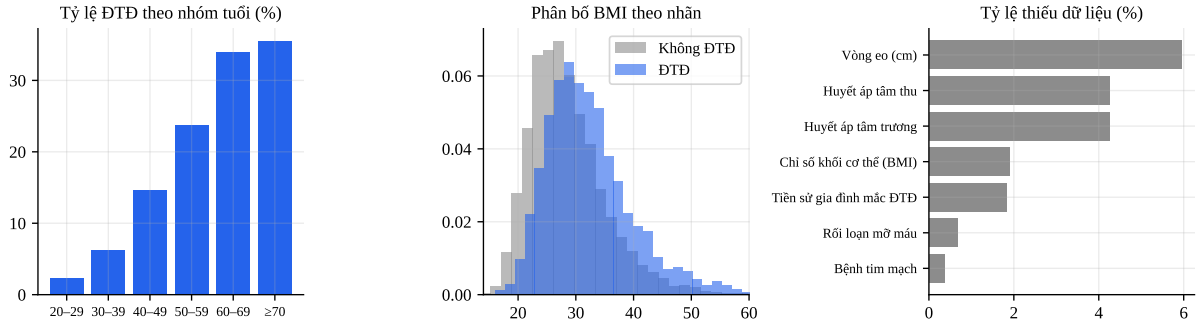
- **Nguồn**: `https://wwwn.cdc.gov/nchs/nhanes/`. Script `scripts/fetch_nhanes.py` tải 40 tệp XPT của 4 chu kỳ 2011–2018 (các mô-đun DEMO, BMX, BPX, BPQ, DIQ, GHB, GLU, MCQ, PAQ, SMQ), ghép theo mã người `SEQN` và lọc người ≥ 20 tuổi.
- **Kích thước**: 20.646 người × 22 cột (tệp CSV 2,4 MB). Bốn chu kỳ gần cân bằng: 5.034 / 5.371 / 5.198 / 5.043 người. Có 2.630 người gốc Á, dùng để đánh giá nhóm con gần với người Việt.
- **Nhãn**: ĐTĐ theo ADA [2]. Một người được gán nhãn ĐTĐ khi *đã được bác sĩ chẩn đoán* (3.142 người), *hoặc* HbA1c ≥ 6,5%, *hoặc* glucose lúc đói ≥ 126 mg/dL. Trong số đó có 943 người *chưa từng được chẩn đoán* nhưng có xét nghiệm vượt ngưỡng. Tổng tỉ lệ dương là 19,8%; tỉ lệ tiền ĐTĐ là 34,3%.
- **Đặc trưng** (13): tuổi, giới, BMI, vòng eo, huyết áp tâm thu và tâm trương (trung bình các lần đo), tăng huyết áp, rối loạn mỡ máu, tiền sử gia đình, hút thuốc, vận động, bệnh tim, đột quỵ. HbA1c và glucose *không* được đưa vào đặc trưng, vì nhãn được định nghĩa từ chính chúng (rò rỉ nhãn). Trong chatbot, xét nghiệm do KnowledgeAgent đối chiếu ngưỡng; mô hình ML ước lượng nguy cơ từ các yếu tố nguy cơ.

Bảng 2.1. Năm mẫu đầu của bộ NHANES đã xử lý (một số cột)

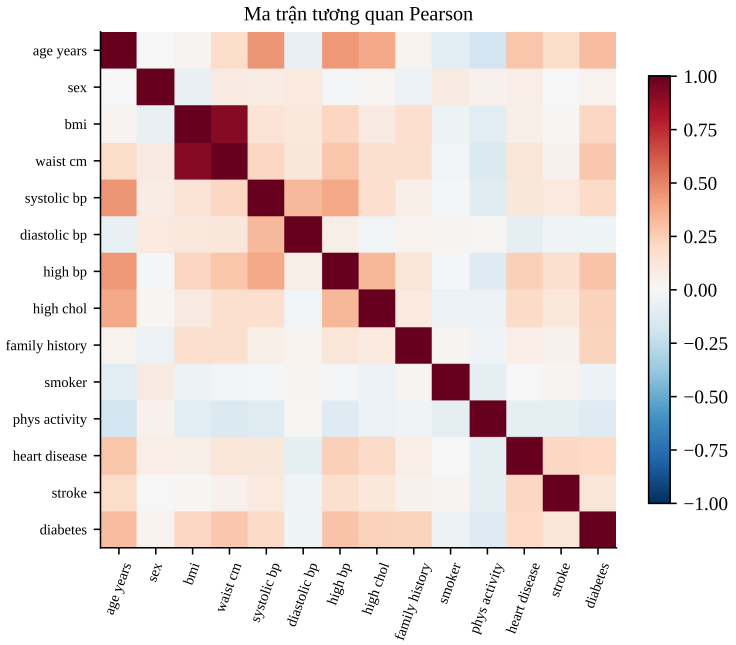
SEQN	Tuổi	Nam	BMI	Eo	HA tâm thu	THA	GĐ	HbA1c	ĐTĐ
62161	22	1	23,3	81,0	110,7	0	1	5,1	0
62164	44	0	23,2	80,1	118,0	0	0	4,9	0
62169	21	1	20,1	69,6	124,7	0	0	5,4	0
62172	43	0	33,3	120,4	102,0	0	1	5,6	0
62174	80	1	33,9	116,5	97,0	1	–	5,0	0

Bảng 2.2. Thống kê mô tả các biến liên tục của NHANES

Biến	Số đo	TB	ĐLC	Min	Trung vị	Max
Tuổi (năm)	20.646	50,2	17,6	20	50	80
BMI (kg/m²)	20.255	29,4	7,1	13,4	28,2	86,2
Vòng eo (cm)	19.416	99,7	16,6	55,5	98,3	177,9
HA tâm thu (mmHg)	19.769	125,0	18,7	64,7	122,0	238,0
HbA1c (%)	20.413	5,82	1,13	3,5	5,5	17,8
Glucose đói (mg/dL)	9.827	111,2	37,6	21	101	479



Hình 2.1. Tỉ lệ ĐTĐ theo tuổi, phân bố BMI theo nhãn và tỉ lệ thiếu dữ liệu



Hình 2.2. Ma trận tương quan Pearson giữa các đặc trưng và nhãn

Nhận xét. (i) Tỉ lệ ĐTĐ tăng mạnh theo tuổi, từ 2,4% ở nhóm 20–29 lên 23,7% ở nhóm 50–59, rồi chững lại quanh 34–36% từ 60 tuổi. Tuổi vì thế là yếu tố mạnh nhất,

khớp với khuyến cáo sàng lọc mọi người từ 35 tuổi của ADA. (ii) Phân bố BMI của nhóm ĐTĐ dịch phải nhưng chồng lấn lớn: BMI đơn lẻ không đủ để phân loại. (iii) Dữ liệu thiếu ít (vòng eo 6,0%, huyết áp 4,3%, BMI 1,9%) nhưng có thật. Người dùng chatbot còn thiếu nhiều hơn thế, nên các pipeline được thiết kế để chịu được giá trị thiếu. (iv) BMI và vòng eo tương quan rất mạnh (r = 0,91); tăng huyết áp tương quan với tuổi (r = 0,43). Tương quan với nhãn chỉ ở mức yếu đến vừa (tuổi 0,31; tăng huyết áp 0,28; vòng eo 0,27; tiền sử gia đình 0,23; BMI 0,21): không có đặc trưng "thần kỳ", và đây là bài toán khó hơn Pima nhiều vì không có glucose. (v) Glucose lúc đói chỉ có ở 9.827 người (mẫu con nhịn đói buổi sáng), đây là đặc điểm thiết kế của NHANES.

2.5. Cài đặt

Dữ liệu được chia 70/10/20 phân tầng theo nhãn với `random_state=42`, cho 14.451 / 2.065 / 4.130 người. Phép chia này trùng với script huấn luyện của chatbot, nên tập kiểm tra cũng là tập dùng ở Chương 7. Với các mô hình không xử lý được NaN, pipeline gồm `SimpleImputer` (trung vị, thêm cột chỉ báo "bị thiếu") rồi `StandardScaler`. Gradient boosting nhận NaN trực tiếp. Mọi bước tiền xử lý nằm trong `Pipeline` của scikit-learn [34], nên tham số chỉ được học trên tập train (Đoạn mã 2.1).

Đoạn mã 2.1. Tám bộ phân loại và pipeline tiền xử lý (`lab/ch2_ml.py`, rút gọn)

```
def imputed(model):
    """Điền trung vị + cột chỉ báo thiếu, chuẩn hóa z-score, rồi tới mô hình."""
    return make_pipeline(SimpleImputer(strategy="median", add_indicator=True),
     ↪  StandardScaler(), model)

classifiers = {
    "logreg": imputed(LogisticRegression(max_iter=3000, C=1.0)),
    "knn":    imputed(KNeighborsClassifier(n_neighbors=51, weights="distance")),
    "nb":     imputed(GaussianNB()),
    "tree":   imputed(DecisionTreeClassifier(max_depth=6, min_samples_leaf=40)),
    "rf":     imputed(RandomForestClassifier(400, min_samples_leaf=10, n_jobs=-1)),
    "gbdt":   HistGradientBoostingClassifier(max_iter=300, learning_rate=0.05),  # tự xử lý
     ↪  NaN
    "svm":    imputed(SVC(C=1.0, gamma=0.01, class_weight="balanced", probability=True)),
    "mlp":    imputed(MLPClassifier((64, 32), alpha=1e-3, early_stopping=True)),
}
for key, model in classifiers.items():
    model.fit(X_tr, y_tr)
    thr = best_threshold(y_val, score(model, X_val))   # ngưỡng Youden chọn trên validation
    p = score(model, X_te)                              # đánh giá một lần trên test
```

Siêu tham số chọn trên validation: k của KNN theo đường cong ở Hình 2.4, độ sâu cây, và (C, γ, class_weight) của SVM theo Bảng 2.3. Một chi tiết kỹ thuật quan trọng: với

SVM, các chỉ số xếp hạng được tính trên `decision_function` chứ không dùng `predict_proba`. Lý do là bộ hiệu chuẩn Platt của scikit-learn được khớp bằng kiểm định chéo nội bộ, nên có thể đảo thứ tự xếp hạng so với mô hình chính.

Bảng 2.3. Dò siêu tham số SVM (RBF) trên tập validation

C	class_weight	γ	Số vector hỗ trợ	AUC validation
1	không	scale	5.707	0,788
1	balanced	scale	8.101	0,830
0,1	balanced	scale	8.711	0,835
1	**balanced**	**0,01**	**8.289**	**0,838**
10	balanced	0,01	8.019	0,830

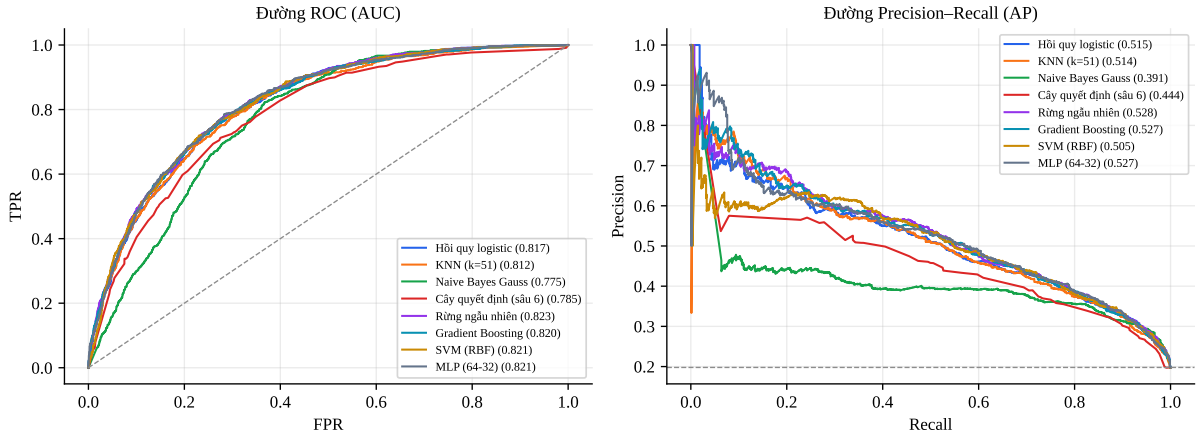
Không có trọng số lớp, SVM tối ưu hàm hinge trên dữ liệu 80/20 nên "bỏ cuộc" với lớp dương ở vùng chồng lấn, và AUC chỉ đạt 0,788. Bù lệch lớp đưa AUC lên 0,830. Giảm γ để nhân RBF "rộng" hơn còn tăng thêm, lên 0,838. Hơn 8.000 trên 14.451 điểm là vector hỗ trợ, cho thấy biên giữa hai lớp rất nhiễu. Điều này cũng giải thích vì sao SVM có thời gian huấn luyện (24 giây) và suy luận lâu nhất.

2.6. Kết quả và nhận xét

2.6.1. Phân loại

Bảng 2.4. Kết quả phân loại ĐTĐ trên tập kiểm tra NHANES (4.130 người, ngưỡng Youden)

Mô hình	ROC-AUC	PR-AUC	Brier	F1	Nhạy	Đặc hiệu	Huấn luyện
Hồi quy logistic	0,817	0,515	0,125	0,525	0,767	0,715	0,03 s
KNN (k=51)	0,812	0,514	0,125	0,525	0,660	0,790	0,02 s
Naive Bayes Gauss	0,775	0,391	0,209	0,489	0,756	0,670	0,02 s
Cây quyết định (sâu 6)	0,785	0,444	0,133	0,504	0,712	0,725	0,04 s
Rừng ngẫu nhiên	**0,823**	**0,528**	**0,123**	**0,540**	0,726	0,763	0,65 s
Gradient Boosting	0,820	0,527	**0,123**	0,531	0,723	0,753	0,20 s
SVM (RBF)	0,821	0,505	0,178	0,510	0,818	0,658	24,1 s
MLP (64–32)	0,821	0,527	0,124	0,515	**0,837**	0,651	1,53 s

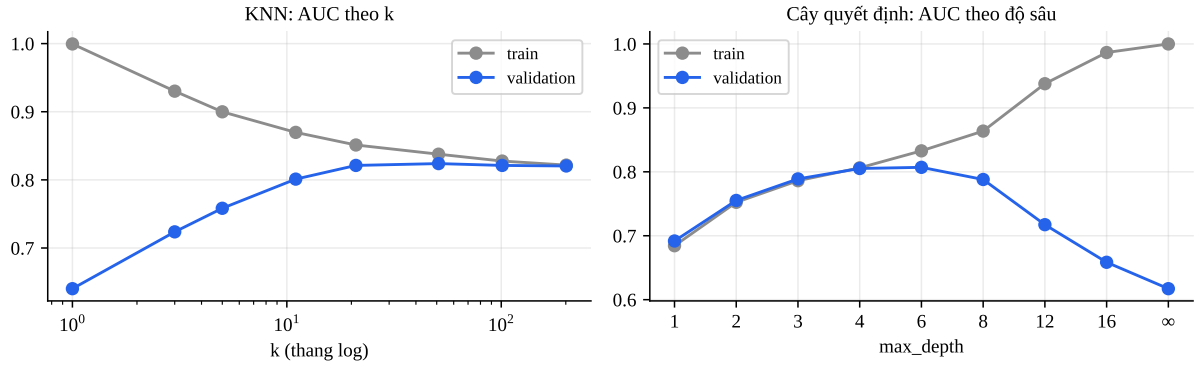


Hình 2.3. Đường ROC và Precision–Recall của tám mô hình trên tập kiểm tra

Nhận xét. Năm mô hình mạnh (RF, MLP, SVM, GBDT, LR) nằm trong khoảng 0,817–0,823 AUC. Chênh lệch 0,006 nhỏ hơn độ lệch chuẩn 0,005–0,006 của kiểm định chéo 5 phần trên tập train (LR 0,826 ± 0,006; GBDT 0,830 ± 0,006). Có thể kết luận: *với 13 yếu tố nguy cơ tự khai, độ khó của bài toán đã chạm trần thông tin*. Phần còn lại nằm ở glucose và HbA1c vốn không được dùng. Mức AUC ≈ 0,82 tương đương các thang điểm sàng lọc dựa trên NHANES trong y văn.

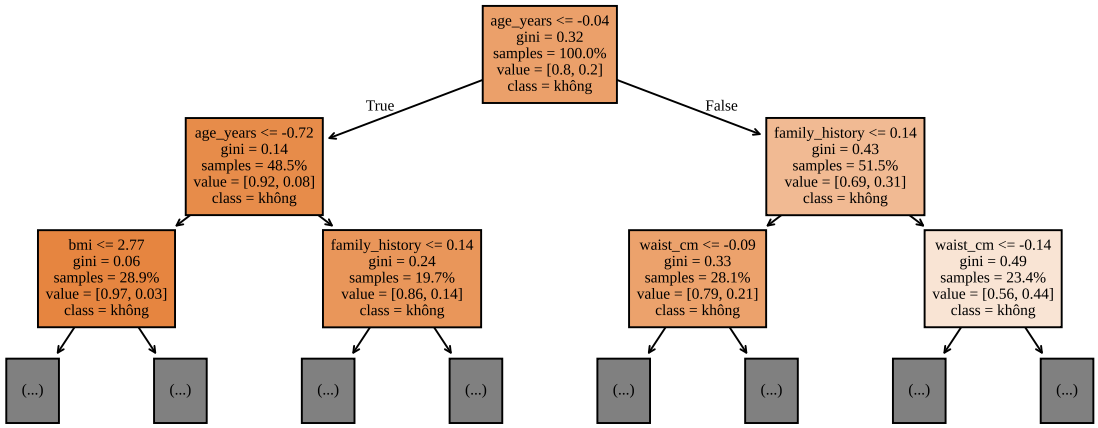
Hai mô hình yếu rõ rệt đều do vi phạm giả định. Naive Bayes giả định độc lập có điều kiện, nhưng BMI–vòng eo và hai chỉ số huyết áp tương quan mạnh nên bằng chứng bị "đếm hai lần". Xác suất bị đẩy về hai cực, thể hiện ở Brier tệ nhất (0,209) và ngưỡng Youden chỉ 0,002. Cây đơn lẻ sâu 6 có phương sai cao và biên quyết định bậc thang. Rừng ngẫu nhiên lấy trung bình 400 cây như vậy và tăng AUC từ 0,785 lên 0,823, minh họa trực tiếp công thức giảm phương sai.

Ở ngưỡng Youden, các mô hình có điểm vận hành khác nhau. MLP và SVM nghiêng về độ nhạy (0,84 và 0,82), còn KNN nghiêng về độ đặc hiệu. Trong sàng lọc, bỏ sót người bệnh (FN) nguy hiểm hơn báo động nhầm (FP), nên ngưỡng có thể hạ thêm. Đó là quyết định lâm sàng chứ không phải kỹ thuật. Xác suất của các mô hình ngoài NB và SVM được hiệu chuẩn tốt (Brier ≈ 0,123–0,125, so với 0,159 của dự đoán hằng số 0,198).

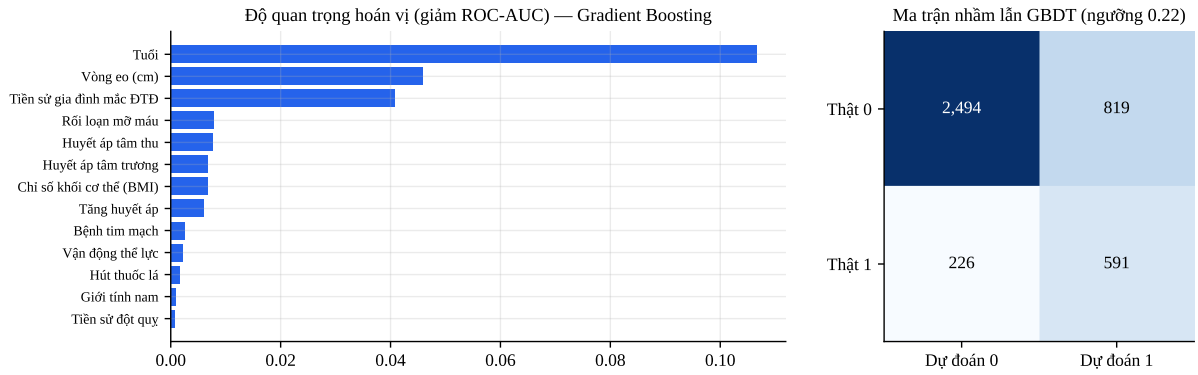


Hình 2.4. Đánh đổi độ chệch – phương sai: AUC train và validation theo k của KNN và độ sâu cây

Hình 2.4 cho thấy đánh đổi độ chệch – phương sai rất rõ. Với $k = 1$, KNN "thuộc lòng" tập train (AUC 0,9995) nhưng chỉ đạt 0,640 trên validation. Đường validation đạt đỉnh 0,824 ở $k = 51$ rồi đi ngang. Cây quyết định cũng vậy: AUC validation tăng tới 0,807 ở độ sâu 6. Ở độ sâu 16, train đạt 0,987 còn validation tụt xuống 0,659. Cây không giới hạn độ sâu đạt 1,000 trên train và 0,617 trên validation.



Hình 2.5. Ba tầng đầu của cây quyết định: câu hỏi đầu tiên là tuổi, sau đó là tuổi (nhánh trẻ) và tiền sử gia đình (nhánh lớn tuổi)



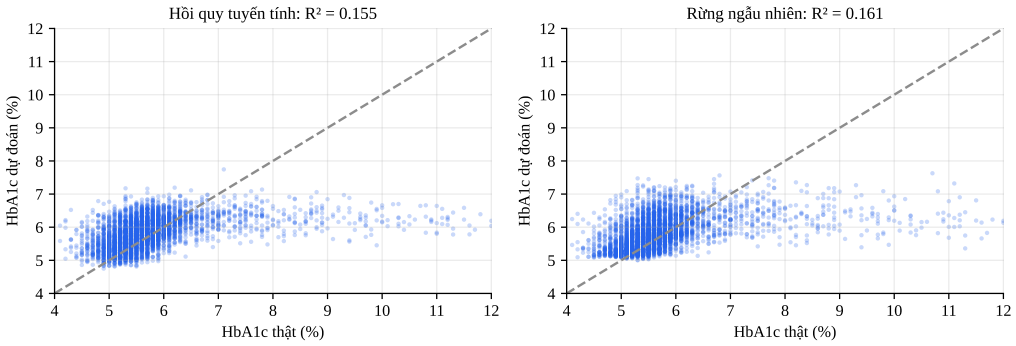
Hình 2.6. Độ quan trọng hoán vị của các đặc trưng (Gradient Boosting) và ma trận nhầm lẫn

Giải thích mô hình. Ba cách nhìn độc lập cho cùng một thứ tự. Thứ nhất, câu hỏi gốc của cây là "tuổi ≤ 49,5?". Nhánh trẻ hỏi tiếp "tuổi ≤ 37,5?", nhánh lớn tuổi hỏi tiền sử gia đình (Hình 2.5). Thứ hai, độ quan trọng hoán vị của GBDT, tức mức giảm AUC khi xáo trộn một cột, xếp tuổi đầu tiên (0,107), rồi vòng eo (0,046) và tiền sử gia đình (0,041); các đặc trưng còn lại dưới 0,01. Thứ ba, tỉ số chênh của hồi quy logistic trên một độ lệch chuẩn là: tuổi OR = 1,93, tiền sử gia đình 1,78, vòng eo 1,56, rối loạn mỡ máu 1,25, tăng huyết áp 1,23, BMI chỉ 1,10. Vòng eo quan trọng hơn BMI khi có mặt cả hai, đúng với kiến thức y khoa rằng béo bụng (mỡ nội tạng) phản ánh đề kháng insulin tốt hơn. Đây cũng là lý do chatbot hỏi thêm vòng eo. Vận động có OR = 0,87 < 1, tức là yếu tố bảo vệ. Ma trận nhầm lẫn của GBDT trên 4.130 người: 591 TP, 226 FN, 819 FP, 2.494 TN.

2.6.2. Hồi quy HbA1c

Bảng 2.5. Dự đoán HbA1c (%) từ 13 yếu tố nguy cơ (20.413 người có HbA1c, chia 80/20)

Mô hình	RMSE (%)	MAE (%)	R^2
Dự đoán bằng trung bình (đối chứng)	1,072	0,636	0,000
Hồi quy tuyến tính	0,984	0,569	0,155
Ridge (α=10)	0,984	0,569	0,155
SVR (RBF, ϵ=0,1; 8.000 mẫu)	1,000	**0,515**	0,128
Rừng ngẫu nhiên	**0,981**	0,558	**0,161**

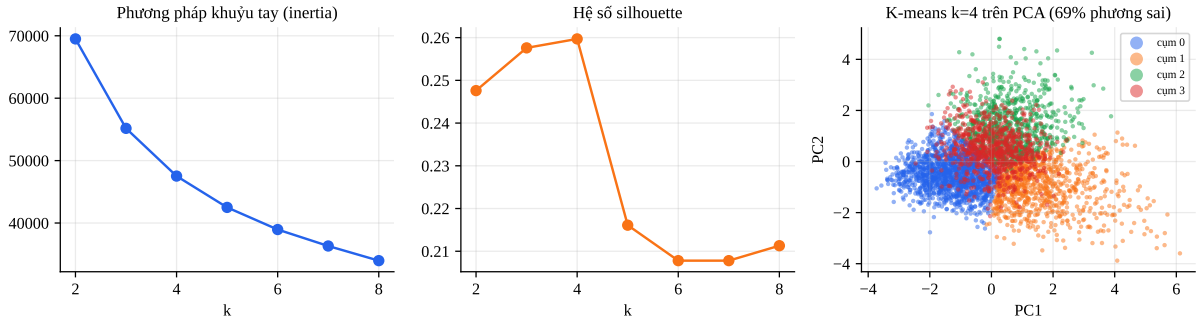


Hình 2.7. HbA1c dự đoán so với thực tế: mô hình co về trung bình

R^2 chỉ đạt 0,16. Yếu tố nguy cơ giải thích được khoảng 16% phương sai của HbA1c, phù hợp với kết luận "chạm trần thông tin" ở trên. Hình 2.7 cho thấy hiện tượng co về trung bình: mọi dự đoán nằm trong khoảng 5–7%, trong khi HbA1c thật trải tới 14%.

Đuôi phải dài chủ yếu là người ĐTĐ đang điều trị. SVR có MAE tốt nhất nhưng RMSE tệ nhất. Hàm mất mát ϵ-không nhạy tương tự trung vị nên khớp tốt phần đông người bình thường và bỏ qua các ca cực trị, trong khi RMSE phạt nặng các ca này. Hệ số hồi quy tuyến tính trên một độ lệch chuẩn: tuổi +0,18%, tiền sử gia đình +0,18%, vòng eo +0,11%, mỡ máu +0,10%, vận động −0,06%. Thứ tự này trùng với bài toán phân loại. Kết luận thực tiễn: *không thể thay xét nghiệm HbA1c bằng mô hình*. Mô hình chỉ giúp quyết định *ai* nên đi xét nghiệm.

2.6.3. Gom cụm không giám sát



Hình 2.8. Chọn k bằng inertia và silhouette; bốn cụm chiếu trên hai thành phần chính

Gom cụm K-means được chạy trên 5 biến liên tục (tuổi, BMI, vòng eo, hai chỉ số huyết áp) đã chuẩn hóa, *không dùng nhãn*. Silhouette cao nhất ở $k = 4$ (0,260), khuỷu tay của inertia cũng ở quanh đây. Hai thành phần chính giữ 69% phương sai. Điểm đáng chú ý là các cụm học không giám sát lại có ý nghĩa lâm sàng (Bảng 2.6). Cụm 0 gồm người trẻ, gầy, huyết áp thấp, tỉ lệ ĐTĐ chỉ 4,4%. Ba cụm còn lại là ba "kiểu hình" nguy cơ khác nhau: béo phì nặng ở tuổi trung niên, tăng huyết áp, và cao tuổi. Tỉ lệ ĐTĐ của ba cụm này cao gấp 5–7 lần cụm 0.

Bảng 2.6. Hồ sơ trung bình của bốn cụm K-means và tỉ lệ ĐTĐ (nhãn không dùng khi gom cụm)

Cụm	Diễn giải	Số người	Tuổi	BMI	Eo	HA tâm thu	Tỉ lệ ĐTĐ
0	Trẻ, khỏe	7.038	35,6	25,3	88,0	112,1	4,4%
1	Béo phì nặng	3.825	45,7	39,3	122,2	124,9	27,2%
2	Tăng huyết áp	3.849	58,4	27,9	98,2	145,6	22,2%
3	Cao tuổi	4.139	68,8	28,1	100,3	126,6	31,3%

2.6.4. So sánh tổng hợp

Bảng 2.7. So sánh định tính các thuật toán trong bối cảnh hệ thống

Thuật toán	**Dễ giải thích**	**Chịu dữ liệu thiếu**	**Vai trò trong hệ thống**
Hồi quy logistic	Cao (OR)	Cần điền	Đối chứng; giải thích yếu tố nguy cơ
KNN	Trung bình	Cần điền	Không dùng (chậm khi dữ liệu lớn)
Naive Bayes	Cao	Tự nhiên	Không dùng (hiệu chuẩn kém)
Cây quyết định	Rất cao	Cần điền	Minh họa luật cho người dùng
Rừng ngẫu nhiên	Thấp	Cần điền	Mô hình dự phòng
Gradient Boosting	Thấp (cần SHAP/occlusion)	Tự nhiên	**Mô hình ML của PredictionAgent**
SVM	Thấp	Cần điền	Không dùng (chậm, xác suất kém)
MLP	Thấp	Cần điền + cờ thiếu	**Mô hình DL của PredictionAgent**

PredictionAgent dùng hai mô hình bổ sung cho nhau: GBDT (tự xử lý NaN, nhanh) và MLP PyTorch (128–64–32, có cờ thiếu dữ liệu cho từng đặc trưng). Cả hai được huấn luyện với kỹ thuật tăng cường "che ngẫu nhiên 40% đặc trưng không bắt buộc" để mô phỏng người dùng chỉ khai một phần. Trên cùng tập kiểm tra, bản production đạt AUC 0,817 (GBDT) và 0,829 (MLP). Khi bị che 40% đặc trưng, AUC chỉ giảm còn 0,802 và 0,809. Trên nhóm người gốc Á (561 người), AUC đạt 0,833 và 0,850. Trung bình hai mô hình là 0,827. Khi hai mô hình lệch nhau ≥ 15 điểm phần trăm (xảy ra ở 8% số ca), ConsensusCriticAgent mở chất vấn (Chương 7).

2.7. Triển khai

Endpoint `POST /api/lab/ch2/nhanes` nhận 13 đặc trưng; trường nào để trống được coi là thiếu. Endpoint trả về xác suất ĐTĐ của bảy mô hình đã huấn luyện (tệp `data/models/ch2_classic.joblib`, nén 3 MB), cùng ngưỡng Youden của từng mô hình và đóng góp của từng đặc trưng theo hồi quy logistic. Trên trang `/lab`, thẻ "Chương 2" vẽ biểu đồ cột so sánh bảy xác suất. Người dùng thấy ngay các mô hình mạnh đồng thuận còn Naive Bayes thì cực đoan. Mô hình production (GBDT + MLP) được dùng trong

luồng chat tại `/api/chat`.

Đoạn mã 2.2. Gọi endpoint Chương 2 (rút gọn)

```
curl -s https://agent.nits.io.vn/api/lab/ch2/nhanes -H 'Content-Type: application/json' \
  -d
   ↪  '{"age_years":58,"sex":0,"bmi":29,"waist_cm":95,"high_bp":1,"high_chol":1,"family_history":1}'
```

2.8. Tiểu kết chương 2

Chương 2 trình bày khung chung của học máy: cực tiểu rủi ro thực nghiệm có điều chuẩn, tối ưu bằng gradient, và đánh giá bằng thước đo phù hợp với bài toán. Thực nghiệm trên 20.646 người NHANES cho ba kết luận. (1) Với yếu tố nguy cơ tự khai, các mô hình tốt đều hội tụ về AUC ≈ 0,82. Chọn đặc trưng và dữ liệu quan trọng hơn chọn thuật toán. (2) Giả định của mô hình quyết định thành bại: Naive Bayes thất bại vì đặc trưng tương quan, SVM thất bại khi không xử lý lệch lớp, cây đơn lẻ thất bại vì phương sai cao. (3) Tuổi, vòng eo và tiền sử gia đình là ba yếu tố nguy cơ chính. Kết luận này nhất quán giữa cây, độ quan trọng hoán vị và tỉ số chênh, và khớp với hướng dẫn ADA. Mô hình ước lượng được *nguy cơ* nhưng không thay được *xét nghiệm*. Ở các chương sau, hệ thống bổ sung các nguồn dữ liệu khác: ảnh (Chương 3), chuỗi thời gian (Chương 4) và tri thức văn bản (Chương 5).

CHƯƠNG 3. MẠNG NƠ-RON TÍCH CHẬP (CNN)

Bệnh võng mạc đái tháo đường (DR) là nguyên nhân hàng đầu gây mù lòa ở người trong độ tuổi lao động. ADA khuyến cáo khám đáy mắt ngay khi chẩn đoán ĐTĐ típ 2 và định kỳ 1–2 năm sau đó [2]. Năm 2016, Gulshan và cộng sự [42] cho thấy một CNN có thể phát hiện DR từ ảnh đáy mắt với độ nhạy ngang bác sĩ nhãn khoa. Chương này phân tích cấu trúc của CNN theo từng lớp, gồm phép tích chập, hàm kích hoạt, gộp, chuẩn hóa theo lô, dropout và lớp phân loại. Sau đó chương huấn luyện bốn mô hình phân độ DR trên bộ RetinaMNIST.

3.1. Động lực của CNN

Một ảnh RGB 128×128 có 49.152 giá trị. Nếu nối thẳng vào một lớp kết nối đầy đủ 512 nơ-ron, riêng lớp đầu đã cần 25 triệu trọng số. Quan trọng hơn, cách làm này *bỏ qua cấu trúc không gian*: dịch ảnh sang phải một điểm ảnh, vector đầu vào thay đổi hoàn toàn. CNN dựa trên ba quan sát về ảnh tự nhiên, lấy cảm hứng từ các tế bào thị giác có trường tiếp nhận cục bộ mà Hubel và Wiesel mô tả [43]:

- **Kết nối cục bộ**: một đặc trưng (cạnh mạch máu, chấm xuất huyết) chỉ phụ thuộc vào vùng lân cận nhỏ.
- **Chia sẻ trọng số**: bộ dò "vi phình mạch" hữu ích ở mọi vị trí trên ảnh, nên dùng chung một bộ lọc. Phép tích chập vì thế *đẳng biến với phép tịnh tiến*.
- **Phân cấp**: cạnh → hình dạng → tổn thương → chẩn đoán. Các lớp sâu tổ hợp đặc trưng của lớp nông.

LeNet-5 (1998) [25] là CNN thương mại đầu tiên. AlexNet (2012) [29] mở ra kỷ nguyên học sâu nhờ ReLU, dropout và GPU.

3.2. Các lớp của CNN

3.2.1. Lớp tích chập

Với đầu vào $\mathbf{X} \in \mathbb{R}^{C_{in} \times H \times W}$ và C_{out} bộ lọc $\mathbf{K}^{(o)} \in \mathbb{R}^{C_{in} \times k \times k}$, bản đồ đặc trưng đầu ra tại kênh o, vị trí (i, j) là:

$$\mathbf{Y}_{o,i,j} = b_o + \sum_{c=1}^{C_{in}} \sum_{u=0}^{k-1} \sum_{v=0}^{k-1} \mathbf{K}^{(o)}_{c,u,v} \, \mathbf{X}_{c,\, si+u-p,\, sj+v-p}, \tag{3.1}$$

với s là bước trượt (stride) và p là số điểm ảnh đệm (padding). Thư viện học sâu thực chất tính *tương quan chéo* (không lật bộ lọc), nhưng vẫn gọi là tích chập. Kích thước đầu ra là

$$H_{out} = \left\lfloor \frac{H + 2p - k}{s} \right\rfloor + 1, \tag{3.2}$$

nên bộ lọc 3×3 với $p = 1, s = 1$ giữ nguyên kích thước (đệm "same"). Số tham số là $C_{out}(C_{in}k^2 + 1)$, *không phụ thuộc kích thước ảnh*. Đó là lợi thế quyết định so với MLP.

Ví dụ tính tay. Ảnh xám 4×4 và bộ lọc dò cạnh dọc 3×3, $s = 1$, không đệm, cho đầu ra 2×2:

$$\mathbf{X} = \begin{bmatrix} 1 & 1 & 0 & 0 \\ 1 & 1 & 0 & 0 \\ 1 & 1 & 0 & 0 \\ 1 & 1 & 0 & 0 \end{bmatrix}, \quad \mathbf{K} = \begin{bmatrix} 1 & 0 & -1 \\ 1 & 0 & -1 \\ 1 & 0 & -1 \end{bmatrix} \Rightarrow \mathbf{Y} = \begin{bmatrix} 3 & 3 \\ 3 & 3 \end{bmatrix}.$$

Phần tử $\mathbf{Y}_{0,0} = (1 + 1 + 1) \cdot 1 + (1 + 1 + 1) \cdot 0 + (0 + 0 + 0) \cdot (-1) = 3$. Bộ lọc phản hồi mạnh ở nơi cường độ chuyển từ sáng sang tối theo chiều ngang, tức là một cạnh dọc. Trong ảnh đáy mắt, các bộ lọc lớp đầu học được tương tự để bắt mép mạch máu và rìa đĩa thị.

3.2.2. Hàm kích hoạt

Nếu không có hàm phi tuyến, chồng nhiều lớp tích chập vẫn chỉ là một phép tuyến tính. ReLU $f(z) = \max(0, z)$ có đạo hàm bằng 1 khi $z > 0$, nên tránh được bão hòa gradient của sigmoid và tính rất rẻ. Một số biến thể: Leaky ReLU (độ dốc nhỏ khi $z < 0$); hard-swish $z \cdot \text{ReLU6}(z + 3)/6$, xấp xỉ rẻ của swish, dùng trong MobileNetV3 [44].

3.2.3. Lớp gộp

Gộp cực đại (max pooling) 2×2, bước 2, lấy giá trị lớn nhất trong mỗi ô: $\mathbf{Y}_{c,i,j} = \max_{u,v \in \{0,1\}} \mathbf{X}_{c,2i+u,2j+v}$. Lớp này giảm một nửa kích thước mỗi chiều, không có tham số, và tạo tính bất biến cục bộ với dịch chuyển nhỏ. Khi lan truyền ngược, gradient chỉ đi qua vị trí đạt cực đại. *Gộp trung bình toàn cục* (GAP) lấy trung bình cả bản đồ $H \times W$ thành một số cho mỗi kênh. GAP thay cho việc "làm phẳng" trước lớp phân loại, giảm mạnh số tham số và cho phép đầu vào có kích thước tùy ý.

3.2.4. Chuẩn hóa theo lô

Batch Normalization [45] chuẩn hóa từng kênh theo thống kê của lô $\mathcal{B}$, rồi cho mạng học lại thang đo γ và độ dời β:

$$\hat{x} = \frac{x - \mu_{\mathcal{B}}}{\sqrt{\sigma^2_{\mathcal{B}} + \epsilon}}, \qquad y = \gamma \hat{x} + \beta. \tag{3.3}$$

Khi suy luận, μ và σ^2 được thay bằng trung bình động tích lũy lúc huấn luyện. BN giúp dùng được tốc độ học lớn hơn, giảm nhạy cảm với khởi tạo, và có tác dụng điều chuẩn nhẹ. Vì β đã đóng vai trò độ lệch, lớp tích chập đứng trước BN không cần bias (`bias=False`).

3.2.5. Dropout, lớp kết nối đầy đủ và softmax

Dropout [46] tắt ngẫu nhiên mỗi nơ-ron với xác suất p khi huấn luyện, buộc mạng không phụ thuộc vào một vài đặc trưng; lúc suy luận dùng toàn bộ mạng. Lớp cuối là một phép biến đổi tuyến tính $\mathbf{z} = \mathbf{W}\mathbf{h} + \mathbf{b}$ cho ra $K = 5$ *logit*, đưa qua softmax $p_k = e^{z_k} / \sum_j e^{z_j}$ để có phân phối xác suất trên 5 mức DR.

3.2.6. Hàm mất mát và huấn luyện

Hàm mất mát là entropy chéo có trọng số lớp và làm mịn nhãn (label smoothing) [47]:

$$\begin{aligned} \mathcal{L} &= -\frac{1}{\sum_i w_{y_i}} \sum_i w_{y_i} \sum_{k=1}^{K} \tilde{y}_{ik} \log p_{ik}, \\ \tilde{y}_{ik} &= (1 - \varepsilon)\, \mathbb{1}[k = y_i] + \frac{\varepsilon}{K}, \qquad w_c = \frac{N}{K\, N_c}. \end{aligned} \tag{3.4}$$

Trọng số w_c tỉ lệ nghịch với số mẫu của lớp c, bù cho lệch lớp nặng của dữ liệu. Làm mịn nhãn với $\varepsilon = 0{,}05$ ngăn mô hình quá tự tin. Điều này hợp lý vì ranh giới giữa hai mức DR liền kề vốn không rõ ràng ngay cả với chuyên gia. Gradient của $\mathcal{L}$ theo logit có dạng rất gọn, $\partial \mathcal{L} / \partial z_k = p_k - \tilde{y}_k$ (với một mẫu). Gradient này lan ngược qua các lớp bằng quy tắc chuỗi. Gradient theo bộ lọc tích chập chính là tương quan chéo giữa đầu vào và gradient đầu ra, nên cũng tính được bằng một phép tích chập. Tối ưu bằng AdamW [48], tức Adam với suy giảm trọng số tách rời, cùng lịch học cosine.

3.2.7. Tăng cường dữ liệu và học chuyển giao

Với vài nghìn ảnh, CNN rất dễ quá khớp. *Tăng cường dữ liệu* tạo biến thể hợp lệ của ảnh ở mỗi epoch. Ảnh đáy mắt không có hướng "trên–dưới" cố định, nên lật ngang, lật dọc, xoay ±20° và thay đổi độ sáng/độ tương phản đều giữ nguyên nhãn. *Học chuyển*

giao khởi tạo mạng bằng trọng số đã học trên ImageNet [28] (1,28 triệu ảnh). Các lớp đầu đã biết dò cạnh và kết cấu, chỉ cần thay lớp phân loại 1000 lớp bằng lớp 5 lớp rồi tinh chỉnh toàn mạng với tốc độ học nhỏ.

3.2.8. Các kiến trúc tiêu biểu

Bảng 3.1. Các kiến trúc CNN tiêu biểu

Kiến trúc (năm)	**Số lớp**	**Tham số**	**Ý tưởng chính**
LeNet-5 (1998) [25]	7	60 nghìn	Conv–pool xen kẽ, đọc chữ số trên séc
AlexNet (2012) [29]	8	60 triệu	ReLU, dropout, huấn luyện trên GPU
VGG-16 (2014) [49]	16	138 triệu	Chỉ dùng conv 3×3 xếp chồng
ResNet (2015) [50]	18–152	11–60 triệu	Kết nối tắt $\mathbf{y} = \mathcal{F}(\mathbf{x}) + \mathbf{x}$
MobileNetV3 (2019) [44]	–	5,4 triệu	Tích chập tách sâu, khối SE, hard-swish

ResNet giải quyết hiện tượng *suy thoái*: mạng sâu hơn lại cho lỗi huấn luyện cao hơn. Mỗi khối học phần dư $\mathcal{F}(\mathbf{x})$ thay vì ánh xạ đầy đủ, và cộng với đầu vào qua kết nối tắt. Đạo hàm $\partial\mathbf{y}/\partial\mathbf{x} = \partial\mathcal{F}/\partial\mathbf{x} + \mathbf{I}$ luôn chứa thành phần đơn vị, nên gradient truyền thẳng về các lớp đầu. ResNet-18 gồm một lớp conv 7×7, bốn tầng (mỗi tầng 2 khối cơ sở, mỗi khối 2 conv 3×3 + BN + ReLU, số kênh $64 \to 128 \to 256 \to 512$), rồi GAP và FC. **MobileNetV3** tách tích chập chuẩn thành *depthwise* (mỗi kênh một bộ lọc $k \times k$) và *pointwise* (conv 1×1 trộn kênh). Chi phí giảm khoảng $1/C_{out} + 1/k^2$ lần. Khối *squeeze-and-excitation* học trọng số cho từng kênh. Kiến trúc này được thiết kế cho thiết bị di động, phù hợp với máy chủ chỉ có CPU.

3.2.9. Giải thích bằng Grad-CAM

Trong y tế, một dự đoán cần đi kèm lý do. Grad-CAM [51] tính tầm quan trọng của kênh k ở lớp tích chập cuối đối với lớp c bằng trung bình gradient, rồi tổng hợp có trọng

số các bản đồ đặc trưng:

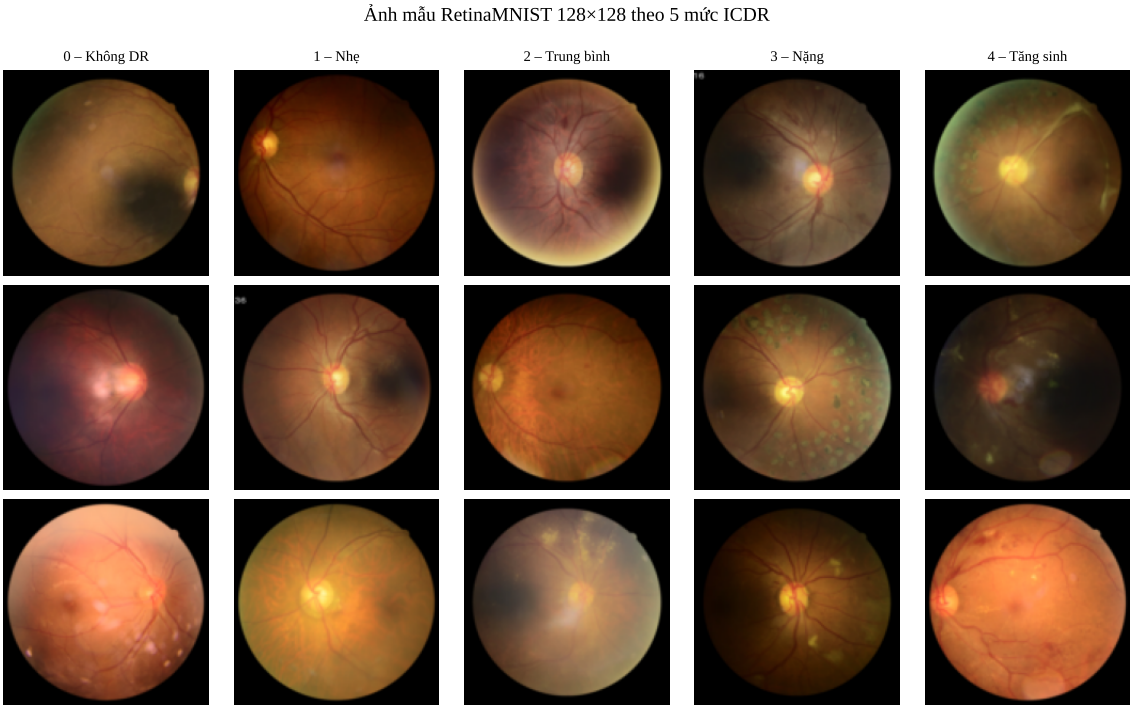
$$\alpha_k^c = \frac{1}{HW}\sum_{i,j}\frac{\partial z_c}{\partial A_{ij}^k}, \qquad L_{\text{Grad-CAM}}^c = \text{ReLU}\Big(\sum_k \alpha_k^c A^k\Big). \tag{3.5}$$

Bản đồ nhiệt được nội suy về kích thước ảnh và phủ lên ảnh gốc, cho thấy vùng nào "thuyết phục" mô hình chọn mức c.

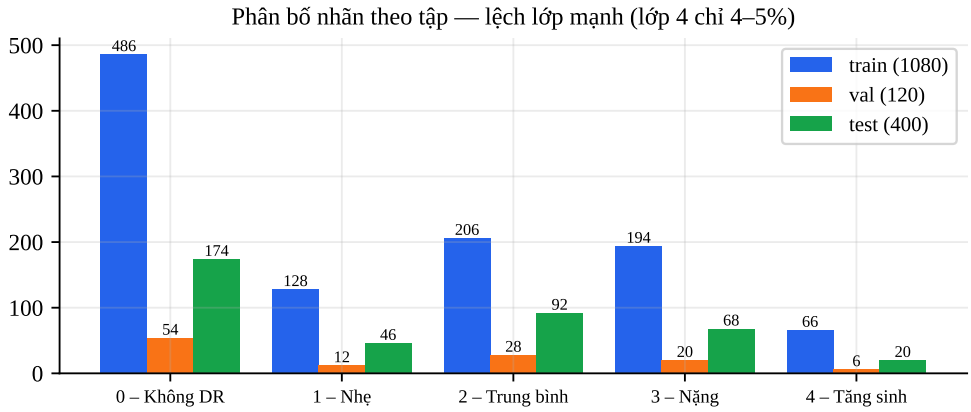
3.3. Dữ liệu RetinaMNIST

RetinaMNIST thuộc bộ MedMNIST v2 [6], được tạo từ bộ dữ liệu của cuộc thi DeepDRiD (ISBI 2020) [52]. Mỗi ảnh đáy mắt được bác sĩ phân độ theo thang quốc tế ICDR [53]: 0 – không DR; 1 – nhẹ (chỉ có vi phình mạch); 2 – trung bình; 3 – nặng; 4 – tăng sinh (tân mạch).

- **Nguồn**: `https://medmnist.com`; tệp `retinamnist_128.npz` tại Zenodo `https://zenodo.org/records/10519652`. Giấy phép CC BY 4.0.
- **Kích thước**: 1.600 ảnh RGB 128×128, chia sẵn 1.080 / 120 / 400 (train/val/test); tệp nén 46 MB.
- **Định dạng**: mảng `uint8` dạng $N \times 128 \times 128 \times 3$, nhãn số nguyên 0–4.



Hình 3.1. Ba ảnh mẫu của mỗi mức DR trong RetinaMNIST



Hình 3.2. Phân bố nhãn theo tập: lớp 0 chiếm 45%, lớp 4 chỉ 5%

Bảng 3.2. Số ảnh theo mức DR và trọng số lớp dùng trong hàm mất mát

Tập	Mức 0	Mức 1	Mức 2	Mức 3	Mức 4	Tổng
Train	486	128	206	194	66	1.080
Validation	54	12	28	20	6	120
Test	174	46	92	68	20	400
Trọng số w_c	0,44	1,69	1,05	1,11	3,27	

Nhận xét. (i) *Lệch lớp nặng*: mức 0 chiếm 45%, mức 4 (nguy hiểm nhất) chỉ 6% tập train. Một mô hình luôn đoán "mức 0" đã đạt accuracy 43,5% trên test. Vì vậy cần các thước đo không bị lớp đa số chi phối: macro-F1, và đặc biệt là *Quadratic Weighted Kappa* (QWK) [54]. QWK là thước đo chuẩn của các cuộc thi DR. Nó phạt sai số theo bình phương khoảng cách giữa hai mức: nhầm mức 4 thành 0 bị phạt 16 lần nặng hơn nhầm 4 thành 3. (ii) Ảnh có *chất lượng không đồng đều*: nhiều ảnh tối, lệch tâm hoặc bị lóa (Hình 3.1). Kênh xanh dương rất tối (trung bình 0,145 so với 0,395 của kênh đỏ), đặc trưng của ảnh đáy mắt. (iii) Tổn thương của mức 1 (vi phình mạch) chỉ rộng vài điểm ảnh ở độ phân giải gốc. Khi thu về 128×128, nhiều chi tiết bị mất, nên phân biệt mức 0 với mức 1 rất khó ngay từ dữ liệu. (iv) Tập validation chỉ có 120 ảnh (6 ảnh mức 4), nên việc chọn epoch tốt nhất có phương sai lớn.

3.4. Cài đặt

Bốn mô hình dùng chung tăng cường dữ liệu, hàm mất mát (3.4), AdamW, lịch cosine, kích thước lô 32. Epoch tốt nhất được chọn theo QWK trên validation. Huấn luyện chạy trên GPU RTX 3050; độ trễ suy luận được đo trên CPU, giống môi trường

deploy.

(1) MLP đối chứng thu ảnh về 64×64, làm phẳng thành 12.288 chiều, rồi qua các lớp 512 → 128 → 5 (ReLU, dropout 0,3). Mô hình này kiểm chứng việc bỏ qua cấu trúc không gian.

(2) CNN tự thiết kế gồm bốn khối *Conv 3×3* → *BN* → *ReLU* → *MaxPool 2×2* với số kênh 32 → 64 → 128 → 256, tiếp theo là GAP, dropout 0,4 và FC 5 (Đoạn mã 3.1). Bảng 3.3 liệt kê kích thước tensor và số tham số qua từng lớp. Theo (3.2), đệm 1 giữ kích thước sau mỗi conv, còn mỗi lớp gộp giảm một nửa: 128 → 64 → 32 → 16 → 8. Trường tiếp nhận của một nơ-ron sau khối 4 là 46×46 điểm ảnh, đủ bao một vùng tổn thương.

Đoạn mã 3.1. CNN tự thiết kế (`lab/ch3_cnn.py`)

```
def conv_block(c_in, c_out):
    """Conv 3×3 (đệm 1, giữ kích thước) → BatchNorm → ReLU → MaxPool 2×2 (giảm một nửa)."""
    return nn.Sequential(nn.Conv2d(c_in, c_out, 3, padding=1, bias=False),
     ↪  nn.BatchNorm2d(c_out),
                         nn.ReLU(inplace=True), nn.MaxPool2d(2))

class RetinaCNN(nn.Module):
    def __init__(self):
        super().__init__()
        self.features = nn.Sequential(conv_block(3, 32), conv_block(32, 64),
                                      conv_block(64, 128), conv_block(128, 256))
        self.head = nn.Sequential(nn.AdaptiveAvgPool2d(1), nn.Flatten(),
                                  nn.Dropout(0.4), nn.Linear(256, 5))
    def forward(self, x):                          # x: [B, 3, 128, 128]
        return self.head(self.features(x))         # logits: [B, 5]

# entropy chéo có trọng số lớp + làm mịn nhãn, AdamW + lịch cosine
loss_fn = nn.CrossEntropyLoss(weight=class_weights, label_smoothing=0.05)
opt = torch.optim.AdamW(model.parameters(), lr=1e-3, weight_decay=1e-4)
sched = torch.optim.lr_scheduler.CosineAnnealingLR(opt, T_max=epochs)
```

Bảng 3.3. Kích thước tensor và số tham số qua từng lớp của CNN tự thiết kế

Lớp	Cấu hình	Đầu ra	Tham số
Đầu vào	ảnh RGB đã chuẩn hóa ImageNet	3×128×128	0
Khối 1	Conv 3×3, 32 bộ lọc + BN + ReLU + MaxPool	32×64×64	864 + 64 = 928
Khối 2	Conv 3×3, 64 + BN + ReLU + MaxPool	64×32×32	18.432 + 128 = 18.560
Khối 3	Conv 3×3, 128 + BN + ReLU + MaxPool	128×16×16	73.728 + 256 = 73.984
Khối 4	Conv 3×3, 256 + BN + ReLU + MaxPool	256×8×8	294.912 + 512 = 295.424
GAP + Dropout(0,4)	trung bình mỗi kênh	256	0
FC + softmax	256 × 5 + 5	5	1.285
Tổng			**390.181**

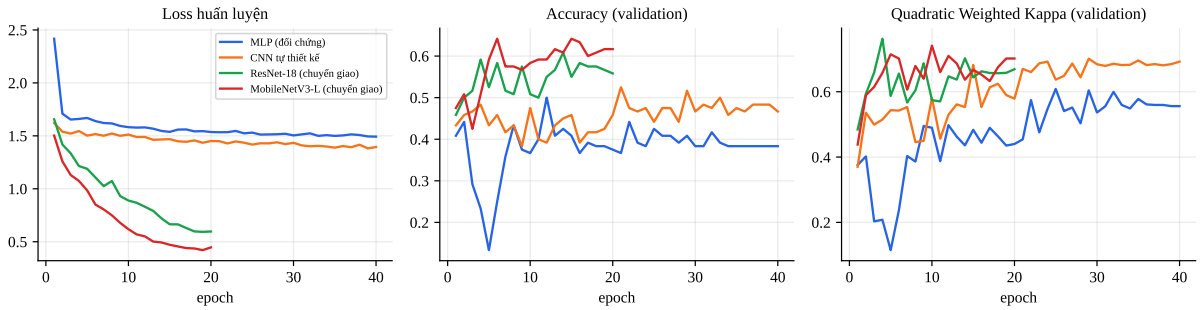
Số tham số của conv tính theo $C_{out} \cdot C_{in} \cdot 3 \cdot 3$, ví dụ khối 2 có $64 \cdot 32 \cdot 9 = 18.432$. BN có 2 tham số học ($\gamma$, β) cho mỗi kênh. CNN chỉ có 390 nghìn tham số, *ít hơn MLP 16 lần* (6,36 triệu), dù làm việc trên ảnh có độ phân giải gấp 4 lần.

(3) ResNet-18 và **(4) MobileNetV3-Large** được tải trọng số ImageNet từ torchvision. Lớp phân loại được thay bằng `Dropout(0,3) + Linear(512, 5)` (ResNet) hoặc `Linear(1280, 5)` (MobileNet), rồi tinh chỉnh toàn mạng 20 epoch với tốc độ học $3 \cdot 10^{-4}$ và $5 \cdot 10^{-4}$. Ảnh được chuẩn hóa bằng trung bình và độ lệch chuẩn của ImageNet, điều bắt buộc để trọng số tiền huấn luyện có nghĩa.

3.5. Kết quả và nhận xét

Bảng 3.4. Kết quả phân độ DR trên tập kiểm tra RetinaMNIST (400 ảnh)

Mô hình	Tham số	Acc.	Macro-F1	QWK	AUC	Huấn luyện	CPU/ảnh
Đoán toàn mức 0	0	0,435	0,121	0,000	0,500	–	–
MLP (đối chứng)	6,36 tr	0,490	0,282	0,515	0,712	16 s	1,0 ms
CNN tự thiết kế	0,39 tr	0,470	0,346	0,557	0,731	46 s	5,2 ms
ResNet-18 (chuyển giao)	11,18 tr	**0,578**	**0,482**	0,700	**0,833**	34 s	8,7 ms
MobileNetV3-L (chuyển giao)	4,21 tr	0,555	0,457	**0,710**	0,766	40 s	17,7 ms



Hình 3.3. Loss huấn luyện, accuracy và QWK trên validation theo epoch

Học chuyển giao là yếu tố quyết định. Hai mô hình tiền huấn luyện vượt xa hai mô hình học từ đầu: QWK 0,70–0,71 so với 0,52–0,56, AUC 0,83 so với 0,73. Với 1.080 ảnh, CNN học từ đầu không đủ dữ liệu để học các bộ lọc tốt. ResNet-18 thừa hưởng các bộ lọc cạnh và kết cấu từ 1,28 triệu ảnh ImageNet, nên chỉ cần 4 epoch để đạt QWK tốt nhất trên validation. Kết quả của ResNet-18 (accuracy 0,578, AUC 0,833) cao hơn mốc chuẩn công bố của MedMNIST v2 cho cùng kiến trúc và độ phân giải: accuracy 0,493 và AUC 0,717 [6]. Khác biệt chủ yếu đến từ học chuyển giao ImageNet, tăng cường dữ liệu và trọng số lớp. Huấn luyện trên GPU không hoàn toàn tất định: chạy lại cùng hạt giống cho accuracy dao động khoảng ±0,03.

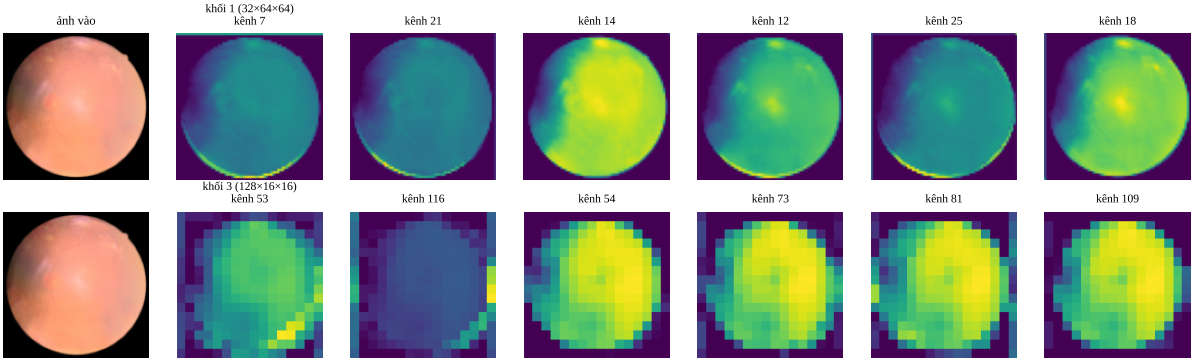
Accuracy dễ gây hiểu lầm. MLP đạt accuracy 0,490, *cao hơn* CNN tự thiết kế (0,470), nhưng macro-F1 chỉ 0,282. Ma trận nhầm lẫn cho thấy MLP dồn 345/400 dự đoán vào hai mức 0 và 3, và không bao giờ dự đoán mức 2 (0/92 ảnh mức 2 đúng). CNN phân bố dự đoán đều hơn trên cả 5 mức. Đây là lý do QWK và macro-F1 là thước đo chính.

CNN và MLP. Với ít tham số hơn 16 lần, CNN đạt QWK và macro-F1 cao hơn MLP, nhờ chia sẻ trọng số và kết nối cục bộ. Tuy vậy, CNN đạt QWK 0,70 trên validation nhưng chỉ 0,56 trên test: với 120 ảnh validation, chọn epoch theo QWK là một quá trình nhiễu. Trường hợp này cho thấy tập validation quá nhỏ cũng có thể gây quá khớp ở bước chọn mô hình.

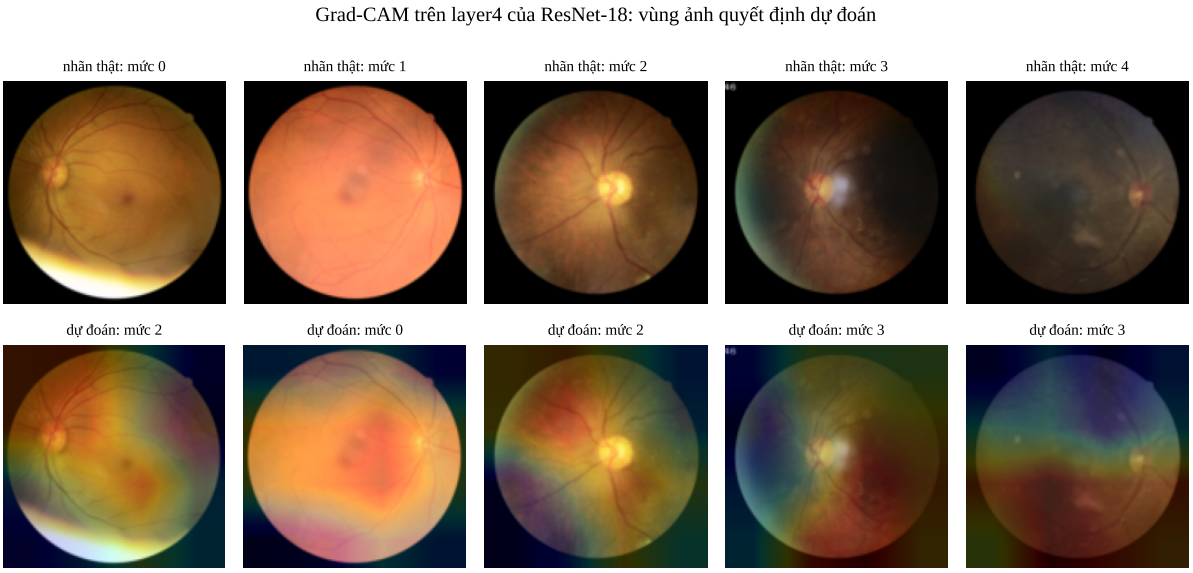


Hình 3.4. Ma trận nhầm lẫn của MobileNetV3-L trên tập kiểm tra (số ảnh và tỉ lệ theo hàng)

Phân tích lỗi. Ma trận nhầm lẫn của mô hình tốt nhất (Hình 3.4) tập trung quanh đường chéo. Trong 178 ảnh dự đoán sai, 131 ảnh lệch một mức (0↔1, 1↔2, 2↔3) và 47 ảnh lệch từ hai mức trở lên. Lỗi lệch một mức là kiểu lỗi mà QWK phạt nhẹ nhất. Mức 4 khó nhất: chỉ 4/20 ảnh đúng, phần lớn bị đoán là mức 3. Hai mức này cùng là DR nặng, đều cần chuyển chuyên khoa, nên trong sàng lọc sai số này ít hậu quả hơn so với bỏ sót bệnh (đoán thành mức 0). ResNet-18 chỉ đoán 2 trên 20 ảnh mức 4 thành mức 0.



Hình 3.5. Bản đồ đặc trưng sau khối 1 (cạnh, độ sáng) và khối 3 (vùng trừu tượng) của CNN tự thiết kế



Hình 3.6. Grad-CAM trên tầng `layer4` của ResNet-18 với một ảnh của mỗi mức DR

Mô hình "nhìn" vào đâu? Sau khối 1, các bản đồ đặc trưng giống ảnh gốc đã lọc: kênh này bắt mép đĩa thị, kênh kia bắt vùng sáng (Hình 3.5). Sau khối 3 (16×16), biểu diễn đã trừu tượng, mỗi kênh phản hồi một vùng. Grad-CAM của ResNet-18 (Hình 3.6) phần lớn rơi vào bên trong vùng võng mạc, quanh các mạch máu lớn, nơi tổn thương xuất hiện. Tuy nhiên, với một số ảnh, bản đồ nhiệt rơi vào viền tối hoặc vùng lóa, dấu hiệu mô hình còn học cả các yếu tố nhiễu do chất lượng ảnh. Đây là hạn chế cần biết trước khi tin vào dự đoán, và là lý do hệ thống luôn khuyến cáo khám nhãn khoa thay vì kết luận.

3.6. Triển khai

Mô hình triển khai được chọn theo QWK trên *validation*, không theo test, để tránh dùng tập kiểm tra cho việc chọn mô hình. Mô hình đó là ResNet-18 (val QWK 0,762), nặng 45 MB và mất 8,7 ms mỗi ảnh trên CPU (khoảng 150–300 ms cho cả lượt gọi API vì gồm giải mã ảnh và tính Grad-CAM). Endpoint `POST /api/lab/ch3/retina` nhận một ảnh JPEG/PNG (multipart). Ảnh được đưa về 128×128 và chuẩn hóa như lúc huấn luyện. Endpoint trả về xác suất 5 mức, mức dự đoán, khuyến cáo theo ADA, và ảnh Grad-CAM (PNG base64). `GET /api/lab/ch3/sample/{i}` trả về ảnh thứ *i* của tập kiểm tra cùng nhãn thật, để thử nhanh trên trang `/lab`. Trong hệ đa tác tử, PredictionAgent dùng cùng lớp `CNNRetinopathyEngine` (`src/models/cnn_vision.py`) khi hồ sơ có ảnh đáy mắt, gửi kèm dưới dạng base64 qua API đánh giá trực tiếp `POST /api/consultation`. Ảnh tải lên trong khung chat hiện vẫn được DocumentAgent đọc như phiếu xét nghiệm; tự nhận diện ảnh đáy mắt là hướng phát triển. Kết quả CNN chỉ là bằng chứng tham khảo, không thay đổi mức nguy cơ ĐTĐ.

Đoạn mã 3.2. Gọi endpoint Chương 3 với một ảnh đáy mắt

```
curl -s https://agent.nits.io.vn/api/lab/ch3/retina -F "file=@fundus.png"
# Ảnh mẫu số 8 của tập kiểm tra (nhãn thật: mức 4)
# {"grade": 3, "label": "3 - Bệnh võng mạc không tăng sinh nặng",
#  "probabilities": [0.0021, 0.0049, 0.0698, 0.8566, 0.0666],
#  "advice": "Chuyển bác sĩ nhãn khoa sớm.", "model": "ResNet-18 (ImageNet → RetinaMNIST)",
#  "gradcam_png": "iVBORw0KGgo...", "latency_ms": 169.0}
```

3.7. Tiểu kết chương 3

Chương 3 phân tích từng thành phần của CNN: tích chập với chia sẻ trọng số và kết nối cục bộ, ReLU, gộp, chuẩn hóa theo lô, dropout, GAP, softmax và hàm mất mát entropy chéo có trọng số. Tiếp theo là các kiến trúc từ LeNet tới ResNet và MobileNet. Thực nghiệm phân độ DR trên RetinaMNIST cho thấy: (1) CNN dùng ít tham số hơn MLP 16 lần nhưng phân loại tốt hơn nhờ khai thác cấu trúc không gian; (2) với dữ liệu y tế nhỏ, học chuyển giao từ ImageNet là yếu tố quyết định, nâng QWK từ 0,56 lên 0,70–0,71; (3) với dữ liệu lệch lớp, cần dùng QWK và macro-F1 thay cho accuracy; (4) Grad-CAM giúp kiểm tra mô hình có nhìn đúng vùng tổn thương không. Ảnh là dữ liệu tĩnh. Chương 4 chuyển sang dữ liệu có chiều thời gian: chuỗi đo đường huyết liên tục.

CHƯƠNG 4. MẠNG NƠ-RON HỒI QUY (RNN)

Cảm biến theo dõi glucose liên tục (CGM) đo đường huyết dưới da mỗi 5–15 phút, tạo ra hàng trăm điểm đo mỗi ngày. Nếu dự báo được đường huyết 30–60 phút tới, người bệnh có thời gian phòng tránh hạ hoặc tăng đường huyết. Đây là bài toán *chuỗi thời gian*: giá trị tiếp theo phụ thuộc vào cả quỹ đạo trước đó, không chỉ vào một thời điểm. Chương này trình bày mạng nơ-ron hồi quy, thuật toán lan truyền ngược theo thời gian, vấn đề biến mất gradient và hai lời giải LSTM, GRU. Sau đó chương so sánh sáu mô hình dự báo glucose trên dữ liệu CGM thật của 112 bệnh nhân.

4.1. Dữ liệu tuần tự và kiến trúc RNN

Dữ liệu tuần tự $\mathbf{x}_1, \dots, \mathbf{x}_T$ có ba đặc điểm khiến MLP và CNN không phù hợp: độ dài thay đổi, thứ tự quan trọng, và phụ thuộc có thể xa. RNN [55] xử lý chuỗi bằng một *trạng thái ẩn* $\mathbf{h}_t$, cập nhật ở mỗi bước bằng *cùng một bộ tham số*:

$$\begin{aligned} \mathbf{h}_t &= \tanh(\mathbf{W}_{xh}\mathbf{x}_t + \mathbf{W}_{hh}\mathbf{h}_{t-1} + \mathbf{b}_h), \\ \hat{\mathbf{y}}_t &= \mathbf{W}_{hy}\mathbf{h}_t + \mathbf{b}_y. \end{aligned} \tag{4.1}$$

$\mathbf{h}_t$ là bản tóm tắt có mất mát của toàn bộ quá khứ $\mathbf{x}_{1:t}$. Khi "mở" theo thời gian, RNN là một mạng sâu T lớp có trọng số dùng chung giữa các lớp. Số tham số không phụ thuộc độ dài chuỗi: với đầu vào d chiều và trạng thái ẩn m chiều, RNN có $m(d+m)+m$ tham số (PyTorch dùng hai vector bias nên là $m(d+m)+2m$).

Tùy cách nối đầu vào với đầu ra, có các dạng bài toán: *nhiều – một* (phân loại chuỗi), *một – nhiều* (sinh chuỗi), *nhiều – nhiều đồng bộ* (gán nhãn từng bước), và *nhiều – nhiều lệch pha* (dịch máy, dự báo nhiều bước). Bài toán của chương thuộc dạng *nhiều – một vector*: đọc 12 điểm gần nhất rồi xuất một vector 4 giá trị cho 15, 30, 45, 60 phút tới.

4.2. Lan truyền ngược theo thời gian và vấn đề biến mất gradient

RNN được huấn luyện bằng BPTT [56]: mở mạng thành đồ thị tính toán T bước rồi áp dụng quy tắc chuỗi. Gradient của mất mát ở bước T theo trạng thái ở bước k là một

tích các ma trận Jacobi:

$$\frac{\partial \mathscr{L}_T}{\partial \mathbf{h}_k} = \frac{\partial \mathscr{L}_T}{\partial \mathbf{h}_T} \prod_{t=k+1}^{T} \frac{\partial \mathbf{h}_t}{\partial \mathbf{h}_{t-1}}, \qquad \frac{\partial \mathbf{h}_t}{\partial \mathbf{h}_{t-1}} = \operatorname{diag}(1 - \mathbf{h}_t^2)\, \mathbf{W}_{hh}. \tag{4.2}$$

Nếu chuẩn của mỗi Jacobi nhỏ hơn 1 (trị riêng lớn nhất của $\mathbf{W}_{hh}$ nhỏ, tanh bão hòa), tích (4.2) co về 0 theo hàm mũ của $T-k$. Đó là *biến mất gradient*: mạng không học được phụ thuộc xa. Nếu chuẩn lớn hơn 1, tích tăng theo hàm mũ, gọi là *bùng nổ gradient* [57], [58]. Bùng nổ được xử lý dễ bằng *cắt gradient*: nếu $\|\mathbf{g}\| > \tau$ thì $\mathbf{g} \leftarrow \tau \mathbf{g}/\|\mathbf{g}\|$. Thực nghiệm dùng $\tau = 1$. Biến mất gradient đòi hỏi thay đổi kiến trúc.

4.3. LSTM và GRU

4.3.1. Bộ nhớ dài – ngắn hạn (LSTM)

LSTM [24] thêm một *trạng thái ô* $\mathbf{c}_t$ được cập nhật chủ yếu bằng phép cộng, cùng ba cổng sigmoid điều khiển dòng thông tin:

$$\begin{aligned}
\mathbf{f}_t &= \sigma(\mathbf{W}_f[\mathbf{h}_{t-1}, \mathbf{x}_t] + \mathbf{b}_f) && \text{cổng quên} \\
\mathbf{i}_t &= \sigma(\mathbf{W}_i[\mathbf{h}_{t-1}, \mathbf{x}_t] + \mathbf{b}_i) && \text{cổng vào} \\
\tilde{\mathbf{c}}_t &= \tanh(\mathbf{W}_c[\mathbf{h}_{t-1}, \mathbf{x}_t] + \mathbf{b}_c) && \text{nội dung ứng viên} \\
\mathbf{c}_t &= \mathbf{f}_t \odot \mathbf{c}_{t-1} + \mathbf{i}_t \odot \tilde{\mathbf{c}}_t && \text{cập nhật ô nhớ} \\
\mathbf{o}_t &= \sigma(\mathbf{W}_o[\mathbf{h}_{t-1}, \mathbf{x}_t] + \mathbf{b}_o), \qquad \mathbf{h}_t = \mathbf{o}_t \odot \tanh(\mathbf{c}_t) && \text{cổng ra}
\end{aligned} \tag{4.3}$$

Điểm mấu chốt là $\partial \mathbf{c}_t / \partial \mathbf{c}_{t-1} = \operatorname{diag}(\mathbf{f}_t)$ (bỏ qua đường gián tiếp qua $\mathbf{h}$). Khi cổng quên gần 1, gradient chảy qua nhiều bước gần như nguyên vẹn, tạo thành một "băng chuyền" (constant error carousel). Mạng tự *học* khi nào cần nhớ, khi nào cần quên. LSTM có bốn khối biến đổi nên số tham số gấp bốn RNN: $4[m(d+m)+m]$.

Trong bài toán glucose, có thể hình dung các cổng như sau. Cổng vào mở mạnh khi glucose bắt đầu tăng nhanh sau ăn. Cổng quên giữ lại xu hướng tăng hay giảm. Cổng ra quyết định phần nào của ô nhớ dùng cho dự báo.

4.3.2. Đơn vị hồi quy có cổng (GRU)

GRU [59] rút gọn LSTM: gộp trạng thái ô vào trạng thái ẩn và chỉ dùng hai cổng:

$$\begin{aligned} \mathbf{z}_t &= \sigma(\mathbf{W}_z[\mathbf{h}_{t-1}, \mathbf{x}_t]), \\ \mathbf{r}_t &= \sigma(\mathbf{W}_r[\mathbf{h}_{t-1}, \mathbf{x}_t]), \\ \tilde{\mathbf{h}}_t &= \tanh(\mathbf{W}[\mathbf{r}_t \odot \mathbf{h}_{t-1}, \mathbf{x}_t]), \\ \mathbf{h}_t &= (1 - \mathbf{z}_t) \odot \mathbf{h}_{t-1} + \mathbf{z}_t \odot \tilde{\mathbf{h}}_t. \end{aligned} \tag{4.4}$$

Cổng cập nhật $\mathbf{z}_t$ nội suy giữa trạng thái cũ và trạng thái ứng viên, đóng vai trò của cả cổng quên lẫn cổng vào. Cổng đặt lại $\mathbf{r}_t$ cho phép "quên" quá khứ khi tính nội dung mới. GRU có ba khối nên ít hơn LSTM khoảng 25% tham số.

4.3.3. Các mở rộng

RNN nhiều tầng xếp chồng các lớp hồi quy: đầu ra $\mathbf{h}_t^{(l)}$ của tầng l là đầu vào của tầng $l + 1$, giúp học biểu diễn phân cấp theo thời gian. *RNN hai chiều* chạy thêm một chiều ngược để mỗi bước thấy cả tương lai, nên chỉ dùng được khi có toàn bộ chuỗi, không dùng cho dự báo. Mô hình *mã hóa – giải mã* [60] và *cơ chế chú ý* [61] cho phép bộ giải mã nhìn lại mọi trạng thái của bộ mã hóa. Từ đó dẫn tới Transformer [62], kiến trúc bỏ hẳn tính hồi quy.

4.3.4. Đánh giá mô hình tuần tự và các bẫy thường gặp

Đánh giá chuỗi thời gian có ba bẫy dễ làm kết quả lạc quan giả tạo. *Thứ nhất, rò rỉ theo thời gian*: xáo trộn ngẫu nhiên các cửa sổ trượt khiến cửa sổ ở test chồng lấn 11/12 điểm với một cửa sổ ở train. Phải chia theo thời gian (quá khứ $\rightarrow$ tương lai) hoặc theo đơn vị độc lập (bệnh nhân). *Thứ hai, thiếu mốc đối chứng*: với chuỗi có tự tương quan cao như glucose, "ngày mai giống hôm nay" (persistence) đã có sai số nhỏ. Một mô hình phức tạp chỉ có giá trị nếu vượt rõ mốc này, và *độ trễ pha*, tức dự báo chỉ là bản sao trễ của đầu vào, phải được kiểm tra bằng mắt. *Thứ ba, thước đo không phản ánh hậu quả*: sai 20 mg/dL ở mức 60 mg/dL (bỏ sót hạ đường huyết) nguy hiểm hơn nhiều so với sai 20 mg/dL ở mức 250 mg/dL. Vì vậy ngành CGM dùng thêm MARD (sai số tương đối) và lưới Clarke (theo hậu quả lâm sàng), bên cạnh RMSE.

4.3.5. RNN và Transformer cho chuỗi thời gian

Transformer xử lý toàn bộ chuỗi song song bằng tự chú ý, mỗi bước nhìn trực tiếp mọi bước khác với đường đi độ dài 1. Nhờ vậy không có vấn đề biến mất gradient theo khoảng cách, và huấn luyện song song được trên GPU. Đổi lại, chi phí tăng theo bình phương độ dài chuỗi, và mô hình cần nhiều dữ liệu hơn vì thiếu thiên kiến quy nạp "thứ tự thời gian" (phải thêm mã hóa vị trí). Với chuỗi ngắn (12 bước), dữ liệu vừa phải và yêu cầu suy luận nhẹ trên CPU như bài toán của chương, RNN có cổng vẫn là lựa chọn hợp lý: LSTM 2 tầng chỉ có 51 nghìn tham số. Với chuỗi dài và đa biến (CGM kèm bữa ăn, insulin, vận động trong nhiều ngày), các biến thể Transformer cho chuỗi thời gian là hướng mở rộng tự nhiên.

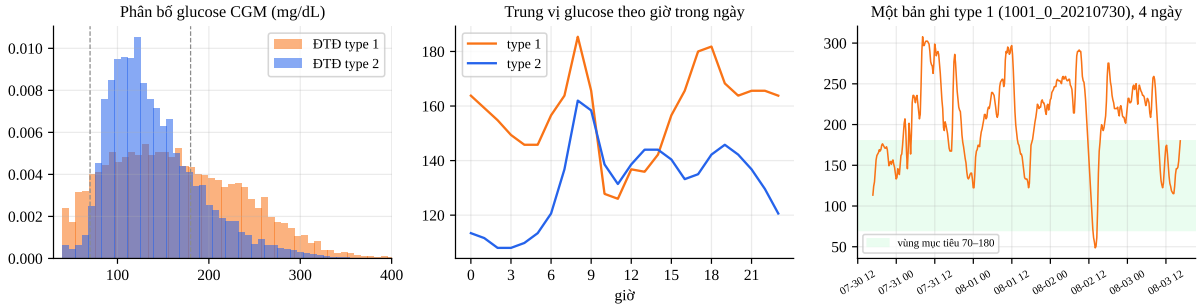
4.4. Dữ liệu ShanghaiT1DM / ShanghaiT2DM

Bộ dữ liệu được công bố trên *Scientific Data* năm 2023 [7]. Dữ liệu thu thập tại các bệnh viện ở Thượng Hải từ người bệnh đeo cảm biến FreeStyle Libre trong 3–14 ngày, kèm thông tin lâm sàng, bữa ăn và liều thuốc.

- **Nguồn**: figshare `https://doi.org/10.6084/m9.figshare.20444397`, tệp `data.zip` 3,4 MB, giấy phép CC BY 4.0.
- **Quy mô**: 12 bệnh nhân ĐTĐ típ 1 và 100 bệnh nhân típ 2, tổng cộng 125 bản ghi (một số người đeo nhiều đợt), 127.982 điểm CGM cách nhau 15 phút. Mỗi người có từ 247 đến 3.724 điểm (trung vị 1.218 điểm, khoảng 12,7 ngày).
- **Định dạng**: mỗi bản ghi một tệp Excel với các cột `Date`, `CGM (mg/dl)`, đường huyết mao mạch, bữa ăn, liều insulin. Tệp tổng hợp có 33 thuộc tính lâm sàng (tuổi, BMI, HbA1c, biến chứng…).

Bảng 4.1. Ba dòng đầu của một bản ghi CGM (bệnh nhân 1001, ĐTĐ típ 1)

Bệnh nhân	Bản ghi	Loại	Thời điểm	CGM (mg/dL)
1001	1001_0_20210730	T1	2021-07-30 16:43	113,4
1001	1001_0_20210730	T1	2021-07-30 16:58	124,2
1001	1001_0_20210730	T1	2021-07-30 17:13	129,6



Hình 4.1. Phân bố glucose theo loại ĐTĐ, trung vị theo giờ trong ngày và một bản ghi típ 1 trong 4 ngày

Bảng 4.2. Chỉ số kiểm soát đường huyết của hai nhóm theo đồng thuận quốc tế về Time in Range [63]

Nhóm	TB (mg/dL)	TIR 70–180	TBR < 70	TAR > 180	CV
Mục tiêu	–	> 70%	< 4%	< 25%	≤ 36%
ĐTĐ típ 1 (12 người)	164,8	53,6%	7,6%	38,7%	43,9%
ĐTĐ típ 2 (100 người)	140,2	78,7%	2,3%	18,9%	36,4%

Nhận xét. (i) Hai nhóm khác biệt rõ. Nhóm típ 1 dao động mạnh (CV 43,9%), chỉ 53,6% thời gian trong khoảng mục tiêu, và có 7,6% thời gian hạ đường huyết, gần gấp đôi ngưỡng an toàn. Nhóm típ 2 gần đạt mục tiêu. (ii) Đường huyết có *nhịp ngày đêm* rõ: trung vị tăng sau các bữa sáng, trưa, tối và thấp nhất vào rạng sáng. Đây là lý do thêm đặc trưng giờ trong ngày (mã hóa sin, cos để 23 giờ và 0 giờ nằm gần nhau). (iii) Có những khoảng mất tín hiệu cảm biến. Chuỗi được cắt tại mọi khoảng trống lớn hơn 20 phút để cửa sổ không nối qua chỗ hở. (iv) Nhóm típ 1 chỉ có 12 người, nên kết quả trên nhóm này có độ bất định cao.

4.5. Cài đặt

Thiết lập bài toán. Từ 12 điểm gần nhất (3 giờ), dự báo 4 điểm tiếp theo (15, 30, 45, 60 phút). Dữ liệu được chia *theo bệnh nhân*, phân tầng típ 1/típ 2: 78 người train (91.147 cửa sổ), 17 người validation (16.801), 17 người test (18.012). Chia theo cửa sổ ngẫu nhiên sẽ để các cửa sổ chồng lấn của cùng một người rơi vào cả train lẫn test, tức là rò rỉ dữ liệu. Chia theo người buộc mô hình phải tổng quát sang người chưa gặp. Glucose được chuẩn hóa z-score bằng trung bình 140,5 và độ lệch chuẩn 53,3 mg/dL của tập train.

Sáu mô hình. (1) *Persistence*: giữ nguyên giá trị cuối, mốc tối thiểu phải vượt qua. (2) *AR tuyến tính*: hồi quy Ridge trên 12 điểm, dự báo phần chênh so với điểm cuối. (3)–

(6) *RNN đơn giản, LSTM, GRU, LSTM 2 tầng* với 64 đơn vị ẩn. Đầu vào mỗi bước là vector 3 chiều $[\tilde{g}_t, \sin\theta, \cos\theta]$. Trạng thái ẩn ở bước cuối qua một lớp tuyến tính $64 \to 4$. Mô hình dự báo *phần dư* so với giá trị cuối: $\hat{\mathbf{y}} = \tilde{g}_T\mathbf{1} + \mathbf{W}\mathbf{h}_T + \mathbf{b}$. Kết nối tắt này giống ResNet: mạng chỉ cần học *thay đổi*, và khởi đầu ngay từ persistence thay vì từ con số 0 (Đoạn mã 4.1).

Đoạn mã 4.1. Mô hình hồi quy dự báo glucose và vòng huấn luyện (`lab/ch4_rnn.py`, rút gọn)

```
class SeqModel(nn.Module):
    def __init__(self, cell="lstm", hidden=64, layers=1):
        super().__init__()
        rnn = {"rnn": nn.RNN, "lstm": nn.LSTM, "gru": nn.GRU}[cell]
        self.rnn = rnn(3, hidden, num_layers=layers, batch_first=True,
                       dropout=0.2 if layers > 1 else 0.0)
        self.head = nn.Linear(hidden, 4)                  # 15, 30, 45, 60 phút

    def forward(self, x):                                 # x: [B, 12, 3]
        out, _ = self.rnn(x)                              # out: [B, 12, 64]
        return x[:, -1:, 0] + self.head(out[:, -1])     # dự báo phần dư so với điểm cuối

for epoch in range(40):
    for b in batches(Xtr, Ytr, size=512):
        loss = F.mse_loss(model(b.x), b.y)
        opt.zero_grad(); loss.backward()
        nn.utils.clip_grad_norm_(model.parameters(), 1.0)   # chặn bùng nổ gradient
        opt.step()
    sched.step(val_mse)                                   # giảm tốc độ học khi validation chững
     ↪ lại
    # dừng sớm sau 8 epoch không cải thiện, giữ trọng số tốt nhất
```

Bảng 4.3. Kích thước tensor và số tham số của các mô hình hồi quy ($d = 3$, $m = 64$)

Mô hình	Công thức số tham số lớp hồi quy	Hồi quy	Tổng (+ lớp ra 260)
RNN	$m(d+m) + 2m = 64 \cdot 67 + 128$	4.416	4.676
GRU	$3[m(d+m) + 2m]$	13.248	13.508
LSTM	$4[m(d+m) + 2m]$	17.664	17.924
LSTM 2 tầng	tầng 1 như trên + $4[m(m+m) + 2m]$	50.944	51.204

Huấn luyện dùng Adam (tốc độ học $2 \cdot 10^{-3}$, giảm một nửa khi validation không cải thiện sau 3 epoch), lô 512, cắt gradient ở chuẩn 1, dừng sớm sau 8 epoch không cải thiện, tối đa 40 epoch. Huấn luyện trên GPU mất 23–43 giây mỗi mô hình.

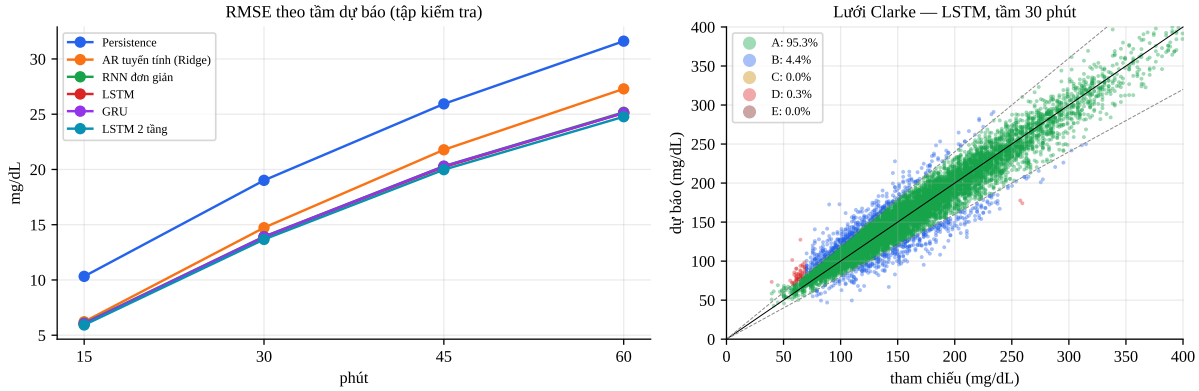
Thước đo. RMSE và MAE (mg/dL); MARD $= \frac{1}{n}\sum |y - \hat{y}|/y$, thước đo chuẩn của

ngành CGM; và *lưới sai số Clarke* [64]. Lưới Clarke chia mặt phẳng (tham chiếu, dự báo) thành 5 vùng theo hậu quả lâm sàng: A (sai ≤ 20% hoặc cả hai < 70 mg/dL), B (sai lệch lành tính), C (dẫn tới điều trị thừa), D (bỏ sót cần điều trị), E (điều trị ngược). Một hệ thống chấp nhận được cần hơn 99% điểm nằm trong vùng A+B.

4.6. Kết quả và nhận xét

Bảng 4.4. Dự báo glucose trên 17 bệnh nhân của tập kiểm tra (18.012 cửa sổ)

		Tầm 30 phút				Tầm 60 phút	
Mô hình	**Tham số**	**RMSE**	**MAE**	**MARD**	**Clarke A**	**RMSE**	**Clarke A**
Persistence	0	19,01	12,97	8,81%	90,1%	31,61	74,5%
AR tuyến tính (Ridge)	52	14,73	10,30	7,25%	94,5%	27,29	79,0%
RNN đơn giản	4.676	13,93	9,81	6,94%	95,2%	25,18	81,4%
LSTM	17.924	13,85	9,72	6,85%	95,3%	25,11	81,5%
GRU	13.508	13,90	9,64	6,75%	95,3%	25,11	82,1%
LSTM 2 tầng	51.204	**13,67**	**9,51**	**6,66%**	**95,6%**	**24,77**	**82,6%**



Hình 4.2. RMSE theo tầm dự báo và lưới sai số Clarke của LSTM ở tầm 30 phút

So với mốc đối chứng. Ở tầm 30 phút, LSTM 2 tầng giảm RMSE 28% so với persistence (19,01 → 13,67 mg/dL) và nâng tỉ lệ vùng Clarke A từ 90,1% lên 95,6%. Vùng A+B đạt 99,75%. Mô hình tuyến tính đã thu được phần lớn cải thiện (14,73): xu hướng tăng hoặc giảm của vài điểm cuối mang nhiều thông tin nhất. Các mô hình hồi quy giảm thêm 0,8–1,1 mg/dL nhờ học được động học phi tuyến, chẳng hạn đỉnh sau ăn "uốn" xuống chứ không tăng tuyến tính mãi.

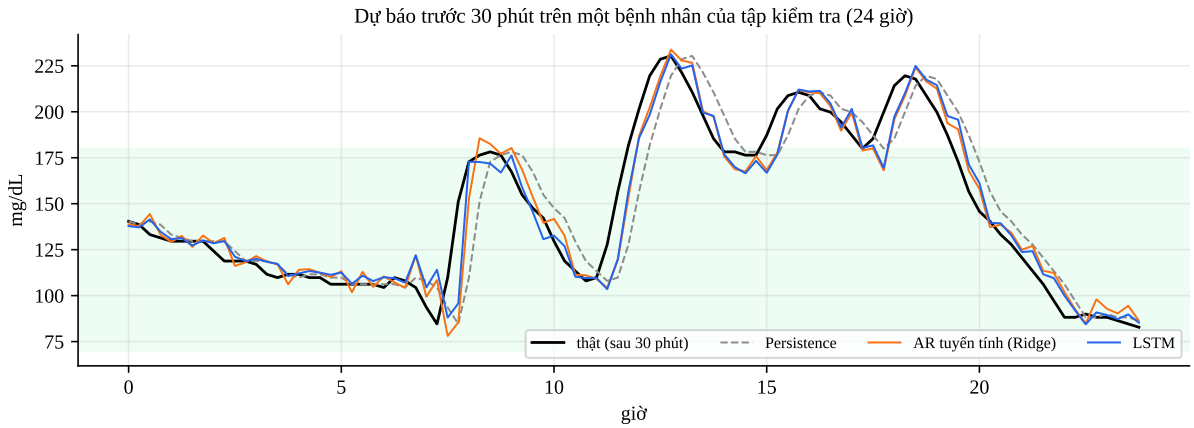
Giữa các mô hình hồi quy. RNN, LSTM, GRU chỉ cách nhau dưới 0,1 mg/dL ở tầm 30 phút; LSTM 2 tầng tốt hơn khoảng 0,2 mg/dL. Với cửa sổ chỉ 12 bước, phụ thuộc xa không phải là nút thắt. RNN đơn giản đủ "nhớ" 3 giờ và không chịu thiệt nhiều từ biến mất gradient. Lợi thế của các cổng thể hiện rõ hơn ở tầm xa: ở 60 phút, GRU và LSTM 2 tầng đạt Clarke A 82,1–82,6% so với 81,4% của RNN. Với dữ liệu nhỏ và yêu cầu độ trễ thấp, GRU là lựa chọn cân bằng: ít tham số hơn LSTM 25%, độ chính xác tương đương.

Sai số tăng nhanh theo tầm dự báo, từ khoảng 6 mg/dL ở 15 phút lên khoảng 25 mg/dL ở 60 phút. Trong một giờ, bữa ăn, insulin và vận động là các yếu tố không quan sát được từ chuỗi CGM mà quyết định quỹ đạo. Để dự báo xa hơn, cần thêm các đầu vào ngoại sinh này, vốn có trong bộ dữ liệu nhưng ghi chép không đều. Các nghiên cứu như GluNet [65] cũng ghi nhận sai số tăng nhanh theo tầm dự báo và dùng thêm thông tin bữa ăn, insulin để cải thiện tầm xa.

Bảng 4.5. Kết quả tầm 30 phút theo loại ĐTĐ trên tập kiểm tra

Nhóm	Cửa sổ	RMSE persistence	RMSE LSTM	MARD LSTM	Clarke A LSTM
Típ 1	2.227	20,80	13,86	6,39%	95,6%
Típ 2	15.785	18,75	13,85	6,92%	95,3%

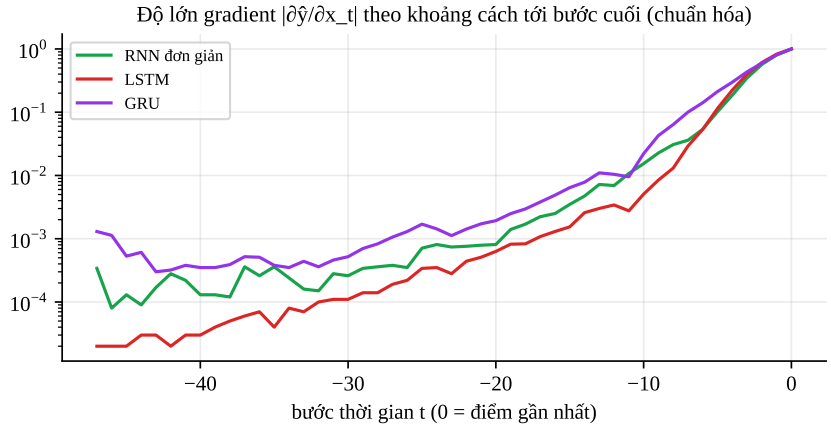
Theo Bảng 4.5, chuỗi típ 1 biến động mạnh hơn nên persistence kém hơn (20,80), nhưng LSTM đưa sai số hai nhóm về cùng mức. Mức giảm tương đối trên nhóm típ 1 là 33%, cao hơn 26% của nhóm típ 2. Mô hình có ích nhất ở chính nhóm cần nó nhất.



Hình 4.3. Dự báo trước 30 phút trên 24 giờ liên tục của một bệnh nhân thuộc tập kiểm tra

Hình 4.3 cho thấy hành vi điển hình: persistence luôn "trễ" đúng 30 phút so với đường thật. AR tuyến tính và LSTM đều bám sát đường thật hơn nhiều, và khác biệt giữa

hai mô hình (0,9 mg/dL RMSE) gần như không thấy bằng mắt. Cả hai đều vượt quá ở những điểm đổi chiều đột ngột (khoảng giờ thứ 7–8): từ chuỗi CGM, mô hình không biết bữa ăn đã kết thúc hay insulin đã có tác dụng.



Hình 4.4. Độ lớn gradient của dự báo theo từng điểm đầu vào trên chuỗi dài 48 bước (chuẩn hóa theo bước cuối)

Mô hình "nhìn" bao xa? Hình 4.4 đo $|\partial \hat{y} / \partial \tilde{g}_t|$ trên chuỗi dài 48 bước (12 giờ). Ở cả ba mô hình đã huấn luyện, độ nhạy giảm nhanh theo khoảng cách. Ở bước cách 11 điểm (khoảng 3 giờ), độ nhạy chỉ còn 1,1% (RNN), 0,3% (LSTM) và 1,0% (GRU) so với điểm cuối. Ở bước xa nhất, các con số giảm 3–5 bậc. Cần đọc kết quả này cẩn thận. Đây là độ nhạy của mô hình *đã học*, nên phản ánh cả hai yếu tố: biến mất gradient theo (4.2), và việc mô hình chủ động học cách "quên" quá khứ xa vì với glucose, vài điểm gần nhất là quan trọng nhất. LSTM có độ nhạy giảm nhanh nhất. Điều này cho thấy cổng quên đã học để loại bỏ thông tin cũ, chứ không phải không thể giữ lại. Muốn kiểm chứng riêng khả năng nhớ xa, cần một bài toán mà đáp án phụ thuộc vào điểm đầu chuỗi, như bài toán sao chép hay phép cộng của Hochreiter.

4.6.1. Phân tích sai số theo dải đường huyết

Sai số trung bình che giấu hành vi ở những vùng quan trọng nhất về lâm sàng. Bảng 4.6 chia các điểm tham chiếu ở tầm 30 phút theo ba dải, dùng mô hình triển khai (LSTM 2 tầng).

Bảng 4.6. Sai số dự báo 30 phút theo dải glucose tham chiếu (tập kiểm tra, LSTM 2 tầng)

Dải glucose thật	Số điểm	RMSE persistence	RMSE LSTM	MARD LSTM
< 70 mg/dL (hạ đường huyết)	206	14,90	12,98	15,0%
70–180 mg/dL (mục tiêu)	14.067	15,73	11,67	6,6%
> 180 mg/dL (tăng đường huyết)	3.739	28,26	19,44	6,4%
Thay đổi nhanh ($\|\Delta\|$ > 60 mg/dL trong 30 phút)	261	74,10	44,37	–

Có ba nhận xét. (i) Ở vùng tăng đường huyết, sai số tuyệt đối lớn nhất (19,4 mg/dL) nhưng sai số tương đối nhỏ (6,4%), và LSTM giảm 31% so với persistence. Đây là vùng đường huyết biến động mạnh sau ăn, nơi mô hình học được xu hướng. (ii) Ở vùng hạ đường huyết, MARD tăng lên 15%, do mẫu số nhỏ, và mức cải thiện so với persistence chỉ 13%. Hạ đường huyết hiếm trong dữ liệu (206/18.012 điểm, khoảng 1,1%), nên mô hình học từ MSE ít "để ý" tới vùng này. (iii) Khi glucose thay đổi nhanh, LSTM giảm sai số 40% so với persistence, nhưng sai số vẫn ở mức 44 mg/dL, cho thấy giới hạn của việc chỉ dùng chuỗi CGM.

Nếu dùng mô hình làm *bộ cảnh báo hạ đường huyết trước 30 phút* (báo khi dự báo < 70 mg/dL), độ nhạy chỉ đạt 0,59 và precision 0,72: cứ 10 cơn hạ đường huyết thì bỏ sót khoảng 4. Đây là lý do hệ thống *không* dùng dự báo để cảnh báo cấp cứu mà chỉ trình bày như thông tin tham khảo. Guardrail hạ đường huyết dựa trên giá trị đo thật (< 54 mg/dL). Để cải thiện, có thể dùng hàm mất mát có trọng số theo vùng nguy hiểm, hoặc huấn luyện thêm một bộ phân loại "sắp hạ đường huyết" riêng.

4.7. Triển khai

Mô hình được chọn theo MSE trên validation là LSTM 2 tầng (`data/models/ch4_glucose_rnn.pt`, 210 KB). Nó được đóng gói trong lớp `RNNTemporalEngine` (`src/models/rnn_temporal.py`), dùng chung cho hai nơi. (1) **Luồng chat**: khi người dùng tải lên tệp CSV đo đường huyết, DocumentAgent trích chuỗi glucose vào hồ sơ. PredictionAgent tính TIR, TAR, TBR, CV và dự báo 15–60 phút nếu có ít nhất 12 điểm. ConsensusCriticAgent thêm điểm (–) khi TBR vượt 4%. (2) **Endpoint** `POST /api/lab/ch4/forecast` nhận chuỗi glucose và giờ của điểm cuối. `GET /api/lab/ch4/sample/{i}` trả về 24 giờ dữ liệu thật của bệnh nhân thuộc tập kiểm tra để thử. Trang `/lab` vẽ chuỗi đầu vào, dự báo, và giá trị thật nếu dùng dữ liệu mẫu.

Đoạn mã 4.2. Gọi endpoint Chương 4

```
curl -s https://agent.nits.io.vn/api/lab/ch4/forecast -H 'Content-Type: application/json' \
  -d '{"glucose":[110,115,124,138,152,166,175,181,184,182,176,168],"last_hour":9}'
```

4.8. Tiểu kết chương 4

Chương 4 trình bày RNN với trạng thái ẩn dùng chung tham số theo thời gian, thuật toán BPTT, nguyên nhân toán học của biến mất và bùng nổ gradient, và hai lời giải có cổng: LSTM (ô nhớ cập nhật cộng tính, ba cổng) và GRU (hai cổng). Trên 127.982 điểm CGM thật của 112 bệnh nhân, chia theo người để tránh rò rỉ: (1) mọi mô hình học được đều vượt xa persistence, và LSTM 2 tầng giảm 28% RMSE ở tầm 30 phút với 95,6% điểm trong vùng Clarke A; (2) với cửa sổ 3 giờ, khác biệt giữa RNN, LSTM và GRU nhỏ, lợi thế của cổng chỉ rõ ở tầm xa; (3) giới hạn chính là thông tin, không phải kiến trúc: bữa ăn và insulin quyết định quỹ đạo sau 1 giờ. Ba chương vừa qua làm việc với dữ liệu *có cấu trúc đều*: bảng, lưới ảnh, chuỗi. Chương 5 chuyển sang tri thức dạng văn bản và đồ thị.

CHƯƠNG 5. ĐỒ THỊ TRI THỨC VÀ SINH VĂN BẢN TĂNG CƯỜNG TRUY XUẤT

Một chatbot y tế không được phép "bịa". Mô hình ngôn ngữ lớn lưu tri thức trong hàng tỉ tham số. Tri thức đó không trích dẫn được, không cập nhật được khi hướng dẫn thay đổi, và có thể bị ảo giác [66]: một câu sai về liều thuốc hay ngưỡng chẩn đoán, diễn đạt trôi chảy, nguy hiểm hơn một câu "tôi không biết". Chương này trình bày hai cách đưa tri thức *tường minh* trở lại hệ thống. *Đồ thị tri thức* (Knowledge Graph – KG) biểu diễn thực thể và quan hệ có cấu trúc. *Sinh văn bản tăng cường truy xuất* (Retrieval-Augmented Generation – RAG) tìm đoạn tài liệu liên quan rồi buộc LLM trả lời dựa trên chúng. Đây là nền tảng của KnowledgeAgent. Thực nghiệm đánh giá tám chiến lược truy xuất trên hai bộ dữ liệu và ba chế độ sinh câu trả lời.

5.1. Đồ thị tri thức

5.1.1. Định nghĩa và mô hình dữ liệu

Đồ thị tri thức là một đồ thị có hướng, có nhãn, trong đó nút là *thực thể* (bệnh, thuốc, xét nghiệm, yếu tố nguy cơ) và cạnh là *quan hệ* giữa chúng [67]. Đơn vị cơ bản là *bộ ba* (chủ thể, vị từ, đối tượng), ví dụ (*Đái tháo đường típ 2*, `DIAGNOSED_BY`, *HbA1c*). Thuật ngữ được Google phổ biến năm 2012 với khẩu hiệu "things, not strings" [68], nhưng ý tưởng có từ mạng ngữ nghĩa thập niên 1960 và các hệ chuyên gia ở Chương 1. Có hai mô hình dữ liệu chính:

- **RDF** (Resource Description Framework): mọi thứ là bộ ba, định danh bằng URI, truy vấn bằng SPARQL; mạnh về chuẩn hóa và liên kết dữ liệu mở (Wikidata, DBpedia, SNOMED CT, UMLS).
- **Đồ thị thuộc tính** (property graph): nút và cạnh đều mang cặp khóa–giá trị, truy vấn bằng Cypher (Neo4j, Apache AGE [69]); trực quan và thuận tiện cho ứng dụng.

Ontology định nghĩa lược đồ: các lớp thực thể, các loại quan hệ được phép, và ràng buộc giữa chúng. KG của hệ thống dùng một ontology nhỏ gồm 6 lớp nút (`Disease`, `RiskFactor`, `Test`, `Drug`, `Complication`, `Intervention`) và 7 loại quan hệ (`HAS_RISK_FACTOR`, `DIAGNOSED_BY`, `CAUSES`, `SCREENED_BY`, `TREATED_BY`, `CONTRAINDICATED_IN`, `PREVENTED_BY`). Ví dụ một truy vấn Cypher tìm các thuốc chống chỉ định với một tình

trạng của bệnh nhân:

Đoạn mã 5.1. Truy vấn Cypher trên Apache AGE: láng giềng hai bước từ các yếu tố nguy cơ của bệnh nhân

```
SELECT * FROM cypher('diabetes_kg', $$
  MATCH p = (s)-[*1..2]-(t)
  WHERE s.name IN ['Thừa cân/béo phì', 'Tăng huyết áp', 'Suy thận nặng (eGFR < 30)']
  UNWIND relationships(p) AS r
  RETURN startNode(r).name, type(r), endNode(r).name, r.chunk_ids
  LIMIT 60
$$) AS (head agtype, relation agtype, tail agtype, chunk_ids agtype);
```

5.1.2. Xây dựng đồ thị tri thức

Quy trình xây dựng KG từ văn bản gồm bốn bước:

1. *Nhận dạng thực thể* (NER): tìm các cụm từ chỉ thực thể ("metformin", "eGFR").
2. *Liên kết thực thể*: ánh xạ các cách gọi khác nhau về cùng một nút ("tiểu đường", "ĐTĐ típ 2", "đái tháo đường type 2").
3. *Trích xuất quan hệ*: xác định vị từ giữa hai thực thể từ ngữ cảnh câu.
4. *Hợp nhất và kiểm định*: gộp bộ ba trùng lặp, gắn nguồn gốc và độ tin cậy.

Trước đây các bước này cần mô hình chuyên biệt cho từng bước. Ngày nay LLM có thể trích bộ ba trực tiếp theo một lược đồ JSON. Hệ thống dùng cả hai nguồn. 40 bộ ba *tuyển chọn thủ công* từ ADA 2024 và QĐ 5481/QĐ-BYT tạo thành lõi đáng tin cậy. Tùy chọn `--llm-triples` cho phép Gemini trích thêm bộ ba từ tài liệu mới. Mỗi cạnh lưu `chunk_ids`, tức danh sách đoạn văn làm bằng chứng. Đây là cầu nối giữa đồ thị và kho vector, nền tảng của GraphRAG.

5.1.3. Suy luận và nhúng đồ thị tri thức

KG cho phép *suy luận*. Suy luận tường minh dùng luật: nếu bệnh nhân có *eGFR* $<$ *30* và *Metformin* `CONTRAINDICATED_IN` *Suy thận nặng* thì cảnh báo. Suy luận ngầm dùng *nhúng KG*, ví dụ TransE [70] biểu diễn mỗi thực thể và quan hệ bằng vector sao cho $\mathbf{e}_h + \mathbf{r} \approx \mathbf{e}_t$, nhờ đó dự đoán được cạnh còn thiếu bằng cách tìm t gần $\mathbf{e}_h + \mathbf{r}$ nhất. Mạng nơ-ron đồ thị ở Chương 6 là bước tổng quát hơn của ý tưởng này.

5.2. Sinh văn bản tăng cường truy xuất

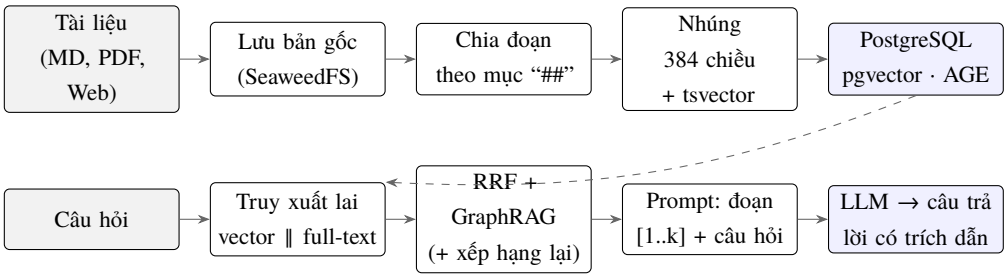
5.2.1. Nguyên lý

RAG [71] tách hai loại tri thức: tri thức *tham số* (ngôn ngữ, lập luận, nằm trong LLM) và tri thức *phi tham số* (sự kiện, nằm trong kho tài liệu bên ngoài). Với câu hỏi q, bộ truy xuất tìm k đoạn liên quan nhất $\{d_1, \dots, d_k\}$, rồi bộ sinh tạo câu trả lời có điều kiện trên cả hai:

$$p(y \mid q) \approx p_{\text{LLM}}(y \mid q, d_1, \dots, d_k), \qquad \{d_i\} = \text{top-}k_{d \in \mathscr{D}}\, s(q, d). \tag{5.1}$$

Lợi ích gồm: cập nhật tri thức chỉ cần thêm tài liệu, không cần huấn luyện lại; trích dẫn được nguồn; giảm ảo giác khi prompt buộc mô hình chỉ dùng ngữ cảnh. So với tinh chỉnh (fine-tuning), RAG phù hợp hơn với tri thức thay đổi và cần kiểm chứng như hướng dẫn lâm sàng. Tinh chỉnh phù hợp hơn khi cần đổi văn phong hoặc kỹ năng của mô hình [72].

5.2.2. Quy trình



Hình 5.1. Hai pha của RAG: nạp tri thức (trên) và trả lời câu hỏi (dưới)

Chia đoạn. Đoạn quá dài làm loãng tín hiệu liên quan và tốn ngữ cảnh. Đoạn quá ngắn mất ngữ cảnh. Các chiến lược thường gặp: kích thước cố định có chồng lấp, theo cấu trúc tài liệu (đề mục, đoạn văn), hoặc theo ngữ nghĩa. KB tiếng Việt được chia theo mục "##" vì mỗi mục của hướng dẫn lâm sàng là một đơn vị ý trọn vẹn. Văn bản không có cấu trúc được chia theo cửa sổ khoảng 1.200 ký tự, chồng lấp 200.

Truy xuất thưa (BM25). BM25 [73] chấm điểm theo từ khóa trùng khớp:

$$\begin{aligned} \text{BM25}(q, d) &= \sum_{t \in q} \text{IDF}(t) \cdot \frac{f(t,d)\,(k_1+1)}{f(t,d) + k_1\big(1 - b + b\,\frac{|d|}{\text{avgdl}}\big)}, \\ \text{IDF}(t) &= \ln\Big(1 + \frac{N - n_t + 0{,}5}{n_t + 0{,}5}\Big), \end{aligned} \tag{5.2}$$

với $f(t,d)$ là tần suất của từ t trong đoạn d; $k_1 = 1{,}5$ điều khiển độ bão hòa tần suất;

$b = 0{,}75$ chuẩn hóa theo độ dài. BM25 bắt chính xác thuật ngữ hiếm ("SGLT2", "monofilament", "eGFR") nhưng không hiểu từ đồng nghĩa ("tiểu đường" và "đái tháo đường").

Truy xuất dày. Bộ mã hóa câu (Sentence-BERT [74]) ánh xạ câu hỏi và đoạn văn vào cùng không gian vector, sau đó chấm điểm bằng độ tương tự cosin $s(q,d) = \frac{\mathbf{e}_q^\top \mathbf{e}_d}{\|\mathbf{e}_q\|\|\mathbf{e}_d\|}$. Mô hình được huấn luyện tương phản: kéo cặp (câu hỏi, đoạn đúng) lại gần nhau và đẩy cặp sai ra xa [75]. Mô hình đa ngữ (huấn luyện bằng chưng cất đa ngữ [76]) đặt câu tiếng Việt và tiếng Anh cùng nghĩa ở gần nhau, cho phép truy xuất chéo ngôn ngữ. Với hàng triệu vector, tìm kiếm chính xác $O(N)$ được thay bằng chỉ mục xấp xỉ HNSW [77], một đồ thị nhiều tầng cho phép tìm láng giềng gần nhất trong khoảng $O(\log N)$. pgvector [78] cung cấp HNSW ngay trong PostgreSQL.

Truy xuất lai và RRF. Hai bộ truy xuất bổ sung cho nhau, nhưng điểm của chúng không cùng thang đo. Reciprocal Rank Fusion [79] chỉ dùng *thứ hạng*:

$$\mathrm{RRF}(d) = \sum_{r \in \mathcal{R}} \frac{1}{k + \mathrm{rank}_r(d)}, \qquad k = 60. \tag{5.3}$$

Hằng số k làm phẳng chênh lệch giữa các hạng đầu, nên một đoạn đứng đầu ở chỉ một danh sách không áp đảo được đoạn đứng khá cao ở cả hai danh sách.

Xếp hạng lại. Bộ mã hóa chéo (cross-encoder) [80] đưa *cặp* (câu hỏi, đoạn) qua cùng một Transformer để chấm điểm liên quan. Mô hình này chính xác hơn nhiều so với so sánh hai vector độc lập, vì attention được tính chéo giữa hai văn bản, nhưng chậm hơn tương ứng. Vì vậy cross-encoder chỉ dùng để xếp lại 10–20 ứng viên đầu.

5.2.3. GraphRAG

RAG thuần vector coi mỗi đoạn là độc lập, nên khó trả lời câu hỏi cần *nối* nhiều mẩu tri thức. Ví dụ: "bệnh thận mạn có albumin niệu nên dùng thuốc gì?" cần nối *Bệnh thận ĐTĐ* → `TREATED_BY` → *Ức chế SGLT2*. GraphRAG [81] khai thác cấu trúc đồ thị để mở rộng ngữ cảnh. Phiên bản trong hệ thống gồm ba bước: (1) liên kết thực thể trong câu hỏi hoặc hồ sơ bệnh nhân với nút KG; (2) duyệt láng giềng 1–2 bước bằng Cypher; (3) lấy các đoạn văn gắn với cạnh tìm được (`chunk_ids`), hợp nhất với kết quả truy xuất lai bằng RRF, và đưa các bộ ba vào prompt như "sự kiện có cấu trúc".

5.2.4. So sánh RAG và tinh chỉnh

Bảng 5.1. So sánh RAG và tinh chỉnh (fine-tuning) trong bối cảnh chatbot y tế

Tiêu chí	RAG	Tinh chỉnh
Cập nhật tri thức	Thêm hoặc sửa tài liệu, có hiệu lực ngay	Phải huấn luyện lại
Trích dẫn nguồn	Có, đến từng đoạn	Không
Kiểm soát ảo giác	Tốt khi prompt buộc chỉ dùng ngữ cảnh	Hạn chế, tri thức nằm trong tham số
Chi phí ban đầu	Thấp: nhúng và chỉ mục	Cao: dữ liệu gán nhãn, GPU
Chi phí mỗi truy vấn	Cao hơn: truy xuất và ngữ cảnh dài	Thấp hơn
Phù hợp với	Sự kiện, ngưỡng, hướng dẫn thay đổi theo thời gian	Văn phong, định dạng, kỹ năng chuyên biệt

Hai cách tiếp cận không loại trừ nhau. Một hệ thống có thể tinh chỉnh mô hình để quen thuật ngữ và văn phong y khoa tiếng Việt, đồng thời dùng RAG để cung cấp sự kiện có nguồn. Với hướng dẫn lâm sàng được cập nhật hằng năm như ADA Standards of Care, RAG là lựa chọn bắt buộc cho phần sự kiện.

5.2.5. Thách thức vận hành

Đưa RAG vào sản phẩm gặp một số thách thức ngoài độ chính xác truy xuất. *Chất lượng nguồn*: RAG chỉ tốt bằng tài liệu được nạp; tài liệu lỗi thời hoặc mâu thuẫn sẽ được trích dẫn một cách tự tin. *Bảo trì chỉ mục*: khi đổi mô hình nhúng phải nhúng lại toàn bộ kho; khi sửa tài liệu phải cập nhật cả đoạn văn lẫn các cạnh KG trỏ tới đoạn đó (hệ thống lưu `chunk_ids` trên cạnh và bỏ qua tài liệu không đổi dựa trên mã băm nội dung). *Độ trễ và chi phí*: mỗi bước thêm vào (xếp hạng lại, mở rộng đồ thị, ngữ cảnh dài hơn) đều tăng độ trễ. Thực nghiệm cho thấy cross-encoder chính xác nhất nhưng chậm nhất. *Đánh giá liên tục*: cần một bộ câu hỏi có đáp án để chạy lại sau mỗi thay đổi, như 60 câu hỏi của chương này. *An toàn*: tài liệu nạp từ web có thể chứa chỉ dẫn độc hại nhắm vào LLM (prompt injection). Vì vậy ngữ cảnh truy xuất được đặt trong vai trò "dữ liệu", và câu trả lời luôn được ràng buộc bởi các quy tắc an toàn của hội đồng (Chương 7).

5.2.6. *Đánh giá hệ thống RAG*

Đánh giá RAG gồm hai tầng. *Tầng truy xuất* dùng các thước đo xếp hạng với tập đoạn đúng G của mỗi câu hỏi: Hit@k (có ít nhất một đoạn đúng trong top-k), Recall@k = $|G \cap \text{top-}k|/|G|$, MRR = $\frac{1}{|Q|}\sum_q 1/\text{rank}_q$ (hạng của đoạn đúng đầu tiên), và nDCG@10 (phần thưởng giảm theo $1/\log_2(\text{hạng} + 1)$). *Tầng sinh* đánh giá câu trả lời có chứa dữ kiện đúng không, có trung thành với ngữ cảnh không (faithfulness), và có trích dẫn đúng nguồn không [82].

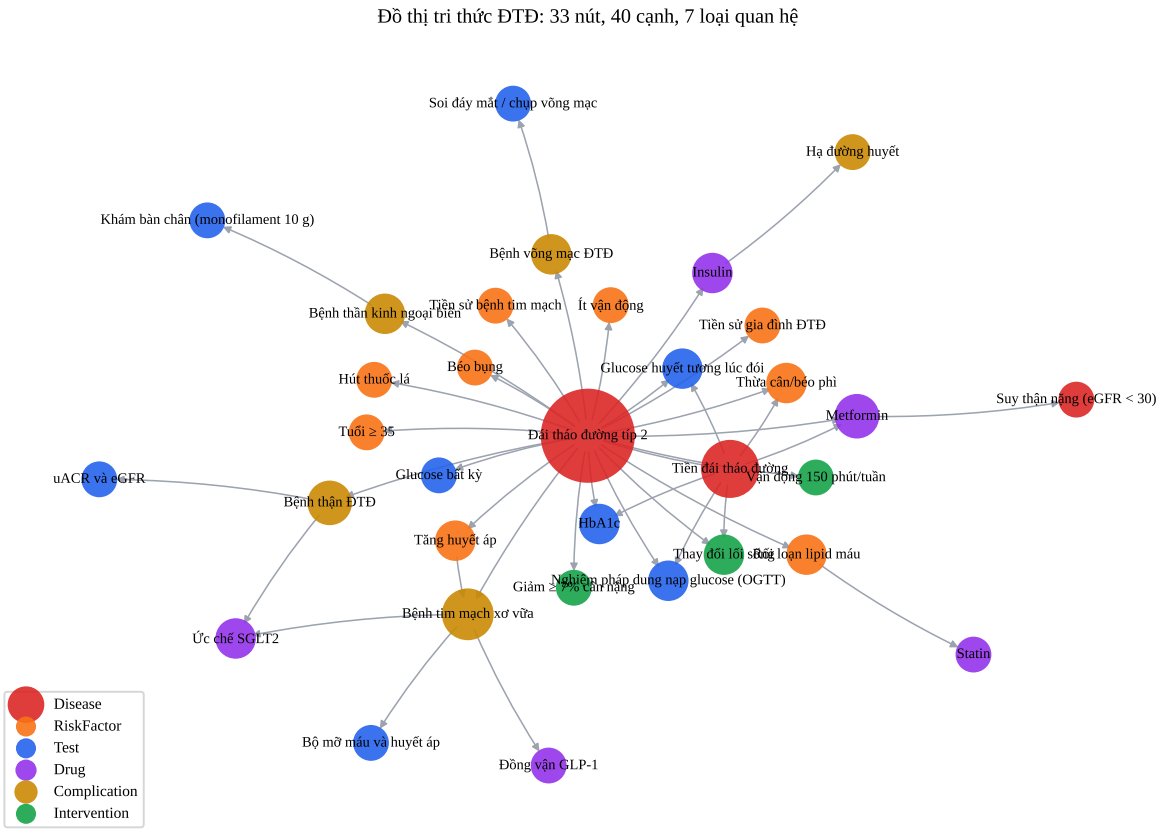
5.3. Dữ liệu

5.3.1. *Knowledge Base tiếng Việt và đồ thị tri thức*

KB của hệ thống gồm 10 tài liệu Markdown (khoảng 18 KB văn bản) tóm tắt từ ADA Standards of Care 2024 [2] và Quyết định 5481/QĐ-BYT [3]. Nội dung bao gồm tiêu chuẩn chẩn đoán, tiền ĐTĐ, sàng lọc người châu Á, mục tiêu điều trị, thuốc, biến chứng, CGM và hạ đường huyết, lối sống, triệu chứng cấp cứu. Script `scripts/ingest_kb.py` lưu bản gốc lên SeaweedFS, chia đoạn theo mục "##" thành **36 đoạn** (trung bình 279 ký tự), nhúng bằng MiniLM đa ngữ 384 chiều vào `kb_chunks` (pgvector, chỉ mục HNSW) kèm cột `tsvector` cho full-text, và nạp **40 bộ ba** tuyển chọn vào đồ thị `diabetes_kg` trên Apache AGE.

Bảng 5.2. Ví dụ đoạn văn trong KB và bộ ba trong đồ thị tri thức

Đoạn #1	*Tiêu chuẩn chẩn đoán đái tháo đường — Bốn tiêu chí chẩn đoán*: "Chẩn đoán đái tháo đường khi có một trong các tiêu chí sau: Glucose huyết tương lúc đói (FPG) ≥ 126 mg/dL (7,0 mmol/L)…"
Bộ ba	(*Đái tháo đường típ 2*, `HAS_RISK_FACTOR`, *Thừa cân/béo phì*) — bằng chứng: "BMI ≥ 23 kg/m² (người châu Á) kèm yếu tố nguy cơ cần được sàng lọc" — nguồn: `sang-loc-nguoi-viet`, mục "Các yếu tố nguy cơ"
Bộ ba	(*Metformin*, `CONTRAINDICATED_IN`, *Suy thận nặng (eGFR < 30)*) — nguồn: `dieu-tri-thuoc`, mục "Metformin"



Hình 5.2. Đồ thị tri thức ĐTĐ: 33 nút thuộc 6 loại, 40 cạnh thuộc 7 loại quan hệ (kích thước nút tỉ lệ với bậc)

Đồ thị có 33 nút: 9 yếu tố nguy cơ, 8 xét nghiệm, 5 biến chứng, 5 thuốc, 3 bệnh, 3 can thiệp. Quan hệ nhiều nhất là `HAS_RISK_FACTOR` (11), `TREATED_BY` (9) và `DIAGNOSED_BY` (7). Đồ thị có dạng "sao": nút *Đái tháo đường típ 2* có bậc 23, tiếp theo là *Tiền ĐTĐ* (7) và *Bệnh tim mạch xơ vữa* (5). Cấu trúc này hợp với GraphRAG vì hầu hết mọi thực thể cách nhau tối đa 2 bước.

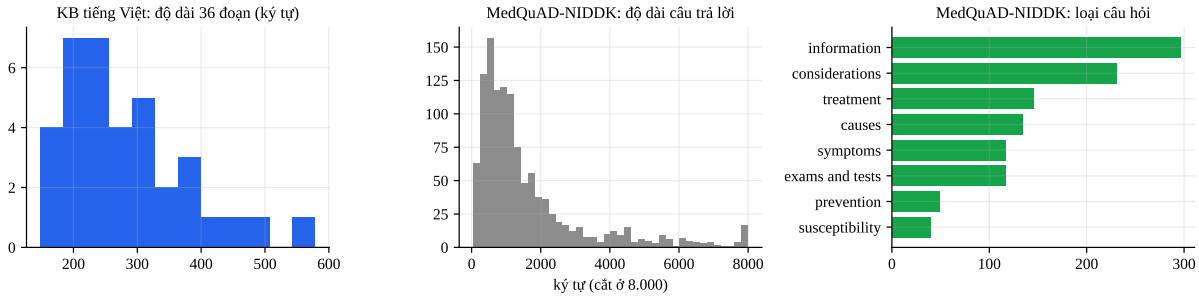
Bộ câu hỏi đánh giá. Do chưa có bộ hỏi–đáp tiếng Việt công khai cho các tài liệu này, tôi tự xây dựng **60 câu hỏi** (tệp `lab/kb_eval_vi.json`). Mỗi câu gắn với 1–2 *đoạn vàng* và các *dữ kiện vàng* dùng để chấm câu trả lời (ví dụ "126" và "7,0"). Câu hỏi được chia bốn loại để đo khía cạnh khác nhau của truy xuất: *từ khóa* (20 câu, dùng đúng thuật ngữ trong tài liệu), *diễn đạt dân dã* (19 câu, ví dụ "tụt đường huyết", "thuốc mỡ máu", "tiểu đường"), *số liệu* (17 câu, hỏi ngưỡng cụ thể), và *đa bước* (4 câu, cần nối hai mục, ví dụ "eGFR 25 thì còn dùng metformin không, thay bằng thuốc gì?").

5.3.2. MedQuAD — phần NIDDK

MedQuAD [8] là bộ 47.457 cặp hỏi–đáp y khoa thu thập từ 12 website của Viện Y tế Quốc gia Hoa Kỳ (NIH). Báo cáo dùng phần **NIDDK** (Viện Đái tháo đường, Tiêu hóa

và Thận) vì đúng chủ đề và có đầy đủ câu trả lời. Một số phần khác của MedQuAD đã bị gỡ câu trả lời vì lý do bản quyền.

- **Nguồn**: `https://github.com/abachaa/MedQuAD`, thư mục `5_NIDDK_QA` (157 tệp XML), giấy phép CC BY 4.0.
- **Quy mô**: 1.192 cặp hỏi–đáp về 148 chủ đề; câu trả lời dài trung bình 1.638 ký tự (trung vị 1.048); 166 câu về đái tháo đường. Chia câu trả lời thành đoạn 800 ký tự (chồng lấp 15%) được **3.381 đoạn**.
- **Loại câu hỏi**: information (296), considerations (231), treatment (145), causes (134), symptoms (117), exams and tests (117)… Câu hỏi được sinh từ mẫu, ví dụ "What is (are) Diabetic Kidney Disease?", "How to prevent …?".



Hình 5.3. Độ dài đoạn của KB tiếng Việt, độ dài câu trả lời và phân bố loại câu hỏi của MedQuAD-NIDDK

Nhận xét. (i) Hai bộ dữ liệu bổ sung cho nhau. KB tiếng Việt nhỏ (36 đoạn) nhưng chính là dữ liệu chạy trong hệ thống. MedQuAD lớn gấp gần 100 lần, đủ để các khác biệt giữa bộ truy xuất có ý nghĩa thống kê. (ii) Với MedQuAD, mỗi chủ đề có nhiều câu hỏi gần giống nhau ("What is X?", "What causes X?", "How to treat X?"), và các câu trả lời của cùng chủ đề dùng chung từ vựng. Bài toán vì thế khó ở chỗ phải chọn đúng *khía cạnh* của chủ đề chứ không chỉ đúng chủ đề. (iii) Để đo truy xuất chéo ngôn ngữ, 166 câu hỏi về ĐTĐ được Gemini dịch sang tiếng Việt tự nhiên (ví dụ "What is (are) 4 Steps to Manage Your Diabetes for Life?" → "4 bước để kiểm soát bệnh tiểu đường suốt đời là gì?").

5.4. Thực nghiệm

5.4.1. Thiết lập

Tám bộ truy xuất được so sánh trên cùng tập đoạn:

1. *BM25* theo (5.2), tự cài đặt, tách từ sau khi chữ thường hóa và bỏ dấu tiếng Việt;

2. –4. *Dày* với ba mô hình nhúng đa ngữ: *MiniLM-L12* (384 chiều, 118 triệu tham số, mô hình đang chạy trong hệ thống), *MPNet-base* (768 chiều, 278 triệu) và *multilingual-E5-large* [83] (1.024 chiều, 560 triệu; câu hỏi có tiền tố `query:`, đoạn văn có tiền tố `passage:`);
5. *Lai*: RRF (5.3) của BM25 và MiniLM;
6. *Lai + GraphRAG*: liên kết thực thể trong câu hỏi với nút KG (theo tên nút và bảng từ đồng nghĩa, sau khi bỏ dấu), lấy các đoạn bằng chứng của cạnh 1 bước, rồi hợp nhất với kết quả lai bằng RRF;
7. *Lai + xếp hạng lại*: 15 ứng viên đầu của bộ lai được chấm lại bằng cross-encoder đa ngữ `bge-reranker-v2-m3` (bản lượng tử int8) [84];
8. *Lai trên PostgreSQL*: đúng câu lệnh SQL của production (pgvector cosin ‖ `tsvector` + `ts_rank_cd`, gộp RRF).

Sinh câu trả lời. Với 60 câu hỏi tiếng Việt, Gemini (`gemini-flash-latest`) trả lời theo ba chế độ: *chỉ LLM* (không có tài liệu), *RAG* (3 đoạn đầu của bộ lai) và *GraphRAG* (3 đoạn đầu của bộ GraphRAG cùng tối đa 12 bộ ba liên quan). Prompt của hai chế độ RAG yêu cầu chỉ dùng tài liệu, trích dẫn dạng [1]–[3], và nói "không đủ dữ liệu" khi tài liệu không có thông tin. Bốn thước đo được dùng: *độ phủ dữ kiện* (tỉ lệ dữ kiện vàng xuất hiện trong câu trả lời, chấp nhận các cách viết tương đương), *trích dẫn đúng* (có trích ít nhất một đoạn vàng), *số liệu không có căn cứ* (tỉ lệ con số trong câu trả lời không xuất hiện trong ngữ cảnh, một chỉ báo của ảo giác), và *tỉ lệ từ chối trả lời*.

Đoạn mã 5.2. Hợp nhất thứ hạng RRF và GraphRAG (`lab/ch5_rag.py`, rút gọn)

```
def rrf(rankings, k=60):
    """Reciprocal Rank Fusion: score(d) = sum_r 1 / (k + rank_r(d))."""
    s = defaultdict(float)
    for ranking in rankings:
        for r, d in enumerate(ranking):
            s[d] += 1 / (k + r + 1)
    return sorted(s, key=lambda d: -s[d])

def graph_candidates(q, triples, chunk_of):
    """Nút KG được nhắc trong câu hỏi → các cạnh 1 bước → đoạn văn làm bằng chứng cho cạnh."""
    ents = link_entities(q, nodes)                  # khớp tên nút + từ đồng nghĩa, đã bỏ dấu
    cands = [chunk_of[(t["source_doc"], t["section"])] for t in triples
             if t["head"] in ents or t["tail"] in ents]
    return ents, list(dict.fromkeys(cands))

hybrid   = rrf([bm25_ranking, dense_ranking])
graphrag = rrf([hybrid[:10], graph_candidates(q, triples, chunk_of)[1]])
```

5.4.2. Kết quả truy xuất trên KB tiếng Việt

Bảng 5.3. Truy xuất trên KB tiếng Việt (60 câu hỏi, 36 đoạn)

Bộ truy xuất	Hit@1	Hit@3	Hit@5	Recall@5	MRR@10	nDCG@10
BM25	0,800	0,933	0,933	0,917	0,869	0,878
Dày — MiniLM (hệ thống)	0,567	0,750	0,833	0,783	0,675	0,698
Dày — MPNet	0,750	0,950	0,950	0,917	0,846	0,859
Dày — E5-large	0,933	0,983	0,983	0,967	0,956	0,952
Lai (BM25 + MiniLM)	0,800	0,900	0,950	0,933	0,858	0,867
Lai + GraphRAG	0,750	0,950	**1,000**	0,958	0,848	0,862
Lai + xếp hạng lại	**0,967**	**1,000**	**1,000**	**0,983**	**0,983**	**0,974**
Lai trên PostgreSQL (production)	0,667	0,850	0,917	0,883	0,774	0,802

Bảng 5.4. Hit@3 theo loại câu hỏi trên KB tiếng Việt

Bộ truy xuất	Từ khóa (20)	Dân dã (19)	Số liệu (17)	Đa bước (4)
BM25	**1,00**	0,89	0,94	0,75
Dày — MiniLM	0,65	0,89	0,71	0,75
Dày — E5-large	0,95	**1,00**	**1,00**	**1,00**
Lai (BM25 + MiniLM)	0,90	0,95	0,88	0,75
Lai + GraphRAG	0,90	**1,00**	0,94	**1,00**
Lai + xếp hạng lại	**1,00**	**1,00**	**1,00**	**1,00**

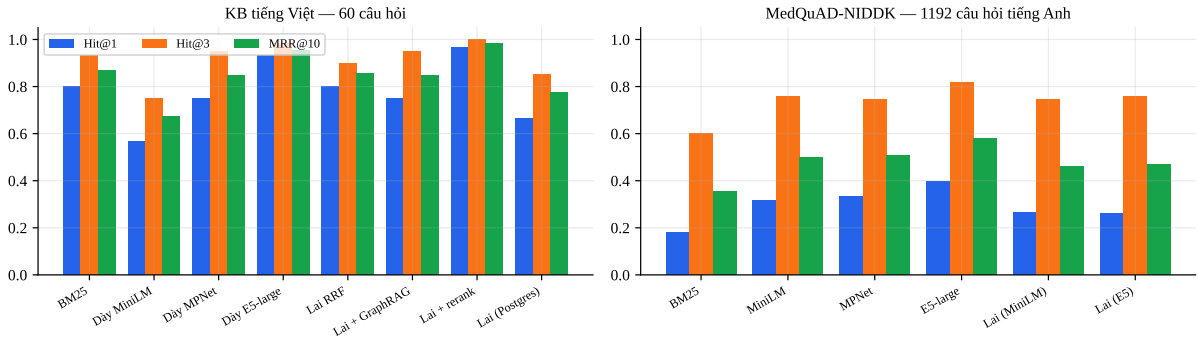
Nhận xét. *BM25 rất mạnh trên KB tiếng Việt* (Hit@1 0,80), vượt xa MiniLM (0,57). Câu hỏi y khoa chứa nhiều thuật ngữ hiếm và con số ("HbA1c", "SGLT2", "monofilament", "eGFR"). BM25 khớp chính xác các từ này, còn MiniLM, một mô hình nhỏ huấn luyện cho tương đồng câu tổng quát, làm mờ chúng: MiniLM chỉ đạt Hit@3 0,65 với câu hỏi từ khóa và 0,71 với câu hỏi số liệu. Ngược lại, với câu hỏi diễn đạt dân dã, MiniLM ngang BM25 (0,89). Đây đúng là tính bổ sung mà truy xuất lai khai thác: bộ lai đạt 0,95 Hit@3 trên nhóm này.

Mô hình nhúng lớn thay đổi cục diện: E5-large đạt Hit@1 0,933 chỉ với truy xuất dày, cho thấy chất lượng mô hình nhúng là yếu tố đơn lẻ quan trọng nhất. *Xếp hạng lại* bằng cross-encoder đưa bộ lai lên mức gần tuyệt đối: Hit@1 0,967, Hit@3 1,000. Đổi lại,

cross-encoder 568 triệu tham số trên CPU mất 3–6 giây cho mỗi câu hỏi (15 cặp), quá chậm cho chatbot. Nó phù hợp hơn với GPU hoặc chạy theo lô.

GraphRAG không cải thiện Hit@1 (0,75 so với 0,80), nhưng là cấu hình duy nhất ngoài reranker đạt Hit@5 = 1,000, và đạt Hit@3 = 1,00 trên cả câu đa bước lẫn câu dân dã. Ví dụ, với câu "Bệnh thận mạn có albumin niệu nên ưu tiên nhóm thuốc nào?", bộ lai trả về [13, 15, 23] và bỏ sót đoạn 25 ("Thuốc bảo vệ tim mạch và thận"). GraphRAG nhận ra thực thể *Bệnh thận ĐTĐ*, đi theo cạnh `TREATED_BY` → *Ức chế SGLT2*, lấy được đoạn bằng chứng và trả về [13, 25, 15], đủ cả hai đoạn vàng. Đồ thị giúp *độ phủ* (recall) hơn là thứ hạng đầu. Ngoài ra, liên kết thực thể chỉ thành công với 67% câu hỏi, nên 1/3 số câu không được đồ thị hỗ trợ.

Bản PostgreSQL thấp hơn bản lai cài bằng Python (Hit@1 0,667 so với 0,800) dù cùng ý tưởng. Nguyên nhân nằm ở nhánh full-text: `ts_rank_cd` xếp hạng theo mật độ phủ của từ, không có IDF như BM25, nên từ phổ biến ("đái tháo đường", "người") có trọng số ngang thuật ngữ hiếm; cột `tsvector` cũng chỉ chứa đề mục và nội dung, không có tiêu đề tài liệu. Kết quả này chỉ ra một cải tiến rõ ràng cho production: thay `ts_rank_cd` bằng BM25 (ví dụ qua phần mở rộng `pg_search`) hoặc nâng mô hình nhúng lên E5.



Hình 5.4. So sánh các bộ truy xuất trên KB tiếng Việt (trái) và MedQuAD-NIDDK (phải)

5.4.3. Kết quả truy xuất trên MedQuAD

Bảng 5.5. Truy xuất câu trả lời trên MedQuAD-NIDDK (1.192 câu hỏi, 3.381 đoạn 800 ký tự)

	Toàn bộ (1.192)					ĐTĐ (166)	
Bộ truy xuất	**Hit@1**	**Hit@3**	**Hit@5**	**MRR@10**	**nDCG@10**	**Hit@1**	**MRR@10**
BM25	0,182	0,420	0,600	0,355	0,468	0,139	0,302
Dày — MiniLM	0,315	0,613	0,758	0,499	0,597	0,259	0,424
Dày — MPNet	0,333	0,610	0,747	0,506	0,602	0,229	0,398
Dày — E5-large	**0,399**	**0,709**	**0,817**	**0,580**	**0,670**	**0,271**	**0,439**
Lai (BM25 + MiniLM)	0,263	0,560	0,746	0,460	0,571	0,199	0,390
Lai (BM25 + E5)	0,262	0,602	0,760	0,471	0,581	0,217	0,405

Trên MedQuAD, thứ tự đảo ngược: *BM25 yếu nhất* (Hit@1 0,18) và *truy xuất lai kém hơn truy xuất dày đơn lẻ*. Lý do nằm ở cấu trúc dữ liệu: các câu hỏi được sinh từ mẫu và chứa nguyên tên chủ đề ("What causes Diabetic Kidney Disease?"), nên BM25 tìm đúng *chủ đề* nhưng không phân biệt được *khía cạnh*. Các câu trả lời "causes", "treatment", "symptoms" của cùng chủ đề đều chứa tên chủ đề với tần suất tương tự. Mô hình nhúng hiểu được ý định "causes" so với "treatment". Khi một thành phần của RRF yếu hơn hẳn thành phần kia, việc gộp kéo kết quả xuống. RRF không phải lúc nào cũng có lợi; nó hiệu quả khi hai bộ truy xuất có chất lượng tương đương và lỗi bổ sung cho nhau, như trên KB tiếng Việt. E5-large vẫn tốt nhất ở mọi thước đo, khẳng định kết luận ở phần trước. Nhóm câu hỏi ĐTĐ khó hơn trung bình (Hit@1 0,27 so với 0,40), vì nhiều tài liệu ĐTĐ của NIDDK có nội dung chồng lấn (bốn tài liệu "Prevent diabetes problems: Keep your … healthy").

Bảng 5.6. Ảnh hưởng của kích thước đoạn (bộ lai BM25 + MiniLM trên MedQuAD)

Kích thước đoạn	**Số đoạn**	**Hit@1**	**Hit@5**	**MRR@10**
400 ký tự	6.291	0,241	0,732	0,446
800 ký tự	3.381	0,263	0,746	0,460
1.600 ký tự	1.991	**0,269**	0,742	**0,466**
Cả câu trả lời	1.192	0,261	**0,753**	0,465

Kích thước đoạn ảnh hưởng ít (MRR 0,446–0,466). Đoạn 400 ký tự kém nhất: nó

cắt câu trả lời thành mảnh mất ngữ cảnh, nhiều mảnh không chứa tên chủ đề. Từ 800 ký tự trở lên, kết quả gần như bão hòa. Với chatbot, đoạn ngắn có lợi thế là tốn ít ngữ cảnh LLM hơn. Chia theo mục "##" như KB tiếng Việt (trung bình 279 ký tự, mỗi mục một ý trọn vẹn) là cách chia theo cấu trúc, tránh được nhược điểm cắt ngang ý của đoạn cố định.

Bảng 5.7. Truy xuất chéo ngôn ngữ: 166 câu hỏi ĐTĐ dịch sang tiếng Việt, tìm trên kho tiếng Anh

Bộ truy xuất	Hit@1	Hit@5	MRR@10	MRR tiếng Anh gốc	Giữ lại
BM25	0,006	0,018	0,015	0,302	5%
Dày — MiniLM	0,145	0,476	0,275	0,424	65%
Dày — MPNet	0,157	0,476	0,293	0,398	74%
Dày — E5-large	**0,187**	**0,572**	**0,352**	0,439	**80%**

Truy xuất chéo ngôn ngữ là kịch bản thực tế của chatbot: người dùng hỏi tiếng Việt, còn phần lớn tài liệu y khoa chất lượng cao là tiếng Anh. BM25 gần như vô dụng (MRR 0,015), vì không có từ chung giữa hai ngôn ngữ. Mô hình nhúng đa ngữ giữ được 65–80% chất lượng so với khi hỏi bằng tiếng Anh, và E5-large giữ tốt nhất. Kết quả này ủng hộ việc mở rộng KB bằng tài liệu gốc tiếng Anh của ADA mà không cần dịch, với điều kiện dùng mô hình nhúng đa ngữ mạnh.

5.4.4. Kết quả sinh câu trả lời

Bảng 5.8. Chất lượng câu trả lời của Gemini theo ba chế độ (60 câu hỏi tiếng Việt)

Chế độ	n	Phủ dữ kiện	Trích dẫn đúng	Số không căn cứ	Từ chối	Độ dài
Chỉ LLM	52	0,904	–	–	0%	340 ký tự
RAG (lai, 3 đoạn)	51	**0,951**	0,902	0%	7,8%	182 ký tự
GraphRAG (3 đoạn + bộ ba)	55	0,945	**0,945**	0%	7,3%	181 ký tự

Cột *n* là số câu có câu trả lời. Các câu còn lại bị lỗi API của Gemini (quá tải hoặc giới hạn tần suất) sau 3 lần thử, và được loại khỏi phép tính.

Nhận xét. (i) *LLM đơn thuần đã biết nhiều*: Gemini trả lời đúng 90% dữ kiện mà không cần tài liệu, vì các ngưỡng ADA là kiến thức phổ biến. Điều này cho thấy giá trị chính của RAG trong y tế *không chỉ là độ chính xác*. (ii) RAG nâng độ phủ dữ kiện lên

0,95 và cho câu trả lời *ngắn gọn bằng một nửa* (182 so với 340 ký tự). (iii) Quan trọng nhất, RAG cho câu trả lời *kiểm chứng được*: 90–95% câu trích đúng đoạn vàng, và *không có con số nào* trong câu trả lời nằm ngoài ngữ cảnh được cung cấp. Câu trả lời của chế độ chỉ LLM không có nguồn, nên người dùng không có cách nào biết 10% sai nằm ở đâu. (iv) Khoảng 7–8% câu, mô hình chọn trả lời "không đủ dữ liệu". Đó là hành vi mong muốn khi ngữ cảnh truy xuất không chứa đáp án (độ phủ ngữ cảnh 92–95%), thay vì đoán. (v) GraphRAG có tỉ lệ ngữ cảnh chứa đoạn vàng cao hơn RAG (0,945 so với 0,922), nhờ đó tỉ lệ trích dẫn đúng cao hơn 4 điểm. Điều này khớp với kết quả truy xuất: đồ thị cải thiện độ phủ.

5.4.5. Ví dụ: một câu hỏi đa bước qua ba chế độ

Bảng 5.9 trình bày câu trả lời thật của Gemini cho câu hỏi đa bước "Bệnh nhân đái tháo đường có eGFR 25 thì còn dùng metformin được không, nên thay bằng thuốc gì?". Hai dữ kiện vàng là ngưỡng chống chỉ định "eGFR < 30" và một nhóm thuốc thay thế phù hợp.

Bảng 5.9. Câu trả lời thật của ba chế độ cho một câu hỏi đa bước

Chế độ	Câu trả lời (rút gọn)	Đánh giá
Chỉ LLM	"…eGFR 25 **chống chỉ định** với Metformin do nguy cơ nhiễm toan lactic. Nên thay bằng …GLP-1 RA hoặc SGLT2i (lưu ý chỉnh liều theo eGFR). **Thuốc ức chế DPP-4 (như Linagliptin)** cũng là một lựa chọn an toàn…"	Phủ 1/2; không nguồn; có thông tin ngoài KB
RAG	"Metformin bị chống chỉ định khi eGFR < 30 mL/phút/1,73 m² [1], vì vậy bệnh nhân có eGFR 25 không được tiếp tục dùng thuốc này. **Tài liệu không đề cập cụ thể** bệnh nhân nên được thay thế bằng thuốc gì… [3]."	Phủ 1/2; trích đúng; trung thực
GraphRAG	"Bệnh nhân có eGFR 25 (eGFR < 30) có chống chỉ định dùng metformin [1]. Tài liệu không đề cập việc thay thế cụ thể…, nhưng **việc dùng thuốc ức chế SGLT2** có chỉ định giúp làm chậm tiến triển bệnh thận đái tháo đường [3]."	Phủ 2/2; trích đúng

Ví dụ này cho thấy rõ khác biệt về bản chất giữa ba chế độ. LLM đơn thuần trả lời

trôi chảy, phần lớn đúng theo kiến thức chung, nhưng đưa ra một nhóm thuốc (DPP-4) không có trong tài liệu nguồn và không kiểm chứng được trong hệ thống. RAG trung thành với ba đoạn truy xuất được: các đoạn này không nói về thuốc thay thế, nên mô hình nói rõ "tài liệu không đề cập" thay vì đoán. GraphRAG đi từ thực thể *Suy thận nặng* và *Bệnh thận ĐTĐ* theo cạnh `TREATED_BY` tới *Ức chế SGLT2*, đưa đúng đoạn "Thuốc bảo vệ tim mạch và thận" vào ngữ cảnh, nhờ đó trả lời đủ cả hai ý và vẫn có trích dẫn. Đây là kiểu câu hỏi "nối hai mẩu tri thức" mà GraphRAG được thiết kế để giải quyết.

5.5. Triển khai

KnowledgeAgent chạy truy xuất lai trên PostgreSQL (pgvector + full-text, gộp RRF) kết hợp GraphRAG bằng truy vấn Cypher trên Apache AGE, như đã mô tả ở trên. Ba endpoint phục vụ thử nghiệm:

- `POST /api/lab/ch5/search` với `mode` ∈ {`vector`, `fulltext`, `hybrid`, `graphrag`}: trả về các đoạn, điểm, độ trễ; với GraphRAG còn trả về thực thể nhận ra và các cạnh đã đi qua.
- `POST /api/lab/ch5/ask`: hỏi đáp đầy đủ qua hệ đa tác tử (Orchestrator phân loại ý định `KNOWLEDGE_QUESTION` → KnowledgeAgent → ResponseAgent), trả về câu trả lời có trích dẫn.
- `GET /api/lab/ch5/graph`: toàn bộ đồ thị tri thức dạng JSON (nút, cạnh, bằng chứng, nguồn).

Đoạn mã 5.3. Gọi endpoint truy xuất GraphRAG của Chương 5 và kết quả (rút gọn)

```
curl -s https://agent.nits.io.vn/api/lab/ch5/search -H 'Content-Type: application/json' \
  -d '{"query":"Bệnh thận mạn có albumin niệu nên dùng thuốc gì?","mode":"graphrag","k":5}'
# {"mode": "graphrag", "graph": {"entities": ["Bệnh thận ĐTĐ"],
#    "edges": ["Đái tháo đường típ 2 —CAUSES→ Bệnh thận ĐTĐ",
#              "Bệnh thận ĐTĐ —TREATED_BY→ Ức chế SGLT2",
#              "Bệnh thận ĐTĐ —SCREENED_BY→ uACR và eGFR"]},
#  "results": [{"chunk_id": 25, "heading": "Thuốc bảo vệ tim mạch và thận", ...}, ...]}
```

5.6. Tiểu kết chương 5

Chương 5 trình bày đồ thị tri thức (bộ ba, ontology, Cypher, xây dựng và suy luận) và RAG (chia đoạn, truy xuất thưa BM25, truy xuất dày, RRF, xếp hạng lại, GraphRAG). Thực nghiệm trên KB tiếng Việt và MedQuAD cho năm kết luận. (1) Chất lượng mô hình nhúng là yếu tố quan trọng nhất: E5-large vượt MiniLM trên cả hai bộ và trong truy xuất chéo ngôn ngữ. (2) BM25 rất mạnh với thuật ngữ y khoa tiếng Việt nhưng vô dụng

khi khác ngôn ngữ. RRF chỉ có lợi khi hai thành phần cân sức. (3) Xếp hạng lại cho kết quả gần tuyệt đối nhưng quá chậm trên CPU. (4) GraphRAG cải thiện độ phủ và câu hỏi đa bước, giúp trích dẫn đúng hơn. (5) Với Gemini, RAG không chỉ tăng độ chính xác mà quan trọng hơn là biến câu trả lời thành thứ *kiểm chứng được*: có trích dẫn, không có số liệu ngoài nguồn, biết từ chối khi thiếu thông tin. Các bước nâng cấp tiếp theo cho production đã rõ: dùng E5 thay MiniLM và BM25 thay `ts_rank_cd`.

CHƯƠNG 6. HỌC SÂU TRÊN ĐỒ THỊ

Nhiều loại dữ liệu y sinh có dạng đồ thị: bài báo trích dẫn nhau, protein tương tác với nhau, thuốc tương tác với thuốc, bệnh đồng mắc với bệnh. Ở Chương 5, đồ thị tri thức được duyệt bằng truy vấn viết tay. Chương này để mạng nơ-ron *tự học* từ cấu trúc đồ thị. Nội dung gồm nguyên lý truyền thông điệp, ba kiến trúc tiêu biểu (GCN, GraphSAGE, GAT), hiện tượng làm trơn quá mức, và thực nghiệm phân loại 19.717 bài báo PubMed về đái tháo đường. Trong hệ thống, mô hình này dùng để gán nhãn "thực nghiệm / típ 1 / típ 2" cho y văn trước khi đưa vào Knowledge Base.

6.1. Dữ liệu đồ thị và các bài toán

Một đồ thị $G = (V, E)$ có $n = |V|$ nút, ma trận kề $\mathbf{A} \in \{0,1\}^{n\times n}$, ma trận bậc $\mathbf{D} = \mathrm{diag}(d_1, \dots, d_n)$ với $d_i = \sum_j A_{ij}$, và ma trận đặc trưng nút $\mathbf{X} \in \mathbb{R}^{n\times d}$. Đồ thị khác ảnh và chuỗi ở ba điểm: số láng giềng thay đổi theo nút; *không có thứ tự* giữa các láng giềng; và các nút không độc lập với nhau. Do đó mô hình phải *bất biến với hoán vị*, tức là đánh số lại nút thì kết quả không đổi. Có ba họ bài toán: *cấp nút* (phân loại bài báo, phát hiện gian lận), *cấp cạnh* (dự đoán liên kết, gợi ý thuốc, tác dụng phụ khi phối hợp thuốc [85]), và *cấp đồ thị* (dự đoán tính chất phân tử).

Trước GNN, đã có hai hướng. *Lan truyền nhãn* [86] chỉ dùng cấu trúc đồ thị, với giả định các nút nối nhau thường cùng nhãn (*đồng tính*, homophily [87]). *Nhúng nút* như DeepWalk [88] học vector cho từng nút từ các bước đi ngẫu nhiên, nhưng không tận dụng đặc trưng nút và không tổng quát được sang nút mới.

6.2. Mạng nơ-ron đồ thị

6.2.1. Khung truyền thông điệp

Gilmer và cộng sự [89] thống nhất phần lớn các GNN vào khung *truyền thông điệp*. Ở tầng l, mỗi nút v nhận thông điệp từ láng giềng, gộp chúng bằng một hàm bất biến hoán vị (tổng, trung bình, max), rồi cập nhật biểu diễn của mình:

$$\begin{aligned} \mathbf{m}_v^{(l)} &= \operatorname*{AGG}_{u\in\mathcal{N}(v)} \phi^{(l)}\big(\mathbf{h}_u^{(l)}, \mathbf{h}_v^{(l)}\big), \\ \mathbf{h}_v^{(l+1)} &= \psi^{(l)}\big(\mathbf{h}_v^{(l)}, \mathbf{m}_v^{(l)}\big), \qquad \mathbf{h}_v^{(0)} = \mathbf{x}_v. \end{aligned} \tag{6.1}$$

Sau L tầng, $\mathbf{h}_v^{(L)}$ tổng hợp thông tin của vùng lân cận L bước quanh v, tương tự trường tiếp nhận của CNN. CNN thực chất là trường hợp đặc biệt của truyền thông điệp trên đồ thị lưới, trong đó mỗi điểm ảnh có đúng 8 láng giềng theo thứ tự cố định. GNN tổng quát hóa phép tích chập sang láng giềng không có thứ tự và số lượng thay đổi [90].

6.2.2. Nền tảng phổ: từ biến đổi Fourier trên đồ thị đến GCN

GCN có thể được dẫn ra từ lý thuyết đồ thị phổ. Ma trận Laplace chuẩn hóa $\mathbf{L} = \mathbf{I} - \mathbf{D}^{-1/2}\mathbf{A}\mathbf{D}^{-1/2}$ là ma trận đối xứng nửa xác định dương, phân rã được thành $\mathbf{L} = \mathbf{U}\Lambda\mathbf{U}^\top$ với các trị riêng $0 = \lambda_1 \le \dots \le \lambda_n \le 2$. Các vector riêng $\mathbf{U}$ đóng vai trò "các tần số" trên đồ thị. Vector riêng ứng với trị riêng nhỏ biến thiên chậm giữa các nút kề nhau (tần số thấp), trị riêng lớn ứng với tín hiệu dao động mạnh giữa các nút kề nhau (tần số cao). *Biến đổi Fourier trên đồ thị* của tín hiệu $\mathbf{x} \in \mathbb{R}^n$ là $\hat{\mathbf{x}} = \mathbf{U}^\top\mathbf{x}$, và *tích chập phổ* với bộ lọc g_θ là:

$$g_\theta \star \mathbf{x} = \mathbf{U}\, g_\theta(\Lambda)\, \mathbf{U}^\top\mathbf{x}. \tag{6.2}$$

Công thức (6.2) đẹp nhưng không dùng được trực tiếp: phân rã trị riêng tốn $O(n^3)$, bộ lọc không cục bộ, và không áp dụng được cho đồ thị khác. ChebNet xấp xỉ $g_\theta(\Lambda)$ bằng đa thức Chebyshev bậc K, $g_\theta \approx \sum_{k=0}^{K} \theta_k T_k(\tilde{\mathbf{L}})$. Bộ lọc khi đó chỉ phụ thuộc lân cận K bước và tính được bằng phép nhân ma trận thưa. Kipf và Welling chọn $K = 1$, xấp xỉ $\lambda_{\max} \approx 2$ và dùng chung một tham số $\theta = \theta_0 = -\theta_1$, được $g_\theta \star \mathbf{x} \approx \theta(\mathbf{I} + \mathbf{D}^{-1/2}\mathbf{A}\mathbf{D}^{-1/2})\mathbf{x}$. Toán tử $\mathbf{I} + \mathbf{D}^{-1/2}\mathbf{A}\mathbf{D}^{-1/2}$ có trị riêng trong $[0, 2]$, nên chồng nhiều tầng có thể làm số liệu bất ổn. Để tránh điều này, họ dùng *thủ thuật chuẩn hóa lại*: thay bằng $\tilde{\mathbf{D}}^{-1/2}\tilde{\mathbf{A}}\tilde{\mathbf{D}}^{-1/2}$ với $\tilde{\mathbf{A}} = \mathbf{A} + \mathbf{I}$, chính là $\hat{\mathbf{A}}$ trong (6.3). Góc nhìn phổ cũng giải thích bản chất của GCN: nó là một *bộ lọc thông thấp*, giữ thành phần biến thiên chậm (các nút kề nhau giống nhau) và triệt thành phần dao động. Bộ lọc này phù hợp với đồ thị đồng tính, và áp dụng lặp lại nhiều lần thì dẫn tới làm trơn quá mức.

6.2.3. Mạng tích chập đồ thị (GCN)

Kipf và Welling [91] rút ra GCN từ xấp xỉ bậc nhất của tích chập phổ trên đồ thị. Thêm khuyên vào mỗi nút để nút giữ thông tin của chính mình, $\tilde{\mathbf{A}} = \mathbf{A} + \mathbf{I}$, $\tilde{\mathbf{D}} = \mathrm{diag}(\tilde{\mathbf{A}}\mathbf{1})$. Một tầng GCN là:

$$\mathbf{H}^{(l+1)} = \sigma\!\left(\hat{\mathbf{A}}\,\mathbf{H}^{(l)}\mathbf{W}^{(l)}\right), \qquad \hat{\mathbf{A}} = \tilde{\mathbf{D}}^{-1/2}\tilde{\mathbf{A}}\tilde{\mathbf{D}}^{-1/2}. \tag{6.3}$$

Viết theo từng nút: $\mathbf{h}'_v = \sigma\big(\sum_{u \in \mathcal{N}(v)\cup\{v\}} \frac{1}{\sqrt{\tilde{d}_u \tilde{d}_v}} \mathbf{W}\mathbf{h}_u\big)$. Mỗi tầng thực hiện hai phép toán. (i) *Biến đổi đặc trưng* $\mathbf{HW}$, giống một lớp tuyến tính dùng chung cho mọi nút. (ii) *Gộp láng giềng* có chuẩn hóa đối xứng theo bậc: một nút có nhiều liên kết (bài tổng quan được trích dẫn nhiều) đóng góp ít hơn cho mỗi láng giềng, nên không lấn át. $\hat{\mathbf{A}}$ là ma trận thưa với $|E|$ phần tử khác 0, nên một tầng tốn $O(|E|\,d)$. Mô hình hai tầng cho phân loại nút:

$$\begin{aligned} \mathbf{Z} &= \operatorname{softmax}\Big(\hat{\mathbf{A}}\ \operatorname{ReLU}\left(\hat{\mathbf{A}}\mathbf{X}\mathbf{W}^{(0)}\right)\mathbf{W}^{(1)}\Big), \\ \mathscr{L} &= -\sum_{v \in V_L} \sum_{c} Y_{vc} \log Z_{vc}, \end{aligned} \tag{6.4}$$

với mất mát chỉ tính trên tập nút có nhãn V_L. Học bán giám sát diễn ra tự nhiên: nhãn của vài nút lan qua cạnh tới các nút chưa có nhãn nhờ phép gộp, trong khi đặc trưng của mọi nút (kể cả nút không nhãn) đều tham gia vào lan truyền xuôi.

Ví dụ tính tay. Xét đồ thị hình sao 4 nút: nút 2 là bài tổng quan được ba bài 1, 3, 4 trích dẫn (đánh số 1–4). Bậc sau khi thêm khuyên là $\tilde{\mathbf{d}} = (2, 4, 2, 2)$, và

$$\hat{\mathbf{A}} = \tilde{\mathbf{D}}^{-1/2}(\mathbf{A} + \mathbf{I})\tilde{\mathbf{D}}^{-1/2} = \begin{bmatrix} 0{,}5 & 0{,}354 & 0 & 0 \\ 0{,}354 & 0{,}25 & 0{,}354 & 0{,}354 \\ 0 & 0{,}354 & 0{,}5 & 0 \\ 0 & 0{,}354 & 0 & 0{,}5 \end{bmatrix},$$

với $\hat{A}_{12} = 1/\sqrt{2 \cdot 4} = 0{,}354$ và $\hat{A}_{22} = 1/4$. Giả sử một đặc trưng là "bài có từ *mice*" với $\mathbf{x} = (1, 0, 0, 1)^\top$, tức bài 1 và 4 nghiên cứu trên chuột. Một bước gộp cho $\hat{\mathbf{A}}\mathbf{x} = (0{,}5;\ 0{,}707;\ 0;\ 0{,}5)$. Bài tổng quan (nút 2) nhận tín hiệu "thực nghiệm" mạnh nhất (0,707) từ hai láng giềng, dù bản thân nó không có từ đó. Bài 3 vẫn bằng 0 vì chưa có đường đi độ dài 1 tới nút nguồn. Sau bước thứ hai, $\hat{\mathbf{A}}^2\mathbf{x} = (0{,}5;\ 0{,}53;\ 0{,}25;\ 0{,}5)$: tín hiệu đã lan tới bài 3 qua bài tổng quan, đồng thời giá trị các nút bắt đầu "san bằng". Ví dụ này cho thấy cả hai mặt của truyền thông điệp: *lan truyền thông tin* theo cấu trúc trích dẫn, và *làm trơn*. Tiếp tục nhân nhiều lần, mọi nút sẽ hội tụ về cùng một giá trị (§6.2.6).

6.2.4. GraphSAGE

GCN cần toàn bộ ma trận $\hat{\mathbf{A}}$ khi huấn luyện (transductive). GraphSAGE [92] học *hàm gộp* thay vì học trên một đồ thị cố định, nên áp dụng được cho nút mới (inductive):

$$\mathbf{h}'_v = \sigma\Big(\mathbf{W}_{\text{self}}\mathbf{h}_v + \mathbf{W}_{\text{neigh}} \cdot \frac{1}{|\mathcal{N}(v)|}\sum_{u\in\mathcal{N}(v)}\mathbf{h}_u\Big). \tag{6.5}$$

Khác biệt với GCN là biểu diễn của chính nút và của láng giềng có *hai ma trận trọng số riêng*. Mô hình nhờ vậy học được mức tin vào bản thân và mức tin vào láng giềng. Với đồ thị lớn, GraphSAGE lấy mẫu một số láng giềng cố định để giới hạn chi phí.

6.2.5. Mạng chú ý trên đồ thị (GAT)

GCN gán trọng số cố định $1/\sqrt{\tilde{d}_u\tilde{d}_v}$ cho mọi láng giềng. GAT [93] *học* trọng số đó bằng cơ chế chú ý:

$$\begin{aligned} e_{vu} &= \text{LeakyReLU}(\mathbf{a}^\top[\mathbf{W}\mathbf{h}_v \,\|\, \mathbf{W}\mathbf{h}_u]),\\ \alpha_{vu} &= \frac{\exp(e_{vu})}{\sum_{k\in\mathcal{N}(v)\cup\{v\}}\exp(e_{vk})},\\ \mathbf{h}'_v &= \Big\|_{k=1}^{K}\,\sigma\Big(\sum_{u}\alpha_{vu}^{k}\mathbf{W}^k\mathbf{h}_u\Big). \end{aligned} \tag{6.6}$$

Hệ số α_{vu} là "mức quan trọng" của láng giềng u đối với v, được chuẩn hóa bằng softmax trên các láng giềng của v. *K đầu chú ý* độc lập được nối lại ($\|$) ở tầng ẩn và lấy trung bình ở tầng ra, giúp ổn định quá trình học. Vì $\mathbf{a}^\top[\mathbf{W}\mathbf{h}_v\|\mathbf{W}\mathbf{h}_u] = \mathbf{a}_{\text{dst}}^\top\mathbf{W}\mathbf{h}_v + \mathbf{a}_{\text{src}}^\top\mathbf{W}\mathbf{h}_u$, điểm chú ý tính được theo từng cạnh chỉ với hai phép tra chỉ số. Trọng số α còn có giá trị giải thích: trang thử nghiệm hiển thị các bài báo láng giềng mà GAT "chú ý" nhiều nhất.

6.2.6. Làm trơn quá mức

Mỗi phép nhân với $\hat{\mathbf{A}}$ là một bước khuếch tán, kéo biểu diễn của các nút kề nhau về gần nhau. Khi $L \to \infty$, $\hat{\mathbf{A}}^L$ hội tụ về một ma trận hạng một (với đồ thị liên thông), nên mọi nút có cùng biểu diễn, chỉ khác một hệ số tỉ lệ theo bậc [94]. Đó là hiện tượng *làm trơn quá mức* (over-smoothing). Vì vậy GNN thường chỉ có 2–3 tầng, rất khác với CNN hàng trăm lớp. Chen và cộng sự [95] đo hiện tượng này bằng MAD (Mean Average Distance), tức khoảng cách cosin trung bình giữa biểu diễn các cặp nút; MAD tiến về 0 khi các nút không còn phân biệt được.

6.2.7. Năng lực biểu diễn và phép thử Weisfeiler–Lehman

GNN có thể phân biệt được những cấu trúc nào? Xu và cộng sự [96] chứng minh rằng mọi GNN truyền thông điệp đều *không mạnh hơn* phép thử đẳng cấu đồ thị Weisfeiler–Lehman bậc 1 (1-WL). Phép thử này lặp lại việc gán cho mỗi nút một "màu" mới bằng cách băm màu hiện tại cùng *đa tập* màu của láng giềng. Một GNN đạt được sức mạnh của 1-WL khi hàm gộp là *đơn ánh trên đa tập*. Tổng thỏa mãn điều kiện này, còn trung bình và max thì không. Ví dụ, trung bình không phân biệt được láng giềng $\{a, b\}$ với $\{a, a, b, b\}$, còn max không phân biệt được $\{a, b\}$ với $\{a, b, b\}$. Từ đó họ đề xuất GIN, $\mathbf{h}'_v = \mathrm{MLP}\big((1+\epsilon)\mathbf{h}_v + \sum_{u \in \mathcal{N}(v)} \mathbf{h}_u\big)$. Với bài toán phân loại nút trên đồ thị trích dẫn như Pubmed, đặc trưng nút giàu thông tin và giả định đồng tính quan trọng hơn khả năng phân biệt cấu trúc tinh vi. Vì vậy các bộ gộp "yếu" như trung bình (GraphSAGE) hay tổng có chuẩn hóa (GCN) vẫn hoạt động tốt. Năng lực biểu diễn trở nên quyết định ở bài toán cấp đồ thị, như phân biệt hai phân tử có cùng thành phần nguyên tử nhưng khác cấu trúc vòng.

6.2.8. Các bài toán khác và ứng dụng y sinh

Dự đoán liên kết. Bộ mã hóa GNN tạo biểu diễn nút $\mathbf{z}_v$. Xác suất có cạnh được giải mã bằng $\sigma(\mathbf{z}_u^\top \mathbf{z}_v)$ (Graph Auto-Encoder) hoặc bằng một MLP trên cặp $(\mathbf{z}_u, \mathbf{z}_v)$, huấn luyện với cạnh thật làm mẫu dương và cạnh ngẫu nhiên làm mẫu âm. *Phân loại đồ thị.* Sau các tầng truyền thông điệp, một phép *đọc ra* (readout) gộp biểu diễn của mọi nút thành một vector cho cả đồ thị, bằng tổng, trung bình, hoặc pooling phân cấp. Phép đọc ra phải bất biến với hoán vị.

GNN có nhiều ứng dụng trong y sinh liên quan tới đái tháo đường. *Tác dụng phụ khi phối hợp thuốc*: Decagon [85] dự đoán tác dụng phụ của từng cặp thuốc trên đồ thị gồm protein, thuốc và các loại tương tác. Đây là điều quan trọng với người ĐTĐ thường dùng nhiều thuốc (hạ đường huyết, hạ áp, statin). *Tái định vị thuốc*: dự đoán liên kết thuốc–bệnh trên đồ thị tri thức y sinh. *Mạng bệnh đồng mắc*: nút là bệnh nhân hoặc bệnh, cạnh là đồng xuất hiện trong hồ sơ, giúp dự báo biến chứng tim mạch, thận. *Phân tử*: dự đoán tính chất của phân tử ứng viên thuốc, coi nguyên tử là nút và liên kết hóa học là cạnh. *Gán nhãn y văn*: bài toán của chương này, giúp xây dựng KB có cấu trúc từ hàng triệu bài báo.

6.3. Dữ liệu Pubmed-Diabetes

Pubmed-Diabetes [9], [97] là một trong ba bộ chuẩn kinh điển của GNN (cùng Cora và Citeseer), được dùng trong bài báo gốc của GCN và GAT. Mỗi nút là một bài báo trong cơ sở dữ liệu PubMed liên quan tới đái tháo đường. Bài báo được biểu diễn bằng vector TF-IDF trên 500 từ gốc (stem) và gán một trong ba nhãn MeSH: *Diabetes Mellitus, Experimental* (nghiên cứu trên động vật), *Type 1* hoặc *Type 2*. Cạnh là quan hệ trích dẫn.

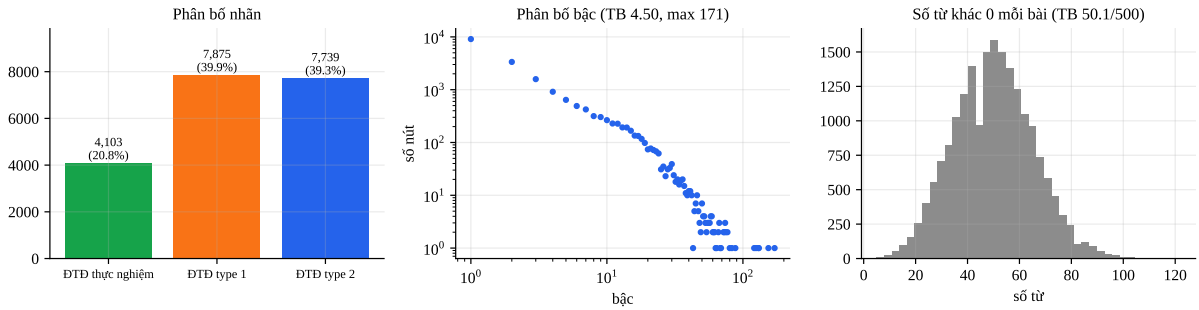
- **Nguồn**: nhóm LINQS, `https://linqs-data.soe.ucsc.edu/public/Pubmed-Diabetes.tgz` (14,7 MB). Bản văn bản gốc dạng TSV được dùng thay cho bản pickle của Planetoid, để đọc được mà không phải chạy mã không tin cậy.
- **Quy mô**: 19.717 nút, 44.338 trích dẫn có hướng. Sau khi chuyển sang vô hướng, bỏ khuyên và cạnh trùng còn 44.324 cạnh. Ma trận đặc trưng 19.717 × 500 rất thưa: trung bình 50,1 từ khác 0 trên mỗi bài.
- **Nhãn**: thực nghiệm 4.103 (20,8%), típ 1 7.875 (39,9%), típ 2 7.739 (39,3%).

Bảng 6.1. Năm nút đầu tiên: mã PubMed, nhãn, bậc và các từ có TF-IDF cao nhất

PubMed ID	Nhãn	Từ nổi bật	Bậc
12187484	Thực nghiệm	retinopathi, mice, abnorm, month, anim, rat	4
2344352	Thực nghiệm	muscl, m, chang, r, stimul, caus	5
14654069	Thực nghiệm	cardiac, state, rat, experiment, defect, mark	2
16443886	Típ 1	a1c, glucos, profil, sampl, predict, larg	1
2684155	Thực nghiệm	secret, cholesterol, h, rate, low, anim	12

Bảng 6.2. Mười từ có TF-IDF trung bình cao nhất của mỗi lớp (dạng từ gốc sau stemming)

Lớp	Từ đặc trưng
ĐTĐ thực nghiệm	rat, mice, nerv, activ, anim, cell, streptozotocin, increas, protein, decreas
ĐTĐ típ 1	cell, iddm, nod, patient, children, mice, t, 0, islet, 1
ĐTĐ típ 2	0, patient, insulin, 2, niddm, glucos, risk, p, type, subject



Hình 6.1. Phân bố nhãn, phân bố bậc (thang log–log) và số từ khác 0 của mỗi bài báo

Nhận xét. (i) Phân bố bậc có *đuôi nặng*: phần lớn bài báo chỉ có 1–5 trích dẫn (bậc trung bình 4,50), một số ít có tới 171. Trên thang log–log, phân bố gần thẳng, đặc trưng của mạng trích dẫn. Không có nút cô lập. (ii) *Tính đồng tính rất cao*: 80,2% cạnh nối hai bài báo cùng nhãn. Đây là điều kiện để GNN phát huy tác dụng; nếu tỉ lệ này thấp, gộp láng giềng sẽ thêm nhiễu thay vì thông tin. (iii) Từ đặc trưng của mỗi lớp hợp lý về mặt y khoa: lớp thực nghiệm có *rat, mice, streptozotocin* (hóa chất gây ĐTĐ trên chuột); típ 1 có *iddm, nod* (chuột NOD), *children, islet*; típ 2 có *niddm, insulin, risk, subject*. (iv) Đồ thị rất thưa (mật độ $2{,}3 \cdot 10^{-4}$), nên phép nhân ma trận thưa trong (6.3) rất rẻ.

6.4. Cài đặt

Cả ba GNN được *tự cài đặt bằng PyTorch thuần*, không dùng PyTorch Geometric [98], để thể hiện rõ từng phép toán của truyền thông điệp. GCN và GraphSAGE dùng ma trận thưa `torch.sparse`: $\hat{\mathbf{A}}$ cho GCN, ma trận trung bình láng giềng $\mathbf{D}^{-1}\mathbf{A}$ cho GraphSAGE. GAT làm việc trên danh sách cạnh: tính điểm chú ý cho từng cạnh, rồi tính softmax theo nút đích bằng `index_reduce` (max, để ổn định số) và `index_add` (tổng) (Đoạn mã 6.1).

Đoạn mã 6.1. Tầng GCN và tầng GAT cài bằng PyTorch thuần (`lab/ch6_gnn.py`, rút gọn)

```
class GCN(nn.Module):                          # H' = sigma(Â H W)
    def forward(self, x, g):
        for i, lin in enumerate(self.lins):
            x = F.dropout(x, self.p, self.training)
            x = torch.sparse.mm(g["A_gcn"], lin(x))     # biến đổi đặc trưng rồi gộp có chuẩn
             ↪  hóa bậc
            if i < len(self.lins) - 1:
                x = F.relu(x)
        return x

class GATLayer(nn.Module):
    def forward(self, x, edge_index):
        src, dst = edge_index                                    # cạnh u → v (đã có khuyên)
        z = self.W(x).view(n, self.heads, self.h)                  # W h, K đầu
        e = F.leaky_relu((z * self.a_src).sum(-1)[src]             # a_src^T W h_u
                         + (z * self.a_dst).sum(-1)[dst], 0.2)     # + a_dst^T W h_v   → [E, K]
        e_max = torch.full((n, K), -1e9).index_reduce_(0, dst, e, "amax")   # max theo nút đích
        ex = torch.exp(e - e_max[dst])
        alpha = ex / torch.zeros(n, K).index_add_(0, dst, ex)[dst]          # softmax trên
         ↪  láng giềng
        out = torch.zeros(n, K, self.h).index_add_(0, dst, alpha[..., None] * z[src])
        return out.reshape(n, -1) if self.concat else out.mean(1)
```

Bảng 6.3. Cấu hình và số tham số của các mô hình (đặc trưng vào 500 chiều, 3 lớp)

Mô hình	Kiến trúc	Tham số	Dùng đồ thị
Hồi quy logistic	Dropout 0,2 → Linear(500, 3)	1.503	Không
MLP	Linear(500, 64) → ReLU → Linear(64, 3), dropout 0,5	32.259	Không
Lan truyền nhãn	$\mathbf{F} \leftarrow 0{,}9\,\hat{\mathbf{A}}\mathbf{F} + 0{,}1\,\mathbf{Y}$, 50 bước	0	Chỉ đồ thị
GCN	2 tầng (6.3), 500 → 64 → 3, dropout 0,5	32.259	Có
GraphSAGE-mean	2 tầng (6.5), 500 → 64 → 3	64.451	Có
GAT	8 đầu × 8 chiều (nối, ELU) → 8 đầu ra (trung bình), dropout 0,6	33.779	Có

GCN và MLP có *đúng cùng số tham số* ($32.259 = 500{\cdot}64 + 64 + 64{\cdot}3 + 3$). Khác biệt duy nhất là phép nhân với $\hat{\mathbf{A}}$. So sánh hai mô hình này vì vậy đo chính xác giá trị của cấu trúc đồ thị.

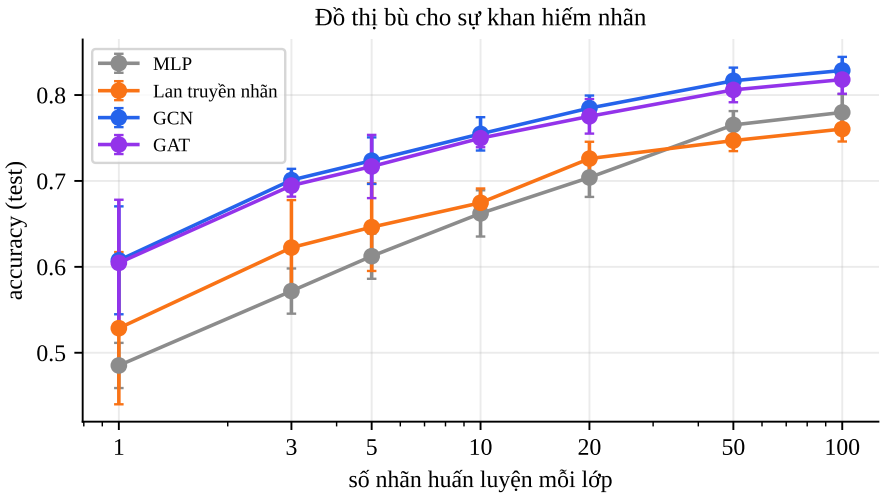
Hai giao thức. (1) *Bán giám sát kiểu Planetoid* [99]: 20 nút có nhãn mỗi lớp (60 nút, tức 0,3% đồ thị), 500 nút validation, 1.000 nút test, lặp với 10 phép chia ngẫu nhiên. (2) *Có giám sát đầy đủ*: chia 60/20/20, lặp 3 lần. Tối ưu Adam (tốc độ học 0,01; suy giảm trọng số $5 \cdot 10^{-4}$, GAT 10^{-3}), tối đa 300–400 epoch, dừng sớm sau 50 epoch không cải thiện accuracy validation. Huấn luyện trên GPU, mỗi lần chạy 0,4–10 giây.

6.5. Kết quả và nhận xét

Bảng 6.4. Accuracy phân loại nút trên Pubmed-Diabetes (trung bình ± độ lệch chuẩn)

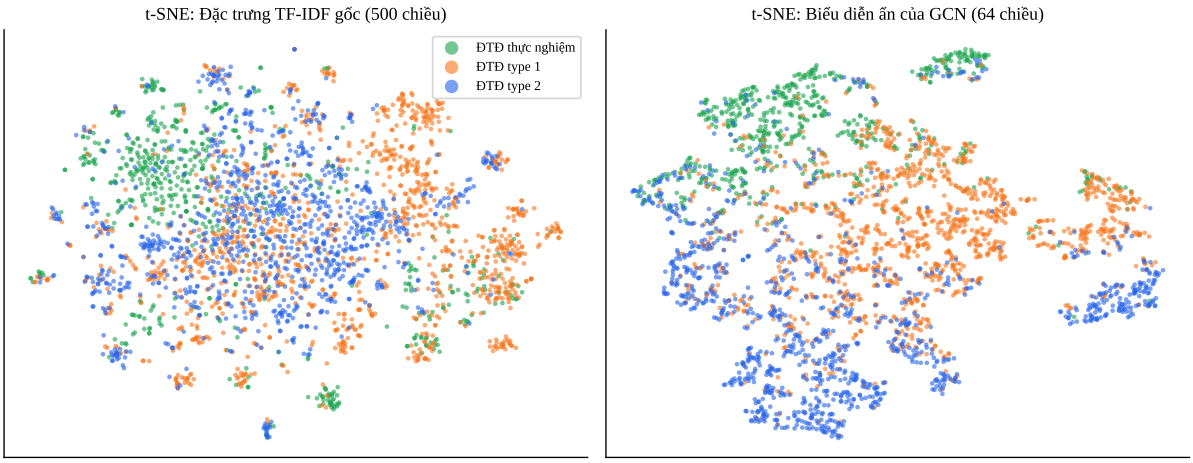
	Bán giám sát (20 nhãn/lớp, 10 lần)		**Có giám sát (60/20/20, 3 lần)**	
Mô hình	**Accuracy**	**Macro-F1**	**Accuracy**	**Macro-F1**
Hồi quy logistic	0,707 ± 0,009	0,712	0,843 ± 0,005	0,843
MLP	0,702 ± 0,019	0,706	0,871 ± 0,004	0,871
Lan truyền nhãn	0,718 ± 0,021	0,702	0,836 ± 0,005	0,824
GCN	**0,780** ± 0,020	**0,777**	0,877 ± 0,005	0,872
GraphSAGE-mean	0,757 ± 0,026	0,755	**0,885** ± 0,001	**0,882**
GAT	0,777 ± 0,019	0,773	0,851 ± 0,003	0,844

Bán giám sát: đồ thị mang lại +7,8 điểm. Khi chỉ có 60 nhãn, GCN đạt 0,780 so với 0,702 của MLP cùng kiến trúc và cùng số tham số. Cả hai nguồn thông tin đều cần thiết. Chỉ dùng đặc trưng (MLP, hồi quy logistic) đạt khoảng 0,70. Chỉ dùng đồ thị (lan truyền nhãn) đạt 0,718. GCN kết hợp cả hai và vượt cả hai. Kết quả của GCN (0,780) và GAT (0,777) sát với con số công bố trên phép chia chuẩn của Planetoid (0,790 cho cả hai [91], [93]). Phần chênh nhỏ đến từ việc báo cáo này dùng 10 phép chia ngẫu nhiên thay cho một phép chia cố định. Trên Pubmed, GAT không vượt GCN: với đồ thị đồng tính cao và bậc thấp, trọng số chuẩn hóa theo bậc của GCN đã gần tối ưu, và chú ý học được thêm ít thông tin.



Hình 6.2. Accuracy theo số nhãn huấn luyện mỗi lớp: đồ thị có lợi nhất khi nhãn khan hiếm

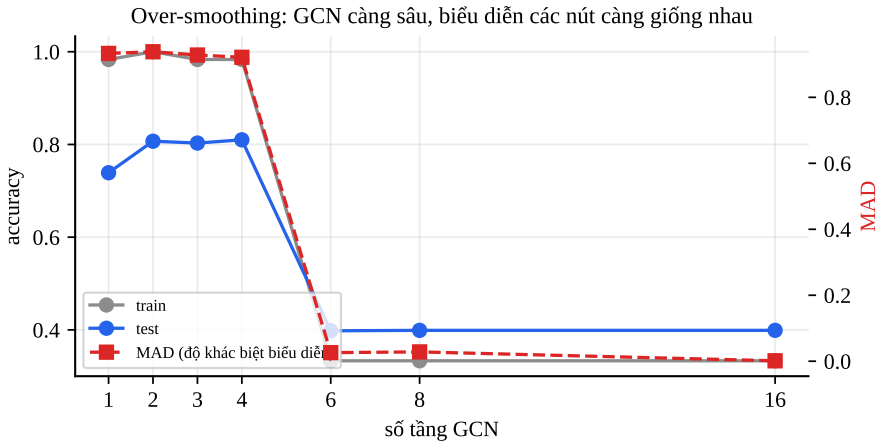
Giá trị của đồ thị giảm khi có nhiều nhãn. Hình 6.2 cho thấy với *một* nhãn mỗi lớp (3 nút có nhãn trên 19.717), GCN đạt 0,608 trong khi MLP chỉ 0,485, chênh 12,2 điểm. Khoảng cách thu hẹp dần: 11,1 điểm ở 5 nhãn/lớp, 8,1 ở 20, 4,9 ở 100. Ở giao thức có giám sát (khoảng 3.900 nhãn mỗi lớp), MLP đã đạt 0,871 và GCN chỉ hơn 0,6 điểm. Khi nhãn dồi dào, đặc trưng TF-IDF của chính bài báo đủ để phân loại, và láng giềng chủ yếu bổ sung thông tin ở những ca khó. GraphSAGE thắng ở giao thức này (0,885) nhờ tách $\mathbf{W}_{\text{self}}$ khỏi $\mathbf{W}_{\text{neigh}}$: với nhiều dữ liệu, mô hình học được cách tin vào đặc trưng của chính nút hơn. Ngược lại, GAT với dropout 0,6 trên cả đặc trưng lẫn hệ số chú ý bị điều chuẩn quá mạnh ở chế độ nhiều dữ liệu (0,851). Siêu tham số này được thiết kế cho chế độ ít nhãn.



Hình 6.3. t-SNE của 3.000 bài báo: đặc trưng TF-IDF gốc (trái) và biểu diễn ẩn của GCN (phải)

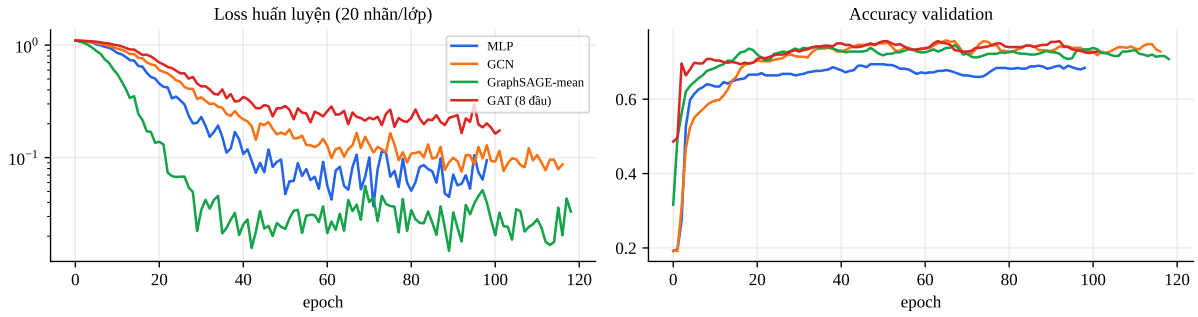
Biểu diễn học được. Trên đặc trưng TF-IDF gốc, ba lớp trộn lẫn, chỉ lớp thực nghiệm hơi tách ra (Hình 6.3, trái). Sau một tầng GCN, dù chỉ huấn luyện với 60 nhãn,

biểu diễn 64 chiều tách thành ba vùng rõ rệt (phải). Phép gộp láng giềng đã "kéo" các bài báo cùng chủ đề lại gần nhau, đúng như giả định đồng tính.



Hình 6.4. Làm trơn quá mức: accuracy và MAD của GCN theo số tầng

Làm trơn quá mức. Hình 6.4 khảo sát GCN từ 1 đến 16 tầng (20 nhãn/lớp). Một tầng chỉ thấy láng giềng trực tiếp (0,739). Hai đến bốn tầng tốt nhất (0,803–0,810). Từ 6 tầng trở đi, mô hình *sụp hoàn toàn*: accuracy 0,399, đúng bằng tỉ lệ lớp típ 1 đông nhất (7.875/19.717). Nghĩa là mô hình dự đoán cùng một lớp cho mọi nút. MAD giảm từ 0,92–0,94 xuống 0,026 (6 tầng) và 0,001 (16 tầng): biểu diễn của mọi nút gần như trùng nhau. Cần nói rõ, sự sụp đổ đột ngột này là tổng hợp của hai nguyên nhân: làm trơn quá mức theo lý thuyết ở trên, và khó tối ưu của mạng sâu không có kết nối tắt, với dropout 0,5 ở mỗi tầng. Các kỹ thuật như kết nối tắt kiểu ResNet, JKNet hay PairNorm được đề xuất để giảm cả hai vấn đề.



Hình 6.5. Loss huấn luyện và accuracy validation theo epoch (20 nhãn/lớp, một phép chia)

Hình 6.5 cho thấy quá trình huấn luyện ở chế độ ít nhãn. Chỉ có 60 nút có nhãn, nên loss huấn luyện giảm rất nhanh: GraphSAGE, với hai ma trận trọng số và gấp đôi tham số, khớp gần như hoàn toàn (loss khoảng 0,03); MLP và GCN dừng quanh 0,1; GAT giữ loss cao nhất (khoảng 0,2) vì dropout 0,6 trên cả đặc trưng lẫn hệ số chú ý. Accuracy

validation mới là tín hiệu thật. GAT tăng nhanh nhất, đạt khoảng 0,7 ngay sau vài epoch. Ba GNN vượt 0,75 sau khoảng 30 epoch, còn MLP dừng ở khoảng 0,70. Loss thấp nhất không tương ứng với accuracy cao nhất: GraphSAGE khớp sát 60 nhãn nhất nhưng tổng quát kém hơn GCN, dấu hiệu quá khớp khi dữ liệu nhãn quá ít. Dừng sớm theo accuracy validation, với kiên nhẫn 50 epoch, giữ lại trọng số tốt nhất.

6.5.1. Ablation các lựa chọn thiết kế của GCN

Bảng 6.5. Ablation GCN trên giao thức bán giám sát (20 nhãn/lớp, 10 phép chia)

Cấu hình	Accuracy	Chênh lệch
GCN chuẩn: khuyên, $\tilde{\mathbf{D}}^{-1/2}\tilde{\mathbf{A}}\tilde{\mathbf{D}}^{-1/2}$, ẩn 64, dropout 0,5	**0,780** ± 0,020	–
Không có khuyên ($\mathbf{A}$ thay cho $\mathbf{A} + \mathbf{I}$)	0,764 ± 0,023	−1,6
Chuẩn hóa theo hàng $\tilde{\mathbf{D}}^{-1}\tilde{\mathbf{A}}$ (trung bình láng giềng)	0,775 ± 0,017	−0,5
Không chuẩn hóa (tổng láng giềng)	0,765 ± 0,017	−1,6
Không dropout	0,776 ± 0,021	−0,5
Kích thước ẩn 16	0,782 ± 0,019	+0,1
Kích thước ẩn 256	0,782 ± 0,014	+0,1

Bảng 6.5 xác nhận các lựa chọn của Kipf và Welling bằng thực nghiệm. (i) *Khuyên* quan trọng nhất: bỏ khuyên, biểu diễn mới của một nút chỉ là tổng hợp của láng giềng, đặc trưng của chính nó bị mất sau mỗi tầng. Với 63% số nút chỉ có 1–2 láng giềng, mất thông tin bản thân là thiệt hại lớn (−1,6 điểm). (ii) *Chuẩn hóa*: không chuẩn hóa thì nút bậc cao có biểu diễn lớn bất thường, làm số liệu bất ổn (−1,6). Chuẩn hóa theo hàng (lấy trung bình) gần bằng chuẩn hóa đối xứng (−0,5). Chuẩn hóa đối xứng còn giảm thêm ảnh hưởng của láng giềng bậc cao, như bài tổng quan được trích dẫn rộng rãi ít đặc trưng cho một chủ đề cụ thể. (iii) *Dropout* giúp nhẹ (+0,5) khi chỉ có 60 nhãn. (iv) *Kích thước ẩn* từ 16 tới 256 gần như không ảnh hưởng. Nút thắt không nằm ở dung lượng mô hình mà ở lượng nhãn và thông tin trong đồ thị. Kết quả này khớp với kết luận của Chương 2.

6.5.2. Học chuyển dẫn và học quy nạp

GCN thường được mô tả là *chuyển dẫn* (transductive): đồ thị đầy đủ, kể cả nút cần dự đoán, có mặt lúc huấn luyện. GraphSAGE nhấn mạnh tính *quy nạp* (inductive): áp dụng được cho nút mới chưa từng thấy. Đây chính là tình huống của hệ thống khi một bài báo mới cần được gán nhãn trước khi nạp vào KB. Thí nghiệm sau kiểm tra trực tiếp

sự khác biệt này. Với phép chia 60/20/20, khi huấn luyện, *xóa toàn bộ nút kiểm tra cùng mọi cạnh nối tới chúng*: chỉ còn 29.175 trên 44.324 cạnh, tức đồ thị mất 34% số cạnh. Lúc suy luận, các nút kiểm tra mới được thêm lại vào đồ thị.

Bảng 6.6. Học chuyển dẫn và học quy nạp trên Pubmed-Diabetes (60/20/20, 3 lần chạy)

Mô hình	Chuyển dẫn	Quy nạp (nút test không có khi huấn luyện)	Chênh lệch
MLP	0,872 ± 0,002	0,872 ± 0,002	0,00
GCN	0,877 ± 0,006	0,876 ± 0,006	−0,09
GraphSAGE-mean	**0,887** ± 0,004	**0,885** ± 0,005	−0,18
GAT	0,848 ± 0,003	0,850 ± 0,003	+0,18

Ở cả bốn mô hình, độ chính xác gần như không đổi. MLP không đổi là hiển nhiên vì nó không dùng đồ thị. Điều đáng chú ý là GCN, mô hình được gọi là "chuyển dẫn", chỉ mất 0,09 điểm. Lý do: GCN cài đặt theo (6.3) học các ma trận $\mathbf{W}^{(l)}$ *dùng chung cho mọi nút*. Lúc suy luận, chỉ cần áp dụng lại cùng phép gộp trên đồ thị mới. Tính "chuyển dẫn" trong bài báo gốc là một thiết lập thí nghiệm (đặc trưng của nút không nhãn tham gia lan truyền khi huấn luyện), không phải giới hạn của kiến trúc. GraphSAGE vẫn tốt nhất ở cả hai chế độ, và lợi thế thật sự của nó trên đồ thị rất lớn nằm ở *lấy mẫu láng giềng* để huấn luyện theo lô nhỏ. Kết quả thực tiễn cho hệ thống: một bài báo mới chỉ cần danh sách trích dẫn và vector TF-IDF là gán nhãn được bằng mô hình đã huấn luyện, không cần huấn luyện lại.

6.5.3. Chi phí tính toán

Bảng 6.7. Số tham số và thời gian huấn luyện trung bình một lần chạy (GPU RTX 3050)

Mô hình	Tham số	Bán giám sát	Có giám sát	Độ phức tạp một tầng
Hồi quy logistic	1.503	0,45 s	0,34 s	$O(nd)$
MLP	32.259	0,36 s	1,41 s	$O(ndh)$
GCN	32.259	2,01 s	3,34 s	$O(ndh + \|E\|h)$
GraphSAGE-mean	64.451	2,79 s	9,94 s	$O(ndh + \|E\|d)$
GAT (8 đầu)	33.779	3,70 s	4,11 s	$O(ndhK + \|E\|hK)$

Phép gộp láng giềng chỉ tốn $O(|E|)$ phép toán trên mỗi chiều đặc trưng, nhờ ma trận kề rất thưa (44 nghìn cạnh trên 389 triệu cặp nút có thể có). Toàn bộ đồ thị 19.717 nút

nằm gọn trong bộ nhớ GPU 4 GB, và mỗi lần huấn luyện chỉ mất vài giây (Bảng 6.7). GAT đắt hơn GCN vì phải tính hệ số chú ý cho từng cạnh và từng đầu. GraphSAGE đắt nhất ở chế độ có giám sát vì nhân ma trận trung bình láng giềng với đặc trưng 500 chiều *trước* khi giảm chiều; đổi thứ tự (giảm chiều trước rồi gộp) như GCN sẽ rẻ hơn. Suy luận của GAT triển khai trên CPU cho cả đồ thị mất dưới 1 giây và chỉ chạy một lần khi khởi động dịch vụ; sau đó mỗi truy vấn chỉ là tra bảng kết quả.

6.5.4. Phân tích lỗi và trọng số chú ý của mô hình triển khai

Bảng 6.8. Ma trận nhầm lẫn của GAT triển khai trên 3.944 bài báo giữ riêng (20%)

Thật \ Dự đoán	Thực nghiệm	Típ 1	Típ 2	Recall
Thực nghiệm (802)	**647**	100	55	0,807
Típ 1 (1.551)	83	**1.233**	235	0,795
Típ 2 (1.591)	61	66	**1.464**	0,920

Lỗi lớn nhất là 235 bài típ 1 bị đoán thành típ 2. Hai nhóm dùng chung nhiều thuật ngữ lâm sàng (*insulin*, *glucos*, *patient*, *hba1c*), và nhiều bài báo bàn về cả hai loại ĐTĐ hoặc về biến chứng chung (võng mạc, thận). Lớp típ 2 dễ nhất (recall 0,92) nhờ các từ đặc trưng như *niddm*, *risk*, *obes*. Lớp thực nghiệm chủ yếu bị nhầm thành típ 1 (100 bài): nghiên cứu ĐTĐ típ 1 thường dùng mô hình chuột NOD, nên từ vựng giao thoa với nghiên cứu trên động vật.

Theo bậc của nút, accuracy thấp nhất ở các bài có 1–2 trích dẫn (0,830, nhóm chiếm 63% tập kiểm tra), và đạt 0,87–0,89 khi bậc từ 3 trở lên. Bài báo ít liên kết nhận ít thông tin từ láng giềng, nên mô hình phải dựa gần như hoàn toàn vào đặc trưng TF-IDF của chính nó, giống MLP. Đây chính là hiệu ứng "đồ thị bù cho thiếu thông tin" ở mức từng nút.

GAT có thực sự "chú ý có chọn lọc"? Lấy hệ số chú ý α_{vu} của tầng 1 (trung bình 8 đầu), nhân với $\deg(v) + 1$ để chuẩn hóa: giá trị 1 nghĩa là chia đều như nhau cho mọi láng giềng. Trung bình trên các cạnh nối hai bài *cùng nhãn* là 0,9996, trên các cạnh nối hai bài *khác nhãn* là 1,0001. Nói cách khác, GAT đã học được một phân phối chú ý gần như đồng đều, không "hạ trọng số" các láng giềng khác lớp. Hệ số tự chú ý (khuyên) trung bình là 0,34, xấp xỉ $1/(\deg+1)$ với bậc điển hình 1–2. Đây là một phát hiện quan trọng để diễn giải kết quả. Trên đồ thị có 80% cạnh cùng nhãn và bậc thấp, chú ý học được hội tụ về gần cách gộp chuẩn hóa theo bậc của GCN. Vì vậy GAT không vượt GCN

ở Bảng 6.4, và trọng số chú ý dùng để giải thích trên trang thử nghiệm chỉ nên được đọc như một tín hiệu yếu. Chú ý phát huy tác dụng rõ hơn trên đồ thị có nhiều cạnh nhiễu hoặc dị tính (heterophily).

6.5.5. Hạn chế

Thực nghiệm của chương có bốn hạn chế. (i) Đặc trưng TF-IDF trên 500 từ gốc là biểu diễn nghèo so với mô hình ngôn ngữ y sinh hiện đại (như các mô hình BERT huấn luyện trên PubMed). Với đặc trưng giàu hơn, khoảng cách giữa MLP và GNN có thể thu hẹp thêm. (ii) Pubmed-Diabetes là đồ thị đồng tính cao. Kết luận "GCN tốt hơn MLP" không tự động đúng với đồ thị dị tính, nơi các nút kề nhau thường khác nhãn. (iii) Các nhãn MeSH được gán theo chủ đề chính của bài báo, trong khi nhiều bài thuộc cả hai loại ĐTĐ, nên một phần lỗi típ 1 ↔ típ 2 là do bản chất của nhãn. (iv) Mô hình triển khai mới chỉ phân loại các bài báo có sẵn trong đồ thị. Để gán nhãn tài liệu mới nạp vào KB, cần thêm bước trích danh sách tham khảo và tính TF-IDF theo đúng bộ từ vựng 500 từ. Thí nghiệm quy nạp ở trên cho thấy bước này khả thi mà không cần huấn luyện lại.

6.6. Triển khai

Mô hình triển khai là GAT huấn luyện theo giao thức có giám sát (accuracy 0,848 trên 20% nút giữ riêng). GAT được chọn thay vì GraphSAGE vì trọng số chú ý cho phép *giải thích* dự đoán bằng các bài báo láng giềng quan trọng nhất. Trọng số mô hình (`ch6_gat.pt`, 170 KB) và đồ thị nén (`ch6_pubmed_graph.npz`: đặc trưng float16, danh sách cạnh, mã PubMed) được nạp bởi lớp `PubmedGraphClassifier` (`src/models/deep_graph.py`). Endpoint `GET /api/lab/ch6/paper?pid=<PubMed ID>` trả về nhãn thật, nhãn dự đoán, xác suất ba lớp, các từ khóa nổi bật, và tối đa 8 bài báo láng giềng xếp theo hệ số chú ý α trung bình 8 đầu, kèm liên kết tới PubMed. Không truyền `pid` thì endpoint chọn ngẫu nhiên một bài thuộc tập kiểm tra.

Đoạn mã 6.2. Gọi endpoint Chương 6

```
curl -s "https://agent.nits.io.vn/api/lab/ch6/paper?pid=12187484"
```

6.7. Tiểu kết chương 6

Chương 6 trình bày khung truyền thông điệp thống nhất các mạng nơ-ron đồ thị, cùng ba kiến trúc với cách gộp láng giềng khác nhau: GCN (chuẩn hóa đối xứng theo bậc), GraphSAGE (tách biểu diễn bản thân và láng giềng), GAT (trọng số chú ý học

được). Trên đồ thị trích dẫn 19.717 bài báo về ĐTĐ: (1) với 60 nhãn, GCN tăng 7,8 điểm accuracy so với MLP cùng số tham số, và cao hơn hẳn việc chỉ dùng đặc trưng hoặc chỉ dùng đồ thị; (2) lợi ích của đồ thị lớn nhất khi nhãn khan hiếm, giảm dần khi có nhiều nhãn; (3) GNN nông là đủ, từ 6 tầng mô hình sụp về một lớp do làm trơn quá mức cộng với khó tối ưu. Đồ thị giúp phân loại tri thức; Chương 7 trình bày cách các tác tử dùng tri thức đó để phối hợp ra quyết định.

CHƯƠNG 7. HỆ ĐA TÁC TỬ

Sáu chương trước xây dựng sáu "chuyên gia" riêng lẻ: luật lâm sàng, mô hình học máy trên bảng, CNN cho ảnh, RNN cho chuỗi, RAG cùng đồ thị tri thức cho văn bản, GNN cho y văn. Chương này trả lời câu hỏi còn lại: làm sao để các chuyên gia đó *cùng làm việc* trong một cuộc hội thoại với người dùng, biết hỏi lại khi thiếu dữ liệu, phát hiện khi chúng mâu thuẫn, và đưa ra một kết luận an toàn có trích dẫn. Câu trả lời là một *hệ đa tác tử* (Multi-Agent System – MAS). Chương trình bày cơ sở lý thuyết về tác tử và MAS, kiến trúc của hệ thống trong báo cáo, giao thức giao tiếp và tranh biện, rồi đánh giá đầu–cuối trên 400 ca của tập kiểm tra NHANES.

7.1. Tác tử và hệ đa tác tử

7.1.1. Khái niệm tác tử

Theo Wooldridge [100], *tác tử* là một hệ thống tính toán đặt trong một môi trường, có khả năng *hành động tự chủ* trong môi trường đó để đạt mục tiêu được giao. Một tác tử thông minh có bốn tính chất [101]:

- **Tự chủ**: tự quyết định hành động mà không cần con người can thiệp từng bước.
- **Phản ứng**: nhận biết thay đổi của môi trường và phản hồi kịp thời.
- **Chủ động**: hướng tới mục tiêu, tự khởi xướng hành động.
- **Xã hội**: tương tác với tác tử khác (và con người) qua một ngôn ngữ giao tiếp.

Russell và Norvig [11] phân loại tác tử theo độ phức tạp của chương trình bên trong: phản xạ đơn giản (luật điều kiện–hành động), phản xạ có mô hình (duy trì trạng thái nội tại), dựa trên mục tiêu (lập kế hoạch), dựa trên lợi ích (tối đa hóa kỳ vọng), và tác tử học. Mỗi loại đều có mặt trong hệ thống: Safety Guardrail là tác tử phản xạ đơn giản; Orchestrator là tác tử dựa trên mục tiêu; TreatmentRLAgent là tác tử học và dựa trên lợi ích.

7.1.2. Hệ đa tác tử

MAS gồm nhiều tác tử tương tác để giải quyết bài toán vượt quá khả năng hoặc hiểu biết của từng tác tử. So với một hệ thống nguyên khối, MAS có các lợi thế sau: *module hóa* (mỗi tác tử phát triển, kiểm thử, thay thế độc lập), *song song* (các nhánh độc lập chạy cùng lúc), *chịu lỗi* (một tác tử lỗi không làm sập cả hệ), và *đa góc nhìn* (các tác tử với dữ liệu và phương pháp khác nhau có thể kiểm tra chéo lẫn nhau). Cái giá phải trả là chi phí

giao tiếp, độ phức tạp điều phối, và nguy cơ "hội chứng tập thể" khi các tác tử củng cố sai lầm của nhau.

Mô hình tổ chức. Các kiểu tổ chức phổ biến gồm:

- *Phân cấp / Supervisor*: một tác tử điều phối giao việc và tổng hợp kết quả.
- *Blackboard* [102]: các tác tử đọc và ghi vào một vùng nhớ dùng chung. Mỗi tác tử đóng góp khi trạng thái chung khớp với chuyên môn của nó.
- *Contract Net* [103]: tác tử quản lý gửi lời mời thầu, các tác tử khác đặt giá, và nhiệm vụ được giao cho bên tốt nhất.
- *Ngang hàng / tranh biện*: các tác tử bình đẳng, trao đổi lập luận tới khi đồng thuận.

Hệ thống của báo cáo kết hợp ba kiểu: *Supervisor* (Orchestrator lập kế hoạch), *Blackboard* (hồ sơ bệnh nhân `PatientProfile` dùng chung, tích lũy qua nhiều lượt chat) và *tranh biện* (ConsensusCriticAgent chất vấn các tác tử chuyên môn).

Ngôn ngữ giao tiếp. FIPA-ACL [104] chuẩn hóa thông điệp giữa các tác tử: mỗi thông điệp có người gửi, người nhận, nội dung, và một *performative* thể hiện ý định giao tiếp (theo lý thuyết hành động ngôn từ): `request`, `inform`, `agree`, `refuse`, `failure`, `propose`… Performative tách "nói gì" khỏi "muốn gì". Nhờ đó tác tử nhận biết cần trả lời, thực hiện hay chỉ ghi nhận.

7.1.3. Tác tử dựa trên mô hình ngôn ngữ lớn

Từ 2023, LLM trở thành "bộ não" phổ biến của tác tử. ReAct [105] xen kẽ suy luận và hành động (gọi công cụ). Toolformer [106] cho thấy LLM tự học cách dùng API. Các khung đa tác tử như AutoGen [107] và MetaGPT [108] để nhiều LLM với vai trò khác nhau hội thoại với nhau. Du và cộng sự [109] cho thấy *tranh biện đa tác tử* (nhiều LLM đưa câu trả lời, phê bình lẫn nhau qua nhiều vòng) cải thiện tính đúng sự thật và khả năng suy luận. Các khảo sát [110], [111] chỉ ra ba thách thức chính: ảo giác lan truyền giữa các tác tử, chi phí và độ trễ của nhiều lần gọi LLM, và khó kiểm soát hành vi khi tác tử tự quyết định luồng.

Hệ thống của báo cáo chọn hướng *lai*. LLM chỉ làm những việc nó giỏi và ít rủi ro: hiểu ngôn ngữ tự nhiên (trích chỉ số từ tin nhắn, phân loại ý định), đọc ảnh và PDF scan, viết câu trả lời từ dữ liệu có sẵn. Mọi *quyết định lâm sàng* (phân tầng nguy cơ, kích hoạt guardrail, chọn chất vấn) đều do các mô hình đã đánh giá ở các chương trước cùng luật tường minh đảm nhận. Nhờ vậy hệ thống chạy được hoàn toàn *không cần LLM* (dùng regex và mẫu câu thay thế), kết quả tái lập được, và ảo giác bị chặn ở tầng diễn đạt.

7.2. Kiến trúc hệ thống

7.2.1. Tám tác tử

Bảng 7.1. Các tác tử, thông điệp chúng xử lý và kỹ thuật bên trong

Tác tử	Nhận (`msg_type`)	Vai trò và kỹ thuật
Orchestrator	– (khởi tạo mỗi lượt)	Phân loại ý định (LLM, dự phòng luật), lập kế hoạch động, điều phối song song bằng `asyncio`
ConversationAgent	`EXTRACT_FROM_TEXT`	Bóc tách chỉ số và yếu tố nguy cơ từ tin nhắn tiếng Việt (LLM structured output; dự phòng regex)
DocumentAgent	`PARSE_DOCUMENT`	Đọc PDF, ảnh, DOCX, CSV; quy đổi đơn vị; trích chuỗi glucose cho RNN
PredictionAgent	`PREDICT_RISK`, 3 loại `CHALLENGE_*`	GBDT + MLP (Ch. 2), RNN (Ch. 4), CNN (Ch. 3); bảo vệ hoặc nhượng bộ khi bị chất vấn
KnowledgeAgent	`RETRIEVE_EVIDENCE`, `ANSWER_QUESTION`, 2 loại `CHALLENGE_*`	Đối chiếu ngưỡng ADA (Ch. 1), RAG lai + GraphRAG (Ch. 5)
ConsensusCriticAgent	`DELIBERATE`	Phát hiện mâu thuẫn, tranh biện nhiều vòng, liệt kê (+)/(−), phân tầng, Safety Guardrail
TreatmentRLAgent	`RECOMMEND_TREATMENT`	Q-learning trên MDP 24 trạng thái × 4 hành động, bị ràng buộc bởi hội đồng và chống chỉ định trong KG
ResponseAgent	`COMPOSE_*`	Viết câu trả lời chỉ từ dữ liệu của các tác tử, có trích dẫn (LLM; dự phòng mẫu câu)

7.2.2. Giao tiếp qua MessageBus

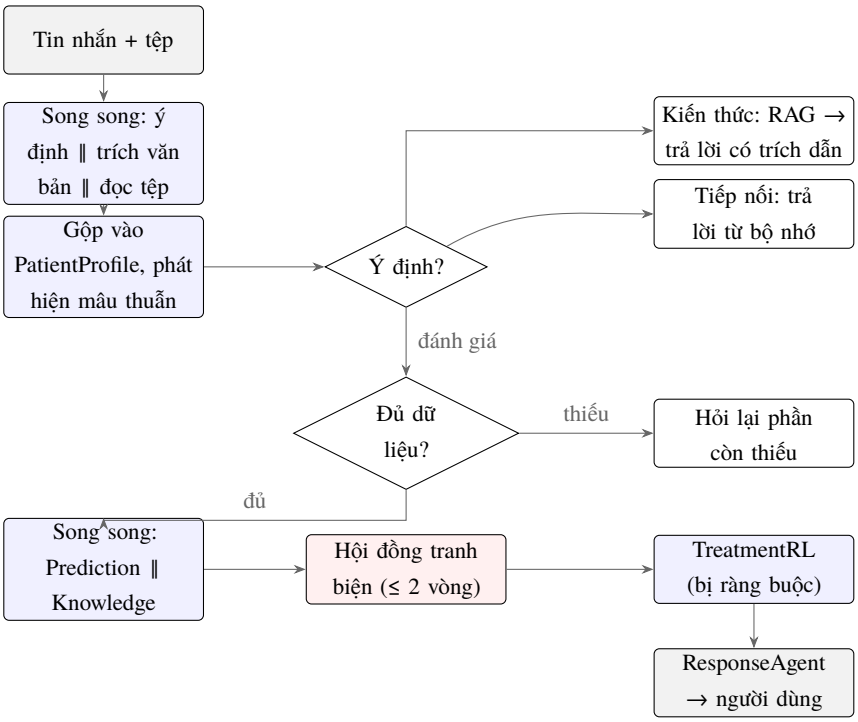
Các tác tử *không gọi hàm của nhau*. Mọi tác tử kế thừa `BaseAgent`, tự đăng ký vào `MessageBus` với tập `handles` (các loại thông điệp nó hiểu), và chỉ nhận việc qua `handle(AgentMessage)`. Thông điệp theo tinh thần FIPA-ACL với năm performative: `REQUEST`, `INFORM`, `CHALLENGE`, `RESPONSE`, `FAILURE` (Đoạn mã 7.1). Bus làm ba việc: định tuyến theo `receiver`; ghi toàn bộ hội thoại (audit trail) kèm thời gian xử lý vào bảng `agent_messages`; và *cô lập lỗi*, tức là ngoại lệ trong một tác tử được chuyển thành thông điệp `FAILURE` thay vì làm sập cả lượt.

Đoạn mã 7.1. Thông điệp và MessageBus (`src/communication/message_bus.py`, rút gọn)

```python
@dataclass
class AgentMessage:
    sender: str
    receiver: str
    performative: str            # REQUEST | INFORM | CHALLENGE | RESPONSE | FAILURE
    msg_type: str                # PREDICT_RISK, CHALLENGE_ML_VS_LABS, ...
    payload: Dict[str, Any]
    conversation_id: str = ""
    in_reply_to: Optional[str] = None
    elapsed_ms: Optional[int] = None

class MessageBus:
    async def send(self, msg: AgentMessage) -> AgentMessage:
        self._record(msg)                                  # audit trail
        agent = self.agents.get(msg.receiver)
        if agent is None or msg.msg_type not in agent.handles:
            reply = failure(msg, "NOT_UNDERSTOOD")
        else:
            try:
                reply = await agent.handle(msg)
           except Exception as exc:                        # lỗi một tác tử không làm sập hội đồng
                reply = agent.reply(msg, f"{msg.msg_type}_FAILED", {"error": str(exc)},
                                    performative="FAILURE")
        reply.elapsed_ms = ...                             # thời gian xử lý
        self._record(reply)
        return reply
```

7.2.3. Luồng xử lý một lượt chat



Hình 7.1. Luồng xử lý một lượt chat: Orchestrator rẽ nhánh theo ý định và mức đủ dữ liệu

Orchestrator lập *kế hoạch động* cho mỗi lượt (Hình 7.1). (1) *Tiếp nhận song song*: phân loại ý định, trích xuất từ văn bản và đọc từng tệp chạy đồng thời. (2) *Gộp hồ sơ*: dữ liệu từ tệp được gộp trước, lời người dùng gộp sau, nên lời đính chính trực tiếp được ưu tiên. Mỗi trường lưu nguồn, độ tin cậy, đoạn trích làm bằng chứng và cờ cảnh báo (`unit_inferred`, `fasting_unclear`). Hai nguồn khác nhau thì ghi thành mâu thuẫn. Hệ thống *không bao giờ tự điền giá trị mặc định* cho chỉ số lâm sàng. (3) *Rẽ nhánh*: câu hỏi kiến thức chỉ đi nhánh RAG; câu hỏi tiếp nối ("tại sao nguy cơ cao?") được trả lời từ bộ nhớ hai tầng (hồ sơ có cấu trúc và 10 tin nhắn gần nhất) mà không chạy lại hội đồng; thiếu tuổi, BMI hoặc xét nghiệm đường huyết thì hỏi lại. (4) Khi đủ dữ liệu, hai nhánh *Prediction* (DB → ML/DL) và *Knowledge* (KB → RAG) chạy song song, rồi đến hội đồng tranh biện, gợi ý can thiệp và viết câu trả lời.

7.2.4. Giao thức tranh biện

ConsensusCriticAgent không chỉ tổng hợp. Khi phát hiện vấn đề, nó gửi `CHALLENGE` tới tác tử liên quan, nhận `RESPONSE` và cập nhật trạng thái. Phản hồi ở vòng trước có thể mở ra câu hỏi ở vòng sau, tối đa `MAX_DEBATE_ROUNDS = 2` vòng (Bảng 7.2).

Bảng 7.2. Giao thức tranh biện của hội đồng

Vấn đề phát hiện	Chất vấn	Phản hồi và ảnh hưởng
Dữ liệu thật < 35% số đặc trưng của mô hình	Prediction · `CHALLENGE_DATA_QUALITY`	Hạ độ tin cậy, liệt kê đặc trưng còn thiếu → (−)
GBDT và MLP lệch ≥ 15 điểm %	Prediction · `CHALLENGE_MODEL_DISAGREEMENT`	Chỉ ra đặc trưng hai mô hình đánh giá khác nhau nhất; dùng trung bình → (−)
ML ≥ 30% nhưng xét nghiệm bình thường	Prediction · `CHALLENGE_ML_VS_LABS`	Giữ quan điểm "nguy cơ dài hạn" → (−) và **mở vòng 2**
(vòng 2) ML giữ quan điểm	Knowledge · `CHALLENGE_SCREENING_PLAN`	Lịch tầm soát ADA có trích dẫn → (+)
ML < 15% nhưng xét nghiệm ở ngưỡng ĐTĐ	Prediction · `CHALLENGE_ML_VS_LABS`	Nhượng bộ cho xét nghiệm → (−)
Xét nghiệm ở ngưỡng ĐTĐ	Knowledge · `CHALLENGE_VERIFY_CRITERIA`	Kiểm tra quy tắc "2 kết quả bất thường" của ADA → "đủ tiêu chí" hoặc "cần xét nghiệm lại"

Phân tầng cuối cùng lấy *xét nghiệm làm căn cứ chính*, với ML bổ sung nguy cơ dài hạn: *Rất cao* (xét nghiệm ở ngưỡng ĐTĐ), *Cao* (tiền ĐTĐ hoặc ML ≥ 30%), *Trung bình* (ML 15–30%), *Thấp*. **Safety Guardrail** luôn được ưu tiên bất kể mô hình nói gì: glucose ≥ 600 mg/dL (cấp cứu), glucose ≥ 300 hoặc HbA1c ≥ 10% (khám sớm), glucose < 54 (hạ đường huyết mức 2), nhóm nguy cơ "đỏ" (BMI ≥ 25, tăng huyết áp, và tiền sử gia đình hoặc tim mạch).

7.2.5. Tác tử gợi ý can thiệp bằng Q-learning

TreatmentRLAgent mô hình hóa việc chọn bước can thiệp như một quá trình quyết định Markov. Trạng thái là tổ hợp 4 mức glucose × 3 mức BMI × 2 mức bệnh đồng mắc, tức 24 trạng thái. Hành động gồm 4 lựa chọn: thay đổi lối sống, metformin, phối hợp thuốc có lợi ích tim–thận, và khám chuyên khoa/insulin. Q-learning [112], [113] cập

nhật giá trị hành động theo sai số thời gian:

$$Q(s,a) \leftarrow Q(s,a) + \alpha\big[r + \gamma \max_{a'} Q(s',a') - Q(s,a)\big], \qquad \alpha = 0{,}1,\ \gamma = 0{,}9, \quad (7.1)$$

với chính sách khám phá ϵ-tham lam ($\epsilon = 0{,}15$). Hàm thưởng mô phỏng mức cải thiện đường huyết trừ đi chi phí và độ xâm lấn. Điểm quan trọng về an toàn là chính sách học được *không được hành động một mình*. Nếu hội đồng chưa xác định ĐTĐ, hành động bị giới hạn ở thay đổi lối sống. Nếu guardrail khẩn được kích hoạt, hệ chuyển sang khám chuyên khoa. Nếu chẩn đoán chưa được xác nhận, metformin bị hoãn. Chống chỉ định trong đồ thị tri thức (metformin khi eGFR < 30) luôn được nêu ra. Đây là ví dụ cụ thể của nguyên tắc "tác tử học được đặt dưới sự giám sát của tri thức tường minh".

7.2.6. Bộ nhớ, nguồn gốc dữ liệu và chịu lỗi

Blackboard có nguồn gốc. `PatientProfile` là vùng nhớ dùng chung của mọi tác tử. Mỗi trường không chỉ lưu giá trị mà lưu một bản ghi `FieldValue` gồm giá trị, nguồn (`text`, `file:<tên>`, `derived`, `form`), độ tin cậy, đoạn trích làm bằng chứng và các cờ cảnh báo. Ví dụ, "đường máu 6,4" không ghi đơn vị được hiểu là 6,4 mmol/L = 115 mg/dL, kèm cờ `unit_inferred`. ConsensusCritic chuyển cờ này thành một điểm (−) yêu cầu người dùng xác nhận. Mỗi trường có miền giá trị sinh lý (tuổi 1–120, BMI 10–70, glucose 30–700 mg/dL…). Giá trị ngoài miền bị từ chối và hệ thống hỏi lại thay vì dự đoán trên dữ liệu sai. Khi hai nguồn mâu thuẫn (phiếu xét nghiệm ghi 47 tuổi, người dùng nói 49), hệ thống giữ giá trị mới nhất do người dùng nói trực tiếp và ghi nhận mâu thuẫn để hội đồng báo (−).

Bộ nhớ hai tầng. Mỗi lượt, Orchestrator nạp từ PostgreSQL (i) bộ nhớ có cấu trúc: hồ sơ tích lũy qua cả phiên, và (ii) bộ nhớ hội thoại: 10 tin nhắn gần nhất và kết quả hội chẩn gần nhất. Nhờ vậy hệ thống hiểu câu trả lời cụt ("62" sau khi bot hỏi tuổi) và câu hỏi tiếp nối ("tại sao nguy cơ của tôi cao?"). Với các câu này, hệ thống trả lời từ bộ nhớ mà không chạy lại hội đồng.

Chịu lỗi nhiều tầng. Lỗi trong một tác tử được MessageBus chuyển thành `FAILURE` và không làm sập lượt chat. LLM có chuỗi dự phòng: model chính → model dự phòng khi gặp lỗi 429/503 hoặc quá thời gian → nhánh luật và regex. Hệ thống vì thế vẫn trả lời được khi mất kết nối Internet. Các tác tử chuyên môn (Prediction, Knowledge) không phụ thuộc LLM, nên kết luận lâm sàng không thay đổi theo tình trạng của nhà cung cấp LLM.

7.2.7. So sánh với các framework đa tác tử

Bảng 7.3. So sánh cách tiếp cận của hệ thống với các framework đa tác tử dựa trên LLM

Tiêu chí	**Framework LLM (AutoGen, MetaGPT…)**	**Hệ thống của báo cáo**
Điều phối	LLM quyết định tác tử nào nói tiếp, luồng hội thoại tự do	Kế hoạch động do Orchestrator lập theo ý định và mức đủ dữ liệu; luồng xác định
Giao tiếp	Tin nhắn ngôn ngữ tự nhiên	Thông điệp có cấu trúc (performative, `msg_type`, payload JSON)
Ra quyết định	LLM suy luận	Mô hình đã đánh giá (Ch. 2–6) + luật ADA; LLM chỉ hiểu và diễn đạt ngôn ngữ
Tranh biện	Các LLM phê bình lẫn nhau	Critic phát hiện mâu thuẫn theo quy tắc, chất vấn tác tử chuyên môn
Tái lập / kiểm thử	Khó (đầu ra ngẫu nhiên)	Chạy tất định khi tắt LLM; 21 test tự động
Chi phí, độ trễ	Nhiều lần gọi LLM mỗi lượt	28 ms không LLM; 2–5 s với 2–3 lần gọi LLM
Kiểm toán	Tùy cài đặt	Mọi thông điệp lưu vào `agent_messages`

Lựa chọn tự xây dựng `BaseAgent` và `MessageBus` thay vì dùng framework có hai lý do. Thứ nhất, nó thể hiện rõ cơ chế giao tiếp và tranh biện, đúng mục tiêu của môn học. Thứ hai, trong y tế, *tính kiểm soát được* quan trọng hơn tính linh hoạt: mọi quyết định lâm sàng phải truy vết được tới một mô hình đã được đánh giá hoặc một quy tắc có trích dẫn. Cái giá là hệ thống kém linh hoạt hơn trước những yêu cầu ngoài kịch bản đã thiết kế.

7.3. Đánh giá hệ đa tác tử

7.3.1. Dữ liệu và kịch bản

Một hệ đa tác tử cần được đánh giá *đầu–cuối*: đưa vào một hồ sơ hoặc một tin nhắn, kiểm tra kết luận cuối cùng. Ca kiểm thử được lấy từ **tập kiểm tra của NHANES**, cùng phép chia 70/10/20 (`random_state=42`) với Chương 2, nên PredictionAgent chưa từng thấy các ca này. Chỉ giữ người có cả glucose lúc đói và HbA1c, rồi lấy ngẫu nhiên **400 người**: 23,0% mắc ĐTĐ (69 người đã được chẩn đoán, 23 người chưa được chẩn đoán nhưng có xét nghiệm vượt ngưỡng), 173 người tiền ĐTĐ. Hệ thống chạy *không LLM* (dùng regex và mẫu câu) để kết quả tất định và tái lập được, trừ phần so sánh trích xuất bằng Gemini. Ba kịch bản được thiết kế:

- **K1 — đủ xét nghiệm**: hồ sơ gồm 13 yếu tố nguy cơ, FPG và HbA1c, nhập dạng form. Câu hỏi: hội đồng có phân loại đường huyết đúng ngưỡng ADA và áp đúng quy tắc "2 xét nghiệm bất thường" không? Đồng thời đo các chỉ số vận hành.
- **K2 — chỉ có FPG** (giấu HbA1c), đây là tình huống phổ biến khi người dùng mới thử đường huyết. So sánh với nhãn ĐTĐ thật của NHANES ba cách ra quyết định: *tác tử Tri thức đơn lẻ* (ngưỡng FPG ≥ 126 hoặc ≥ 100), *tác tử Dự đoán đơn lẻ* (xác suất ML ≥ 30%), và *cả hội đồng* (mức Cao hoặc Rất cao). Nhãn NHANES gồm cả người đã chẩn đoán đang điều trị và người chỉ có HbA1c vượt ngưỡng, nên một mình FPG không thể phát hiện hết.
- **K3 — tin nhắn tiếng Việt tự do**: từ mỗi hồ sơ, sinh một tin nhắn theo nhiều cách viết (BMI hoặc chiều cao–cân nặng "nặng 80 ký cao 1m66", mmol/L với dấu phẩy thập phân hoặc mg/dL, câu phủ định "không hút thuốc", ngẫu nhiên lược bớt trường). Chạy toàn bộ pipeline từ văn bản; đo độ chính xác trích xuất và mức nguy cơ so với khi nhập form.

Bảng 7.4. Ví dụ tin nhắn sinh cho kịch bản K3 và giá trị đúng tương ứng

"Em là nữ, năm nay 21 tuổi, nặng 80 ký cao 1m66, vòng eo 85 cm, huyết áp đo được 103/64, gia đình không ai bị tiểu đường, glucose đói 91 mg/dL."
"Tôi 58 tuổi, nữ, chỉ số BMI là 40,5, vòng eo 115 cm, huyết áp đo được 110/79, có bị tăng huyết áp, không hút thuốc, sáng nay thử đường máu lúc chưa ăn được 7,7 mmol/l."

Đoạn mã 7.2. Đánh giá đầu–cuối một ca qua Orchestrator (`lab/ch7_agents.py`, rút gọn)

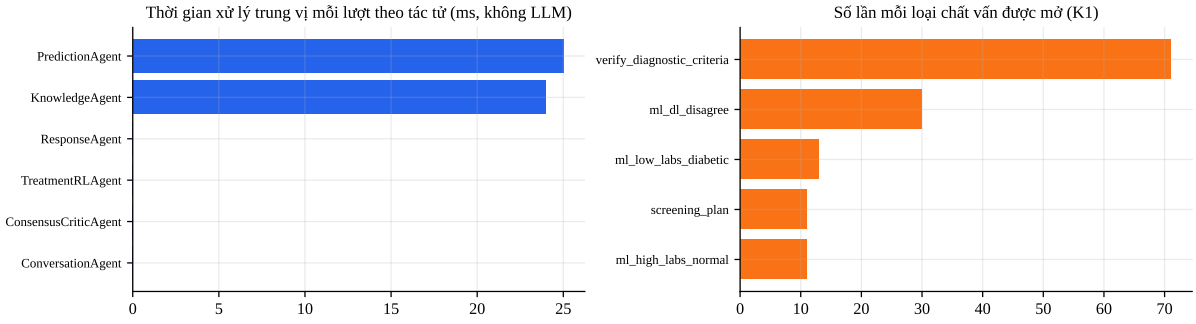
```
orc = OrchestratorAgent()                          # đăng ký 7 tác tử vào MessageBus
for row in cases.itertuples():
    out = await orc.run_turn(profile_from(row, with_a1c=True), "đánh giá nguy cơ giúp tôi",
     ↪  [])
    c = out["data"]["consensus"]
    k1.append({"truth": ada_status(row.fasting_glucose, row.hba1c), "pred":
     ↪  c["glycemic_status"],
               "confirmation": c["confirmation_required"], "rounds": c["debate_rounds"],
               "issues": [d["issue"] for d in c["debate"]]})
    out = await orc.run_turn(profile_from(row, with_a1c=False), "đánh giá nguy cơ giúp tôi",
     ↪  [])
    k2.append({"label": row.diabetes, "tier": out["data"]["consensus"]["risk_level"], ...})
```

7.3.2. K1 — tính đúng đắn và vận hành

Trong 400 ca, 1 ca có BMI 74,8 kg/m², vượt miền sinh lý hợp lệ (10–70) của hồ sơ. Hệ thống loại giá trị này và *hỏi lại* BMI thay vì chạy mô hình trên dữ liệu nghi sai. Đây là hành vi đúng theo thiết kế. 399 ca còn lại đi qua hội đồng.

Bảng 7.5. K1: tính đúng đắn và các chỉ số vận hành của hội đồng (399 ca)

Chỉ số	Kết quả
Phân loại đường huyết đúng ngưỡng ADA	**399/399 (100%)** — 137 bình thường, 191 tiền ĐTĐ, 71 ngưỡng ĐTĐ
Áp đúng quy tắc "2 xét nghiệm bất thường"	**71/71 (100%)**
Số vòng tranh biện	0 vòng: 297 ca · 1 vòng: 91 · 2 vòng: 11
Chất vấn được mở	xác minh tiêu chí ADA 71 · GBDT và MLP bất đồng 30 · ML thấp nhưng xét nghiệm ĐTĐ 13 · ML cao nhưng xét nghiệm bình thường 11 → lịch tầm soát (vòng 2) 11
Safety Guardrail	ưu tiên (nhóm nguy cơ đỏ) 77 ca · khám sớm (glucose ≥ 300 hoặc HbA1c ≥ 10%) 11 ca
Bằng chứng mỗi ca	trung bình 3,7 điểm (+), 0,21 điểm (−)
Gợi ý can thiệp (RL sau ràng buộc)	lối sống 358 · metformin 5 · phối hợp tim–thận 25 · chuyên khoa 11
Thông điệp mỗi lượt	12–18 (trung bình 12,7): 2.394 REQUEST/INFORM, 136 CHALLENGE/RESPONSE
Độ trễ một lượt (không LLM)	trung vị 28 ms, p95 42 ms; PredictionAgent 25 ms, KnowledgeAgent 24 ms (chạy song song)



Hình 7.2. Thời gian xử lý của từng tác tử và tần suất các loại chất vấn trong K1

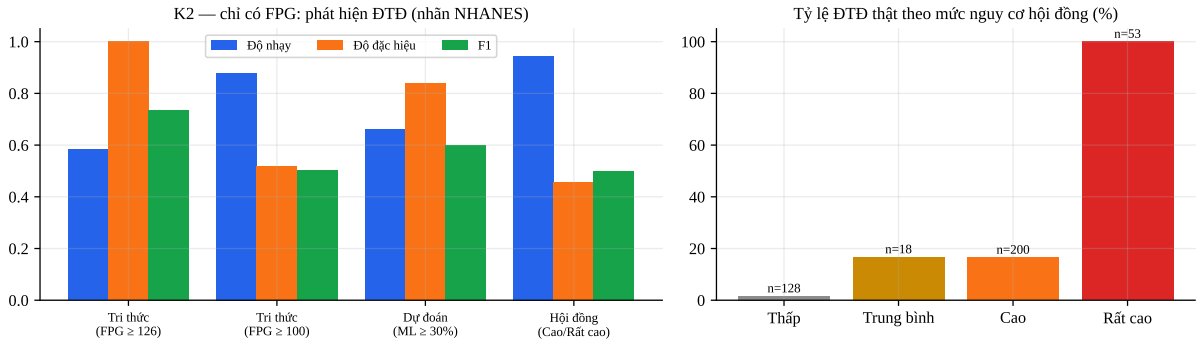
Nhận xét. (i) Lớp luật tường minh (ngưỡng ADA, quy tắc xác nhận) đúng tuyệt đối. Đây là phần "hệ chuyên gia" của Chương 1 và là nền móng an toàn của hệ thống. (ii) Tranh biện chỉ xảy ra khi có lý do: 74% ca không cần chất vấn, 3% ca cần đủ hai vòng. Chất vấn thường gặp nhất là xác minh tiêu chí chẩn đoán (71 ca có xét nghiệm ở ngưỡng ĐTĐ), tiếp theo là bất đồng giữa hai mô hình ML (30 ca, 7,5%). Tỉ lệ bất đồng

này khớp với 8% đo được ở Chương 2. (iii) Ràng buộc an toàn hoạt động đúng thiết kế: chính sách Q-learning chỉ được phép đề xuất thuốc khi hội đồng xác định ĐTĐ, nên 90% ca dừng ở can thiệp lối sống, và mọi ca kích hoạt guardrail khẩn đều được chuyển chuyên khoa. (iv) Không có LLM, một lượt đầy đủ (12–18 thông điệp, 7 tác tử) chỉ mất 28 ms. Thời gian dồn vào hai nhánh song song Prediction và Knowledge (embedding và truy vấn PostgreSQL), các tác tử luật mất dưới 1 ms.

7.3.3. K2 — giá trị của việc phối hợp nhiều tác tử

Bảng 7.6. K2 (chỉ có FPG): phát hiện ĐTĐ so với nhãn NHANES (399 ca, 91 người ĐTĐ)

Cách ra quyết định	Độ nhạy	Đặc hiệu	PPV	F1	Số ca dương
Tri thức đơn lẻ: FPG ≥ 126	0,582	**1,000**	**1,000**	**0,736**	53
Tri thức đơn lẻ: FPG ≥ 100	0,879	0,520	0,351	0,502	228
Dự đoán đơn lẻ: ML ≥ 30%	0,659	0,841	0,551	0,600	109
Hội đồng: mức Cao hoặc Rất cao	**0,945**	0,458	0,340	0,500	253



Hình 7.3. K2: độ nhạy, độ đặc hiệu, F1 của các cách ra quyết định, và tỉ lệ ĐTĐ thật theo mức nguy cơ của hội đồng

Nhận xét. Hai tác tử đơn lẻ bỏ sót những nhóm khác nhau. Ngưỡng FPG ≥ 126 chính xác tuyệt đối (PPV 1,00) nhưng chỉ bắt được 58% người ĐTĐ. **38 người ĐTĐ có FPG dưới 126**: đó là người đang điều trị tốt hoặc người chỉ có HbA1c vượt ngưỡng. ML dựa trên yếu tố nguy cơ bắt được một phần nhóm này nhưng bỏ sót người khác. Hội đồng kết hợp cả hai (xét nghiệm là căn cứ chính, ML bổ sung nguy cơ dài hạn) đạt **độ nhạy 0,945**, phát hiện thêm **33 trong 38** ca mà FPG bỏ sót. Điểm nguy cơ tổng hợp của hội đồng có AUC 0,876, cao hơn FPG đơn lẻ (0,862) và ML đơn lẻ (0,855).

Cái giá phải trả là độ đặc hiệu 0,46: 253 người được xếp mức Cao trở lên. Đây là đánh đổi *có chủ đích* của một công cụ *sàng lọc*. Mức Cao không có nghĩa là "bị bệnh"

mà là "nên làm thêm HbA1c hoặc OGTT", một xét nghiệm rẻ. Bỏ sót một người ĐTĐ thì đắt hơn nhiều. Phân tầng có ý nghĩa rõ ràng (Hình 7.3, phải): mức *Rất cao* có 100% là ĐTĐ thật (53/53); mức *Thấp* chỉ 1,6% (2/128); mức Cao và Trung bình khoảng 17%, gấp 10 lần nhóm Thấp, và đúng là nhóm cần xét nghiệm thêm. Theo F1, ngưỡng FPG ≥ 126 tốt nhất, nhưng F1 coi FN và FP nặng như nhau, điều không đúng trong sàng lọc.

7.3.4. K3 — từ tin nhắn tự do

Bảng 7.7. K3: trích xuất từ tin nhắn tiếng Việt tự do

Bộ trích xuất	Số tin nhắn	Đúng mọi trường	Cùng mức nguy cơ với form	Độ trễ trung vị
Regex tiếng Việt (không LLM)	399	100%	92,2%	0,03 s
Regex tiếng Việt (40 tin đầu)	40	100%	100%	0,02 s
Gemini structured output (40 tin đầu)	40	100%	100%	2,12 s

Cả hai bộ trích xuất đều lấy đúng 100% các trường: tuổi, giới, BMI (kể cả khi phải tính từ "nặng 80 ký cao 1m66"), vòng eo, huyết áp, glucose (kể cả quy đổi "7,7 mmol/l" → 138,6 mg/dL), và các câu phủ định. Cần đọc kết quả này một cách thận trọng. Các tin nhắn được sinh từ mẫu câu *nằm trong phạm vi* mà regex được viết để hiểu, nên 100% là cận trên lạc quan. Với cách diễn đạt tự do thật của người dùng, LLM linh hoạt hơn nhiều, và đó là lý do hệ thống ưu tiên Gemini, chỉ dùng regex làm dự phòng. Đổi lại, LLM chậm hơn khoảng 70 lần (2,1 s so với 0,03 s mỗi lượt, gồm cả phân loại ý định và viết câu trả lời).

7,8% tin nhắn cho mức nguy cơ khác với khi nhập form, dù mọi trường được trích đúng. Nguyên nhân nằm ở *thông tin đầu vào khác nhau* chứ không phải lỗi trích xuất. Tin nhắn ngẫu nhiên lược bớt vòng eo, huyết áp hoặc tiền sử gia đình (như người dùng thật thường làm), và việc làm tròn khi đổi sang mmol/L hay chiều cao–cân nặng làm thay đổi giá trị chút ít. Khi đó xác suất ML thay đổi và ca nằm gần ranh giới 15% hoặc 30% đổi mức. Trường hợp này cho thấy hai cơ chế của hệ thống có ích: ConsensusCritic báo (–) "dữ liệu thiếu" khi độ đầy đủ thấp, và ResponseAgent gợi ý người dùng bổ sung các trường còn thiếu.

7.3.5. Ví dụ một lượt hội chẩn có tranh biện hai vòng

Một lượt chat thật trên hệ thống đã triển khai (có Gemini) với tin nhắn: "Tôi 58 tuổi nữ, BMI 29, vòng eo 95 cm, bị tăng huyết áp, mỡ máu cao, mẹ bị tiểu đường, ít vận động, đường huyết lúc đói 92 mg/dL." Đây là ca điển hình gây mâu thuẫn: xét nghiệm

bình thường nhưng yếu tố nguy cơ *dày đặc*. Bảng 7.8 là toàn bộ luồng thông điệp mà MessageBus ghi lại.

Bảng 7.8. Luồng 16 thông điệp của một lượt hội chẩn thật (thời gian xử lý ghi ở thông điệp trả lời)

#	Gửi	Nhận	Performative	Loại thông điệp	ms
1	Orchestrator	ConversationAgent	REQUEST	EXTRACT_FROM_TEXT	
2	ConversationAgent	Orchestrator	INFORM	PATIENT_DATA_EXTRACTED	1.931
3	Orchestrator	PredictionAgent	REQUEST	PREDICT_RISK	
4	Orchestrator	KnowledgeAgent	REQUEST	RETRIEVE_EVIDENCE	
5	PredictionAgent	Orchestrator	INFORM	RISK_PREDICTION	130
6	KnowledgeAgent	Orchestrator	INFORM	CLINICAL_EVIDENCE	303
7	Orchestrator	ConsensusCritic	REQUEST	DELIBERATE	
8	ConsensusCritic	PredictionAgent	**CHALLENGE**	CHALLENGE_ML_VS_LABS	
9	PredictionAgent	ConsensusCritic	RESPONSE	CHALLENGE_RESPONSE	0
10	ConsensusCritic	KnowledgeAgent	**CHALLENGE**	CHALLENGE_SCREENING_PLAN	
11	KnowledgeAgent	ConsensusCritic	RESPONSE	CHALLENGE_RESPONSE	15
12	ConsensusCritic	Orchestrator	INFORM	CONSENSUS_RESULT	15
13	Orchestrator	TreatmentRLAgent	REQUEST	RECOMMEND_TREATMENT	
14	TreatmentRLAgent	Orchestrator	INFORM	TREATMENT_SUGGESTION	0
15	Orchestrator	ResponseAgent	REQUEST	COMPOSE_ASSESSMENT	
16	ResponseAgent	Orchestrator	INFORM	RESPONSE_TEXT	2.345

Diễn biến như sau. Gemini phân loại ý định là `RISK_ASSESSMENT` và ConversationAgent trích xuất đủ các trường (thông điệp 1–2). Hai nhánh chạy song song (3–6). PredictionAgent cho xác suất 40% (GBDT 34%, MLP 45%), gấp 2,02 lần trung bình quần thể. KnowledgeAgent đối chiếu FPG 92 mg/dL là *bình thường* và khớp 7 nút yếu tố nguy cơ trong KG. Critic phát hiện mâu thuẫn "ML cao nhưng xét nghiệm bình thường" và mở **vòng 1** (8–9): PredictionAgent bảo vệ quan điểm. Mô hình không dùng glucose nên phản ánh *nguy cơ tiến triển*, đồng ý xét nghiệm là căn cứ chẩn đoán, và đề nghị giữ khuyến cáo tầm soát. Phản hồi này mở **vòng 2** (10–11): KnowledgeAgent đưa ra lịch tầm soát theo ADA, có trích dẫn. Kết luận của hội đồng:

(+) Thống nhất kế hoạch theo dõi: sàng lọc lại ít nhất mỗi 3 năm, sớm hơn (mỗi năm) khi có nhiều yếu tố nguy cơ; giảm cân, vận động 150 phút/tuần.

(+) Yếu tố làm tăng nguy cơ nhiều nhất theo mô hình: tiền sử gia đình, tăng huyết áp, rối loạn mỡ máu.

(+) Đồ thị tri thức khớp 7 yếu tố nguy cơ theo ADA.

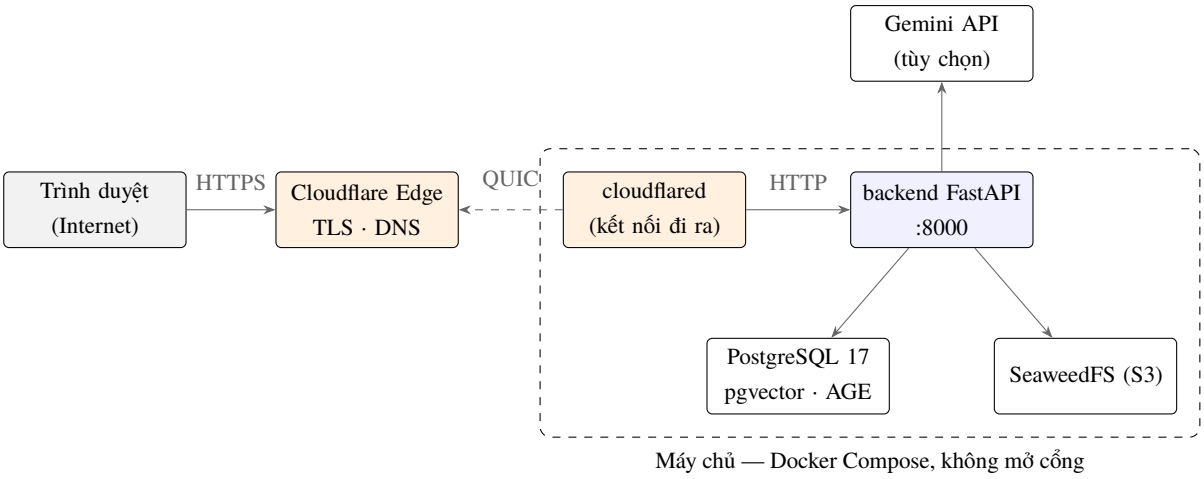
(−) ML cảnh báo nguy cơ cao nhưng xét nghiệm hiện bình thường (kèm phản biện của PredictionAgent).

Phân tầng cuối cùng là *Cao — nhiều yếu tố nguy cơ* (điểm 0,40). TreatmentRLAgent bị ràng buộc ở can thiệp lối sống vì xét nghiệm chưa ở ngưỡng ĐTĐ. ResponseAgent viết câu trả lời có các mục bằng chứng (+), lưu ý (−), khuyến cáo kèm nguồn ("Sàng lọc đái tháo đường típ 2 ở người trưởng thành châu Á và Việt Nam — Các yếu tố nguy cơ"), và lời nhắc không thay thế bác sĩ. Tổng thời gian khoảng 4,7 giây, trong đó hai lần gọi Gemini (trích xuất 1,9 s và viết câu trả lời 2,3 s) chiếm khoảng 90%. Toàn bộ phần "suy nghĩ" của hội đồng (thông điệp 3–14) chỉ mất khoảng 0,45 giây.

Ví dụ này minh họa giá trị của tranh biện: một hệ thống chỉ dùng ngưỡng sẽ trả lời "đường huyết bình thường". Một hệ thống chỉ dùng ML sẽ trả lời "nguy cơ 40%" mà không giải thích vì sao xét nghiệm lại bình thường. Hội đồng đưa cả hai góc nhìn ra, hòa giải chúng thành một kế hoạch hành động cụ thể, và cho người dùng thấy cả điểm đồng thuận lẫn điểm còn mâu thuẫn.

7.4. Triển khai

Toàn bộ hệ thống được đóng gói bằng **Docker Compose** với bốn dịch vụ (Hình 7.4). `postgres` là PostgreSQL 17 với pgvector và Apache AGE (dữ liệu NHANES, KB, đồ thị, phiên chat, audit trail). `seaweedfs` là kho đối tượng tương thích S3, lưu bệnh án tải lên. `backend` là FastAPI chạy Uvicorn; image này chứa mã nguồn, mọi mô hình của 7 chương (`data/models`, 71 MB) và mô hình nhúng tải sẵn; nó phục vụ chatbot (`/`), trang thử nghiệm (`/lab`), API (`/api/...`) và Swagger (`/docs`), với healthcheck tại `/health`. `cloudflared` tạo **Cloudflare Tunnel**: tiến trình mở kết nối QUIC *đi ra* tới mạng biên của Cloudflare, nhận request cho tên miền `agent.nits.io.vn` và chuyển về `http://localhost:8000`. Máy chủ không cần IP công khai và không mở cổng nào trên tường lửa; TLS do Cloudflare đảm nhiệm [10].



Hình 7.4. Kiến trúc triển khai qua Cloudflare Tunnel

Đoạn mã 7.3. Dịch vụ cloudflared trong `docker-compose.yml` và lệnh triển khai

```
  cloudflared:
    profiles: ["app"]
    image: cloudflare/cloudflared:2026.10.7
    command: tunnel --no-autoupdate run
    environment:
      TUNNEL_TOKEN: ${CLOUDFLARE_TUNNEL_TOKEN}   # đọc từ .env, không commit
    network_mode: host                           # để "localhost:8000" trỏ tới backend
    restart: unless-stopped
    depends_on:
      backend:
        condition: service_healthy
# Triển khai:  docker compose --profile app up -d --build
```

Bảng 7.9. Các điểm truy cập của hệ thống đã triển khai

Đường dẫn	Chức năng
`/`	Chatbot đa tác tử (React), tải bệnh án, xem chi tiết hội chẩn
`/lab`	Trang thử nghiệm 7 chương
`POST /api/chat`, `/api/chat/upload`	Một lượt hội chẩn (văn bản / tệp), trả về câu trả lời và luồng thông điệp
`POST /api/consultation`	Đánh giá trực tiếp từ form JSON
`/api/lab/ch1/pima` … `/api/lab/ch6/paper`	Mô hình của từng chương (Chương 1–6)
`GET /api/lab/summary`	Số liệu đánh giá chính của mọi chương
`/docs`, `/health`	Tài liệu OpenAPI, kiểm tra sức khỏe

Thẻ "Chương 7" trên trang `/lab` gửi một tin nhắn tới `/api/chat` và hiển thị toàn bộ luồng thông điệp giữa các tác tử: người gửi, người nhận, performative, loại thông điệp, thời gian xử lý. Các thông điệp `CHALLENGE` được tô màu riêng, nên người xem thấy trực tiếp hội đồng tranh biện. Khi có Gemini, một lượt chat đầu–cuối mất khoảng 2–5 giây; khi Gemini quá tải, hệ thống tự chuyển sang model dự phòng rồi tới nhánh luật, nên không bao giờ dừng hẳn.

7.5. Tiểu kết chương 7

Chương 7 trình bày cơ sở lý thuyết về tác tử và hệ đa tác tử (tính chất của tác tử, mô hình tổ chức Supervisor, Blackboard, Contract Net, tranh biện; ngôn ngữ FIPA-ACL; tác tử dựa trên LLM), rồi mô tả hệ thống tám tác tử giao tiếp qua MessageBus, với giao thức tranh biện nhiều vòng và tác tử Q-learning bị ràng buộc an toàn. Đánh giá đầu–cuối trên 400 ca NHANES chưa từng thấy cho thấy: (1) lớp luật lâm sàng đúng tuyệt đối, và hệ thống từ chối dữ liệu ngoài miền sinh lý thay vì dự đoán bừa; (2) phối hợp tác tử Tri thức với tác tử Dự đoán nâng độ nhạy phát hiện ĐTĐ từ 0,58 lên 0,95, bắt thêm 33/38 ca mà xét nghiệm đường huyết lúc đói bỏ sót, với phân tầng có ý nghĩa (mức Rất cao 100% là ĐTĐ, mức Thấp 1,6%); (3) tranh biện chỉ kích hoạt khi cần (26% ca), một lượt đầy đủ mất 28 ms khi không có LLM; (4) hệ thống chạy công khai qua Cloudflare Tunnel mà không cần mở cổng. Hệ đa tác tử là nơi kết quả của sáu chương trước kết hợp lại: mỗi kỹ thuật làm phần việc nó giỏi nhất, và chúng kiểm tra chéo lẫn nhau.

KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN

1. Kết quả đạt được

Báo cáo đi qua bảy chủ đề của môn Hệ thống thông minh. Mỗi chủ đề được kiểm chứng bằng một thành phần chạy thật của cùng một hệ thống: chatbot đa tác tử hỗ trợ sàng lọc đái tháo đường, triển khai tại `agent.nits.io.vn`. Bảng 7.10 tóm tắt dữ liệu, mô hình tốt nhất và kết quả của từng chương.

Bảng 7.10. Tổng hợp kết quả thực nghiệm của bảy chương

Ch.	Dữ liệu	Mô hình tốt nhất	Kết quả chính (so với đối chứng)
1	Pima (768)	ADALINE / hồi quy logistic	AUC 0,837; hệ chuyên gia 6 luật đạt 0,795; perceptron 0,749 và thất bại với XOR
2	NHANES (20.646)	Rừng ngẫu nhiên	AUC 0,823; năm mô hình mạnh chỉ cách nhau 0,006 — trần thông tin của yếu tố nguy cơ
3	RetinaMNIST (1.600)	ResNet-18 / MobileNetV3 chuyển giao	QWK 0,70–0,71 so với 0,52 (MLP) và 0,56 (CNN học từ đầu)
4	Shanghai CGM (112 người)	LSTM 2 tầng	RMSE 30 phút 13,7 mg/dL (persistence 19,0); 95,6% vùng Clarke A
5	KB ADA + MedQuAD	E5-large; lai + xếp hạng lại	Hit@1 KB tiếng Việt 0,967 (rerank), 0,933 (E5) so với 0,80 (BM25), 0,57 (MiniLM); RAG phủ 95% dữ kiện, 0% số liệu không căn cứ
6	Pubmed-Diabetes (19.717)	GCN (ít nhãn), GraphSAGE (đủ nhãn)	Accuracy 0,780 với 60 nhãn so với 0,702 (MLP cùng tham số); 0,885 có giám sát; GCN sụp từ 6 tầng
7	NHANES test (400 ca)	Hội đồng đa tác tử	Độ nhạy phát hiện ĐTĐ khi chỉ có FPG 0,945 so với 0,582 (FPG ≥ 126) và 0,659 (ML); luật ADA đúng 100%; 28 ms/lượt không LLM

Từ các thực nghiệm có thể rút ra năm bài học chung.

Thứ nhất, dữ liệu và cách đánh giá quan trọng hơn chọn thuật toán. Ở Chương 2, tám thuật toán hội tụ về cùng một mức AUC. Ở Chương 3, accuracy xếp MLP trên CNN,

trong khi QWK cho thấy điều ngược lại. Ở Chương 4, chia theo bệnh nhân thay vì theo cửa sổ là điều kiện để con số có ý nghĩa.

Thứ hai, thiên kiến quy nạp phù hợp với cấu trúc dữ liệu tạo nên khác biệt lớn nhất. Tích chập cho ảnh, hồi quy cho chuỗi, truyền thông điệp cho đồ thị: mỗi kiến trúc thắng mô hình "phẳng" ở đúng loại dữ liệu của nó, với ít tham số hơn.

Thứ ba, tri thức có sẵn rất có giá trị khi dữ liệu nhỏ. Học chuyển giao từ ImageNet nâng QWK thêm khoảng 0,15. Một bộ 6 luật y khoa đánh bại perceptron. Đồ thị trích dẫn bù cho việc chỉ có 20 nhãn mỗi lớp.

Thứ tư, mô hình đơn giản là mốc đối chứng không được bỏ qua. Persistence, hồi quy tuyến tính và BM25 đều mạnh hơn dự kiến, và chỉ những cải thiện vượt được chúng mới thực sự có ý nghĩa.

Thứ năm, trong y tế, hệ thống tốt là hệ thống biết giới hạn của mình. Hội đồng tranh biện liệt kê cả bằng chứng (+) lẫn mâu thuẫn (−). Mô hình học được đặt dưới ngưỡng xét nghiệm và Safety Guardrail. LLM chỉ diễn đạt dữ liệu đã có, và mọi câu trả lời đều kèm trích dẫn cùng lời nhắc không thay thế bác sĩ.

2. Hạn chế

- Dữ liệu huấn luyện chủ yếu đến từ Hoa Kỳ (NHANES, Pima) và Trung Quốc (Shanghai). Chưa có dữ liệu thật của người Việt Nam; nhóm người gốc Á của NHANES chỉ là phép xấp xỉ.
- RetinaMNIST có độ phân giải thấp (128×128) và chỉ 1.600 ảnh. Mô hình CNN chưa đủ tin cậy để dùng lâm sàng; Grad-CAM cho thấy mô hình đôi khi chú ý vào vùng lóa và viền ảnh.
- Knowledge Base tiếng Việt mới có 10 tài liệu tóm tắt; bộ câu hỏi đánh giá 60 câu do người viết báo cáo tự xây dựng nên có thể thiên về cách diễn đạt của chính tài liệu.
- Dự báo glucose chưa dùng thông tin bữa ăn và insulin, nên sai số tăng nhanh sau 30 phút.
- Đánh giá Chương 7 dựa trên hồ sơ có cấu trúc và tin nhắn sinh từ mẫu câu, chưa có người dùng thật.

3. Hướng phát triển

- Thu thập dữ liệu sàng lọc của người Việt (phối hợp cơ sở y tế) để hiệu chỉnh lại mô hình và ngưỡng.
- Thay RetinaMNIST bằng ảnh đáy mắt độ phân giải cao (APTOS, EyePACS), áp dụng mô hình nền tảng thị giác cho ảnh y khoa.
- Dự báo glucose đa biến (bữa ăn, insulin, vận động) bằng Transformer cho chuỗi thời gian; cảnh báo hạ đường huyết sớm.
- Mở rộng KB bằng văn bản gốc của ADA và Bộ Y tế; tự động trích bộ ba bằng LLM có kiểm duyệt; dùng GAT ở Chương 6 để phân loại tài liệu mới trước khi nạp.
- Chuẩn hóa giao tiếp công cụ theo Model Context Protocol [114]; thêm đăng nhập, phân quyền và nhật ký kiểm toán để đáp ứng yêu cầu bảo vệ dữ liệu sức khỏe.

TÀI LIỆU THAM KHẢO

[1] International Diabetes Federation, *IDF Diabetes Atlas*, 10 **edition**. Brussels: International Diabetes Federation, 2021.

[2] American Diabetes Association Professional Practice Committee, "2. diagnosis and classification of diabetes: Standards of care in diabetes—2024," *Diabetes Care*, **journvol** 47, **number** Suppl. 1, S20–S42, 2024.

[3] Bộ Y tế, *Quyết định số 5481/qđ-byt ban hành tài liệu chuyên môn "hướng dẫn chẩn đoán và điều trị đái tháo đường típ 2"*, 2020.

[4] J. W. Smith **andothers**, "Using the ADAP learning algorithm to forecast the onset of diabetes mellitus," **in***Proceedings of the Annual Symposium on Computer Application in Medical Care* 1988, **pages** 261–265.

[5] Centers for Disease Control and Prevention, *National health and nutrition examination survey (NHANES) 2011–2018*, `https://wwwn.cdc.gov/nchs/nhanes/`, 2020.

[6] J. Yang **andothers**, "MedMNIST v2 – a large-scale lightweight benchmark for 2D and 3D biomedical image classification," *Scientific Data*, **journvol** 10, **page** 41, 2023.

[7] Q. Zhao **andothers**, "Chinese diabetes datasets for data-driven machine learning," *Scientific Data*, **journvol** 10, **page** 35, 2023.

[8] A. Ben Abacha **and** D. Demner-Fushman, "A question-entailment approach to question answering," *BMC Bioinformatics*, **journvol** 20, **page** 511, 2019.

[9] G. Namata, B. London, L. Getoor **and** B. Huang, "Query-driven active surveying for collective classification," **in***Workshop on Mining and Learning with Graphs (MLG)* 2012.

[10] Cloudflare, *Cloudflare tunnel documentation*, `https://developers.cloudflare.com/cloudflare-one/connections/connect-networks/`, 2025.

[11] S. Russell **and** P. Norvig, *Artificial Intelligence: A Modern Approach*, 4 **edition**. Pearson, 2021.

[12] A. M. Turing, "Computing machinery and intelligence," *Mind*, **journvol** 59, **number** 236, **pages** 433–460, 1950.

[13] W. S. McCulloch **and** W. Pitts, "A logical calculus of the ideas immanent in nervous activity," *Bulletin of Mathematical Biophysics*, **journvol** 5, **pages** 115–133, 1943.

[14] J. McCarthy, M. L. Minsky, N. Rochester **and** C. E. Shannon, "A proposal for the dartmouth summer research project on artificial intelligence," Dartmouth College, techreport, 1955.

[15] F. Rosenblatt, “The perceptron: A probabilistic model for information storage and organization in the brain,” *Psychological Review*, **journvol** 65, **number** 6, **pages** 386–408, 1958.

[16] B. Widrow **and** M. E. Hoff, “Adaptive switching circuits,” **in***IRE WESCON Convention Record* **volume** 4, 1960, **pages** 96–104.

[17] M. Minsky **and** S. Papert, *Perceptrons: An Introduction to Computational Geometry*. MIT Press, 1969.

[18] J. Lighthill, “Artificial intelligence: A general survey,” Science Research Council, UK, techreport, 1973.

[19] E. H. Shortliffe **and** B. G. Buchanan, “A model of inexact reasoning in medicine,” *Mathematical Biosciences*, **journvol** 23, **number** 3–4, **pages** 351–379, 1975.

[20] D. E. Rumelhart, G. E. Hinton **and** R. J. Williams, “Learning representations by back-propagating errors,” *Nature*, **journvol** 323, **pages** 533–536, 1986.

[21] C. Cortes **and** V. Vapnik, “Support-vector networks,” *Machine Learning*, **journvol** 20, **pages** 273–297, 1995.

[22] L. Breiman, “Random forests,” *Machine Learning*, **journvol** 45, **pages** 5–32, 2001.

[23] J. H. Friedman, “Greedy function approximation: A gradient boosting machine,” *Annals of Statistics*, **journvol** 29, **number** 5, **pages** 1189–1232, 2001.

[24] S. Hochreiter **and** J. Schmidhuber, “Long short-term memory,” *Neural Computation*, **journvol** 9, **number** 8, **pages** 1735–1780, 1997.

[25] Y. LeCun, L. Bottou, Y. Bengio **and** P. Haffner, “Gradient-based learning applied to document recognition,” *Proceedings of the IEEE*, **journvol** 86, **number** 11, **pages** 2278–2324, 1998.

[26] I. Goodfellow, Y. Bengio **and** A. Courville, *Deep Learning*. MIT Press, 2016.

[27] G. E. Hinton, S. Osindero **and** Y.-W. Teh, “A fast learning algorithm for deep belief nets,” *Neural Computation*, **journvol** 18, **number** 7, **pages** 1527–1554, 2006.

[28] J. Deng **andothers**, “ImageNet: A large-scale hierarchical image database,” **in***IEEE Conference on Computer Vision and Pattern Recognition (CVPR)* 2009.

[29] A. Krizhevsky, I. Sutskever **and** G. E. Hinton, “Imagenet classification with deep convolutional neural networks,” **in***Advances in Neural Information Processing Systems (NeurIPS)* 2012.

[30] T. B. Brown **andothers**, “Language models are few-shot learners,” **in***Advances in Neural Information Processing Systems (NeurIPS)* 2020.

[31] L. Ouyang **andothers**, “Training language models to follow instructions with human feedback,” **in***Advances in Neural Information Processing Systems (NeurIPS)* 2022.

[32] J. Jumper **andothers**, “Highly accurate protein structure prediction with AlphaFold,” *Nature*, **journvol** 596, **pages** 583–589, 2021.

[33] Thủ tướng Chính phủ, *Quyết định số 127/qđ-ttg ban hành chiến lược quốc gia về nghiên cứu, phát triển và ứng dụng trí tuệ nhân tạo đến năm 2030*, 2021.

[34] F. Pedregosa **andothers**, "Scikit-learn: Machine learning in Python," *Journal of Machine Learning Research*, **jourvol** 12, **pages** 2825–2830, 2011.

[35] D. P. Kingma **and** J. Ba, "Adam: A method for stochastic optimization," **in***International Conference on Learning Representations (ICLR)* 2015.

[36] T. Cover **and** P. Hart, "Nearest neighbor pattern classification," *IEEE Transactions on Information Theory*, **jourvol** 13, **number** 1, **pages** 21–27, 1967.

[37] G. Ke **andothers**, "LightGBM: A highly efficient gradient boosting decision tree," **in***Advances in Neural Information Processing Systems (NeurIPS)* 2017.

[38] J. Platt, "Probabilistic outputs for support vector machines and comparisons to regularized likelihood methods," *Advances in Large Margin Classifiers*, **jourvol** 10, **pages** 61–74, 1999.

[39] S. P. Lloyd, "Least squares quantization in PCM," **in***IEEE Transactions on Information Theory* **volume** 28, 1982, **pages** 129–137.

[40] P. J. Rousseeuw, "Silhouettes: A graphical aid to the interpretation and validation of cluster analysis," *Journal of Computational and Applied Mathematics*, **jourvol** 20, **pages** 53–65, 1987.

[41] W. J. Youden, "Index for rating diagnostic tests," *Cancer*, **jourvol** 3, **number** 1, **pages** 32–35, 1950.

[42] V. Gulshan **andothers**, "Development and validation of a deep learning algorithm for detection of diabetic retinopathy in retinal fundus photographs," *JAMA*, **jourvol** 316, **number** 22, **pages** 2402–2410, 2016.

[43] D. H. Hubel **and** T. N. Wiesel, "Receptive fields, binocular interaction and functional architecture in the cat's visual cortex," *Journal of Physiology*, **jourvol** 160, **pages** 106–154, 1962.

[44] A. Howard **andothers**, "Searching for MobileNetV3," **in***IEEE/CVF International Conference on Computer Vision (ICCV)* 2019.

[45] S. Ioffe **and** C. Szegedy, "Batch normalization: Accelerating deep network training by reducing internal covariate shift," **in***International Conference on Machine Learning (ICML)* 2015.

[46] N. Srivastava **andothers**, "Dropout: A simple way to prevent neural networks from overfitting," *Journal of Machine Learning Research*, **jourvol** 15, **pages** 1929–1958, 2014.

[47] C. Szegedy **andothers**, "Rethinking the inception architecture for computer vision," **in***IEEE Conference on Computer Vision and Pattern Recognition (CVPR)* 2016.

[48] I. Loshchilov **and** F. Hutter, "Decoupled weight decay regularization," **in***International Conference on Learning Representations (ICLR)* 2019.

[49] K. Simonyan **and** A. Zisserman, "Very deep convolutional networks for large-scale image recognition," **in***International Conference on Learning Representations (ICLR)* 2015.

[50] K. He, X. Zhang, S. Ren **and** J. Sun, "Deep residual learning for image recognition," **in***IEEE Conference on Computer Vision and Pattern Recognition (CVPR)* 2016, **pages** 770–778.

[51] R. R. Selvaraju **andothers**, "Grad-CAM: Visual explanations from deep networks via gradient-based localization," **in***IEEE International Conference on Computer Vision (ICCV)* 2017.

[52] R. Liu **andothers**, "DeepDRiD: Diabetic retinopathy—grading and image quality estimation challenge," *Patterns*, **jourvol** 3, **number** 6, **page** 100 512, 2022.

[53] C. P. Wilkinson **andothers**, "Proposed international clinical diabetic retinopathy and diabetic macular edema disease severity scales," *Ophthalmology*, **jourvol** 110, **number** 9, **pages** 1677–1682, 2003.

[54] J. Cohen, "Weighted kappa: Nominal scale agreement with provision for scaled disagreement or partial credit," *Psychological Bulletin*, **jourvol** 70, **number** 4, **pages** 213–220, 1968.

[55] J. L. Elman, "Finding structure in time," *Cognitive Science*, **jourvol** 14, **number** 2, **pages** 179–211, 1990.

[56] P. J. Werbos, "Backpropagation through time: What it does and how to do it," *Proceedings of the IEEE*, **jourvol** 78, **number** 10, **pages** 1550–1560, 1990.

[57] Y. Bengio, P. Simard **and** P. Frasconi, "Learning long-term dependencies with gradient descent is difficult," *IEEE Transactions on Neural Networks*, **jourvol** 5, **number** 2, **pages** 157–166, 1994.

[58] R. Pascanu, T. Mikolov **and** Y. Bengio, "On the difficulty of training recurrent neural networks," **in***International Conference on Machine Learning (ICML)* 2013.

[59] K. Cho **andothers**, "Learning phrase representations using RNN encoder–decoder for statistical machine translation," **in***Conference on Empirical Methods in Natural Language Processing (EMNLP)* 2014.

[60] I. Sutskever, O. Vinyals **and** Q. V. Le, "Sequence to sequence learning with neural networks," **in***Advances in Neural Information Processing Systems (NeurIPS)* 2014.

[61] D. Bahdanau, K. Cho **and** Y. Bengio, "Neural machine translation by jointly learning to align and translate," **in***International Conference on Learning Representations (ICLR)* 2015.

[62] A. Vaswani **andothers**, "Attention is all you need," **in***Advances in Neural Information Processing Systems (NeurIPS)* 2017.

[63] T. Battelino **andothers**, "Clinical targets for continuous glucose monitoring data interpretation: Recommendations from the international consensus on time in range," *Diabetes Care*, **journvol** 42, **number** 8, **pages** 1593–1603, 2019.

[64] W. L. Clarke **andothers**, "Evaluating clinical accuracy of systems for self-monitoring of blood glucose," *Diabetes Care*, **journvol** 10, **number** 5, **pages** 622–628, 1987.

[65] K. Li **andothers**, "GluNet: A deep learning framework for accurate glucose forecasting," *IEEE Journal of Biomedical and Health Informatics*, **journvol** 24, **number** 2, **pages** 414–423, 2020.

[66] Z. Ji **andothers**, "Survey of hallucination in natural language generation," **in***ACM Computing Surveys* **volume** 55, 2023.

[67] A. Hogan **andothers**, "Knowledge graphs," *ACM Computing Surveys*, **journvol** 54, **number** 4, **pages** 1–37, 2021.

[68] A. Singhal, *Introducing the knowledge graph: Things, not strings*, Google Official Blog, 2012.

[69] Apache Software Foundation, *Apache AGE: Graph database extension for PostgreSQL*, `https://age.apache.org`, 2024.

[70] A. Bordes **andothers**, "Translating embeddings for modeling multi-relational data," **in***Advances in Neural Information Processing Systems (NeurIPS)* 2013.

[71] P. Lewis **andothers**, "Retrieval-augmented generation for knowledge-intensive NLP tasks," **in***Advances in Neural Information Processing Systems (NeurIPS)* 2020.

[72] Y. Gao **andothers**, *Retrieval-augmented generation for large language models: A survey*, arXiv:2312.10997, 2023.

[73] S. Robertson **and** H. Zaragoza, "The probabilistic relevance framework: BM25 and beyond," *Foundations and Trends in Information Retrieval*, **journvol** 3, **number** 4, **pages** 333–389, 2009.

[74] N. Reimers **and** I. Gurevych, "Sentence-BERT: Sentence embeddings using siamese BERT-networks," **in***EMNLP-IJCNLP* 2019.

[75] V. Karpukhin **andothers**, "Dense passage retrieval for open-domain question answering," **in***EMNLP* 2020.

[76] N. Reimers **and** I. Gurevych, "Making monolingual sentence embeddings multilingual using knowledge distillation," **in***EMNLP* 2020.

[77] Y. A. Malkov **and** D. A. Yashunin, "Efficient and robust approximate nearest neighbor search using hierarchical navigable small world graphs," *IEEE Transactions on Pattern Analysis and Machine Intelligence*, **journvol** 42, **number** 4, **pages** 824–836, 2020.

[78] A. Kane, *Pgvector: Open-source vector similarity search for Postgres*, `https://github.com/pgvector/pgvector`, 2024.

[79] G. V. Cormack, C. L. A. Clarke **and** S. Büttcher, "Reciprocal rank fusion outperforms condorcet and individual rank learning methods," **in***ACM SIGIR* 2009, **pages** 758–759.

[80] R. Nogueira **and** K. Cho, "Passage re-ranking with BERT," **in***arXiv:1901.04085* 2019.

[81] D. Edge **andothers**, *From local to global: A graph RAG approach to query-focused summarization*, arXiv:2404.16130, 2024.

[82] S. Es, J. James, L. Espinosa-Anke **and** S. Schockaert, "RAGAS: Automated evaluation of retrieval augmented generation," **in***EACL (System Demonstrations)* 2024.

[83] L. Wang **andothers**, *Multilingual E5 text embeddings: A technical report*, arXiv:2402.05672, 2024.

[84] J. Chen **andothers**, *BGE M3-Embedding: Multi-lingual, multi-functionality, multi-granularity text embeddings through self-knowledge distillation*, arXiv:2402.03216, 2024.

[85] M. Zitnik, M. Agrawal **and** J. Leskovec, "Modeling polypharmacy side effects with graph convolutional networks," **in***Bioinformatics* **volume** 34, 2018, **pages** i457–i466.

[86] D. Zhou **andothers**, "Learning with local and global consistency," **in***Advances in Neural Information Processing Systems (NeurIPS)* 2004.

[87] M. McPherson, L. Smith-Lovin **and** J. M. Cook, "Birds of a feather: Homophily in social networks," *Annual Review of Sociology*, **jourvol** 27, **pages** 415–444, 2001.

[88] B. Perozzi, R. Al-Rfou **and** S. Skiena, "DeepWalk: Online learning of social representations," **in***ACM SIGKDD* 2014.

[89] J. Gilmer **andothers**, "Neural message passing for quantum chemistry," **in***International Conference on Machine Learning (ICML)* 2017.

[90] Z. Wu **andothers**, "A comprehensive survey on graph neural networks," *IEEE Transactions on Neural Networks and Learning Systems*, **jourvol** 32, **number** 1, **pages** 4–24, 2021.

[91] T. N. Kipf **and** M. Welling, "Semi-supervised classification with graph convolutional networks," **in***International Conference on Learning Representations (ICLR)* 2017.

[92] W. L. Hamilton, R. Ying **and** J. Leskovec, "Inductive representation learning on large graphs," **in***Advances in Neural Information Processing Systems (NeurIPS)* 2017.

[93] P. Veličković **andothers**, "Graph attention networks," **in***International Conference on Learning Representations (ICLR)* 2018.

[94] Q. Li, Z. Han **and** X.-M. Wu, "Deeper insights into graph convolutional networks for semi-supervised learning," **in***AAAI Conference on Artificial Intelligence* 2018.

[95] D. Chen **andothers**, "Measuring and relieving the over-smoothing problem for graph neural networks from the topological view," **in***AAAI Conference on Artificial Intelligence* 2020.

[96] K. Xu, W. Hu, J. Leskovec **and** S. Jegelka, "How powerful are graph neural networks?" **in***International Conference on Learning Representations (ICLR)* 2019.

[97] P. Sen **andothers**, "Collective classification in network data," *AI Magazine*, **journvol** 29, **number** 3, **pages** 93–106, 2008.

[98] M. Fey **and** J. E. Lenssen, "Fast graph representation learning with PyTorch Geometric," **in***ICLR Workshop on Representation Learning on Graphs and Manifolds* 2019.

[99] Z. Yang, W. W. Cohen **and** R. Salakhutdinov, "Revisiting semi-supervised learning with graph embeddings," **in***International Conference on Machine Learning (ICML)* 2016.

[100] M. Wooldridge, *An Introduction to MultiAgent Systems*, 2 **edition**. Wiley, 2009.

[101] N. R. Jennings, K. Sycara **and** M. Wooldridge, "A roadmap of agent research and development," *Autonomous Agents and Multi-Agent Systems*, **journvol** 1, **pages** 7–38, 1998.

[102] B. Hayes-Roth, "A blackboard architecture for control," *Artificial Intelligence*, **journvol** 26, **number** 3, **pages** 251–321, 1985.

[103] R. G. Smith, "The contract net protocol: High-level communication and control in a distributed problem solver," *IEEE Transactions on Computers*, **journvol** C-29, **number** 12, **pages** 1104–1113, 1980.

[104] Foundation for Intelligent Physical Agents, *FIPA ACL message structure specification (SC00061G)*, 2002.

[105] S. Yao **andothers**, "ReAct: Synergizing reasoning and acting in language models," **in***International Conference on Learning Representations (ICLR)* 2023.

[106] T. Schick **andothers**, "Toolformer: Language models can teach themselves to use tools," **in***Advances in Neural Information Processing Systems (NeurIPS)* 2023.

[107] Q. Wu **andothers**, *AutoGen: Enabling next-gen LLM applications via multi-agent conversation*, arXiv:2308.08155, 2023.

[108] S. Hong **andothers**, "MetaGPT: Meta programming for a multi-agent collaborative framework," **in***International Conference on Learning Representations (ICLR)* 2024.

[109] Y. Du **andothers**, "Improving factuality and reasoning in language models through multiagent debate," **in***International Conference on Machine Learning (ICML)* 2024.

[110] L. Wang **andothers**, "A survey on large language model based autonomous agents," *Frontiers of Computer Science*, **jourvol** 18, **page** 186 345, 2024.

[111] T. Guo **andothers**, "Large language model based multi-agents: A survey of progress and challenges," **in***International Joint Conference on Artificial Intelligence (IJCAI)* 2024.

[112] C. J. C. H. Watkins **and** P. Dayan, "Q-learning," *Machine Learning*, **jourvol** 8, **pages** 279–292, 1992.

[113] R. S. Sutton **and** A. G. Barto, *Reinforcement Learning: An Introduction*, 2 **edition**. MIT Press, 2018.

[114] Anthropic, *Model context protocol specification*, `https://modelcontextprotocol.io`, 2024.